

Tcl

The Complete Guide - From Beginner to Expert

The Substitution Engine · Strings, Lists and Dicts · Regular Expressions ·
Procedures · Namespaces · Packages · TclOO · Coroutines · Channels ·
The Event Loop · Networking · Expect · Tk · EDA Flows · Testing · Threads ·
Safe Interpreters · The C API

Contents	38 chapters in 7 parts, plus 3 appendices
Level	From your first puts to embedding Tcl in C and writing production EDA flows
Style	Learn the substitution rules exactly, then everything else follows from them
Every example runs	All code in this book was executed with tclsh 8.6.14 ; the output shown on green cards is the output it actually produced
Version	Tcl 8.6 throughout, with 8.5 differences and 8.7/9.0 changes called out where they matter
Assumed	Programming experience in some language. No prior Tcl.

How To Use This Book

Tcl is a small language with an unusual property: its entire syntax fits on one page, and **everything else is a library**. There are no keywords, no statements, no operators outside `expr` - only commands and a set of substitution rules that decide how a line of text becomes a command and its arguments. Learn those rules exactly, once, and the rest of the language stops containing surprises.

That is how this book is organised. Chapter 2 gives the eleven rules in full; every later chapter is either a library of commands or a consequence of those rules. Where other books say 'this is a special case', this one shows which rule produced it.

The seven parts

Part	Chapters	What you get out of it
I - The Language	1-5	What Tcl is and where it is used; the substitution rules in full; 'everything is a string' and what it costs; variables, scope and <code>expr</code> ; control flow.
II - Data	6-10	Strings, lists, dictionaries and arrays - what each really is, when to use which, and how to keep them fast - plus regular expressions.
III - Structure	11-15	Procedures and their return codes; <code>upvar</code> and <code>uplevel</code> ; error handling with <code>try</code> ; namespaces and ensembles; packages, modules and Tcllib.
IV - Advanced Language	16-20	TclOO and the alternatives; dynamic evaluation done safely; coroutines and generators; introspection and metaprogramming.
V - The Outside World	21-26	Files and channels; buffering, encodings and transformations; running processes; the event loop; sockets and HTTP; Expect.
VI - Applied Tcl	27-33	Tk GUIs; Tcl in EDA flows; testing with <code>tcldtest</code> ; debugging and performance; threads; safe interpreters and security; embedding and extending Tcl in C.
VII - Practice	34-38	Idioms and style; the anatomy of a production script; versions, deployment and the ecosystem; a cookbook; and one complete project built stage by stage.

Reading paths

- **Complete beginner:** 1 -> 2 -> 4 -> 5 -> 6 -> 7 -> 11, typing every example into `tcldsh` as you go. Chapter 2 repays being read twice.
- **You already program and want Tcl quickly:** 2, 3, 7, 8, 11, 13, 14 - the rules, the data model, procedures, errors and namespaces. Then use Part V as reference.
- **Handed a large EDA script to maintain:** 2, 3, 5, 7, 11, 13, then 28 (EDA flows) and 34 (idioms). Chapter 3 explains most of the confusing behaviour you will meet.
- **Automating interactive tools:** 21-23, then 24 (event loop) and 26 (Expect).
- **Writing a GUI:** 24 (the event loop is the prerequisite), then 27.
- **Making slow Tcl fast:** 3 (shimmering), 7 (list representations), 30 (measurement), 34 (idioms).
- **Embedding Tcl in an application:** 18, 32 (safe interpreters) and 33 (the C API).

Conventions

Style	Meaning
A grey card	A script, exactly as you would type or save it
A green card	The output that script produced when it was run under <code>tcldsh</code> 8.6.14 while this book was written
<code>%</code>	The interactive <code>tcldsh</code> prompt, when interaction is being shown
Coloured boxes	KEY IDEA = the one thing to remember. MATH = a derivation or a measurement. INTUITION = the mental picture. PRACTICAL TIP = what to do. PITFALL = a mistake that reaches production. EXPERT CORNER = depth you can skip on a first read.

KEEP AN INTERPRETER OPEN WHILE YOU READ

Run `tclsh` in another window and type the examples. Tcl's interactive loop is unusually informative - it prints the result of every command, so the substitution rules become visible rather than theoretical. Nothing in this book takes longer than a few seconds to try, and the ones that surprise you are the ones worth typing twice.

Table of Contents

PART I - The Language	11
1. What Tcl Is, and Where You Will Meet It	12
1.1 Your first script, and a useful one	12
1.2 Where Tcl is actually used	12
1.3 Running Tcl	13
1.4 The whole language, in one paragraph	13
1.5 What Tcl is good at, honestly	14
2. The Substitution Rules: All of Tcl's Syntax	15
2.1 How a command is evaluated	15
2.2 The rules, with examples	15
2.3 Rules 4 and 5: the two grouping characters	15
2.4 Rule 11: substitution never creates word boundaries	16
2.5 Rule 9: where a comment may start	16
2.6 Rule 10: substitution happens once	17
2.7 Reading a line like the parser does	17
3. Everything Is a String (and What That Costs)	18
3.1 The principle, demonstrated	18
3.2 A list is a string with quoting rules	18
3.3 Under the surface: Tcl_Obj and dual representations	18
3.4 Shimmering, and why it costs	19
3.5 Numbers are strings that look like numbers	20
3.6 Binary data	20
4. Variables, Scope and Expressions	22
4.1 Setting, reading and removing	22
4.2 Scope: local by default, and only two levels of it	22
4.3 expr: the expression sub-language	23
4.4 Comparison: the two families of operator	23
4.5 Precision and formatting	24
5. Control Flow Is Just Commands	25
5.1 if, and why the brace must be on the same line	25
5.2 Loops	25
5.3 switch: pattern dispatch	26
5.4 Writing your own control structure	26
5.5 Timing a loop	27
PART II - Data	28
6. Strings	29
6.1 The essential subcommands	29

6.2 Classification with <code>`string is`</code>	30
6.3 <code>format</code> and <code>scan</code>	30
6.4 Splitting, joining and building	30
6.5 Two ways to build text	31
6.6 Unicode and encodings, briefly	31
7. Lists	32
7.1 Building and indexing	32
7.2 Nested lists	32
7.3 Searching	32
7.4 Sorting	33
7.5 Destructuring, stacks and queues	33
7.6 <code>`concat`</code> versus <code>`list`</code> - the difference that bites	34
7.7 Performance notes	34
8. Dictionaries	36
8.1 The basics	36
8.2 Iteration	36
8.3 Nested dicts: structured configuration	36
8.4 The accumulation commands	37
8.5 Grouping and inverting - the two patterns you will write most	37
8.6 Dict or array?	38
9. Arrays	39
9.1 Basics	39
9.2 Converting between arrays, dicts and lists	39
9.3 Traces: the reason arrays still exist	39
9.4 Special arrays you already have	40
10. Regular Expressions	42
10.1 The two commands	42
10.2 The options that do the real work	42
10.3 Substitution with <code>regsub</code>	43
10.4 Greediness, and the trap it sets	43
10.5 Syntax reference	43
10.6 Long patterns: use <code>-expanded</code>	44
10.7 Practical parsing	44

PART III - Structure **46**

11. Procedures	47
11.1 Arguments: defaults and variadics	47
11.2 Return values and the implicit result	47
11.3 Introspection: procedures are inspectable	48
11.4 Recursion, and its limit	48
11.5 Anonymous procedures with <code>`apply`</code>	48

11.6 Renaming and deleting commands	48
12. upvar, uplevel and Traces	50
12.1 upvar: taking an argument by name	50
12.2 uplevel: running a script in another frame	50
12.3 Knowing where you are: `info level`	51
12.4 Traces on execution	51
13. Errors, Exceptions and Cleanup	53
13.1 catch: the low-level primitive	53
13.2 try: the modern form	53
13.3 Raising errors well	54
13.4 Stack traces	54
13.5 Patterns that keep scripts honest	55
14. Namespaces and Ensembles	56
14.1 Creating and using	56
14.2 Names, resolution and navigation	56
14.3 Ensembles: building `string`-style commands	57
14.4 Namespaces and state	57
15. Packages, Modules and Libraries	59
15.1 The three commands	59
15.2 A complete package, on disk	59
15.3 Versions	59
15.4 Tcl Modules: the simpler alternative	60
15.5 What is already available	60
15.6 Structuring your own library	60
PART IV - Advanced Language	62
16. TclOO: Object Orientation	63
16.1 A class, an object, a lifetime	63
16.2 Inheritance and `next`	63
16.3 Mixins and filters: composition without inheritance	64
16.4 Introspection	64
16.5 Class-level methods and instance registries	64
17. The Other Object Systems	66
18. Dynamic Evaluation, Done Safely	67
18.1 `eval` and the expansion operator	67
18.2 Why string-built commands break	67
18.3 `subst` and its switches	68
18.4 Aliases, `tailcall` and friends	68
19. Coroutines	69
19.1 The mechanism	69
19.2 Generators over data	69

19.3 State machines without a state variable	69
19.4 Where coroutines really pay: event-driven code	70
20. Introspection and Metaprogramming	72
20.1 The <code>`info`</code> command	72
20.2 Generating procedures	72
20.3 Wrapping and memoising existing commands	73
20.4 The <code>`unknown`</code> handler	73
20.5 Knowing where you are: <code>`info frame`</code>	74
PART V - The Outside World	75
21. Files and Directories	76
21.1 Reading and writing	76
21.2 The <code>`file`</code> command	76
21.3 Finding files	77
21.4 Writing files safely	77
22. Channels: Buffering, Encodings and Transforms	79
22.1 <code>`fconfigure`</code> : the five options that matter	79
22.2 Binary channels	79
22.3 Positioning	79
22.4 Stacked transformations	80
22.5 The <code>`chan`</code> command and reflected channels	80
23. Running Other Programs	81
23.1 <code>`exec`</code> basics	81
23.2 Redirection and pipelines	81
23.3 Arguments, quoting and safety	82
23.4 The environment and the process	82
24. The Event Loop	84
24.1 The model	84
24.2 Timers: <code>`after`</code>	84
24.3 Channel events: <code>`fileevent`</code>	84
24.4 Repeating work without blocking	85
24.5 <code>`vwait`</code> , <code>`update`</code> and the traps	85
24.6 Background errors	86
25. Networking: Sockets, HTTP and TLS	87
25.1 A server and a client	87
25.2 The <code>`http`</code> package	87
25.3 TLS	88
25.4 Writing a robust network client	88
26. Expect: Automating Interactive Programs	90
26.1 The three commands	90
26.2 A complete, working session	90

26.3 Patterns and the <code>`expect_out`</code> array	91
26.4 What Expect is used for	91

PART VI - Applied Tcl **93**

27. Tk: Graphical Interfaces	94
27.1 The three ideas	94
27.2 Geometry managers	94
27.3 A complete small application	94
27.4 Keeping the interface alive	95
27.5 The canvas, and where Tk still wins	95
28. Tcl in EDA Flows	97
28.1 What the tool adds, and what it does not	97
28.2 The shape of a robust flow script	97
28.3 The failures that cost tapeout schedules	98
28.4 Making flow scripts testable	98
29. Testing with <code>tcptest</code>	100
29.1 A test file that runs	100
29.2 The options	101
29.3 Structuring a suite	101
29.4 Testing code that talks to the world	101
30. Debugging and Performance	103
30.1 Debugging without a debugger	103
30.2 Measuring, not guessing	103
30.3 What is actually slow in Tcl	104
30.4 A profiling harness in ten lines	104
31. Threads	106
31.1 The model	106
31.2 Creating threads and sending work	106
31.3 A thread pool: the pattern to reach for	107
31.4 When threads are the right answer	107
32. Safe Interpreters and Security	108
32.1 What a safe interpreter cannot do	108
32.2 Giving capability back, deliberately	108
32.3 Resource limits	108
32.4 The threats in ordinary scripts	109
33. Embedding and Extending Tcl in C	110
33.1 Embedding: a scriptable application in thirty lines	110
33.2 Extending: a command written in C	110
33.3 The API in one page	111
33.4 Easier routes to the same place	112

PART VII - Practice	113
34. Idioms, Style and Pitfalls	114
34.1 The rules that prevent most bugs	114
34.2 Idioms worth knowing	114
34.3 Pitfalls, collected	115
34.4 Style	115
35. The Anatomy of a Production Script	116
35.1 The skeleton	116
35.2 What each part buys	117
36. Versions, Portability and Deployment	119
36.1 What arrived when	119
36.2 Portability	119
36.3 Shipping a Tcl program	120
37. Cookbook	122
37.1 Files	122
37.2 Parse a key = value configuration file	122
37.3 Count and rank	122
37.4 Unique, preserving order	122
37.5 Walk a directory tree	123
37.6 Retry with exponential backoff	123
37.7 A formatted table	123
37.8 Temporary files, dates and randomness	124
37.9 Ten one-liners worth memorising	124
38. A Complete Project, Stage by Stage	126
38.1 Stage 0 - The requirement	126
38.2 Stage 1 - The layout	126
38.3 Stage 2 - The package	126
38.4 Stage 3 - The tests, written alongside	127
38.5 Stage 4 - The command-line program	127
38.6 Stage 5 - Running it	128
38.7 What each chapter contributed	129
38.8 Where to take it next	129
APPENDICES	130
Appendix A. Command Reference by Task	132
The eleven rules, condensed	132
Variables and scope	132
Strings	132
Lists	133
Dicts and arrays	133
Control, procedures and errors	133

Files, channels and processes	134
Events, networking and concurrency	134
Regular expressions	134
Introspection and measurement	135
Appendix B. Glossary	136
Appendix C. Roadmap, Projects and Resources	138
A 10-week study plan	138
Projects worth building	138
Where to look things up	138
How to keep improving	138

PART I

The Language

What Tcl is and why it is still everywhere; the eleven substitution rules that constitute its entire syntax; what 'everything is a string' really means and what it costs; variables, scope and expressions; and control flow that is not syntax but ordinary commands.

1. What Tcl Is, and Where You Will Meet It

Tcl - the **Tool Command Language**, pronounced 'tickle' - was designed by John Ousterhout in 1988 to solve a specific problem: every research tool in his lab had invented its own bad little command language. His answer was to write one good one, designed from the start to be **embedded** in an application and extended with that application's own commands.

That origin explains everything about the language. It is why Tcl looks like a shell rather than like C, why its syntax is tiny and its command set is large, and why - three decades later - it is the scripting language inside chip design tools, network switches, test equipment and build systems, quietly running flows that no one advertises.

1.1 Your first script, and a useful one

```
puts "Hello, world!"
```

```
Hello, world!
```

Now something that does real work. Note that there is no syntax here at all - **set**, **foreach**, **file** and **puts** are all just commands:

```
set files [list report.log build.log test.log]

foreach f $files {
    set base [file rootname $f]
    puts "[format %-12s $f] -> ${base}.txt"
}
```

```
report.log  -> report.txt
build.log   -> build.txt
test.log    -> test.txt
```

1.2 Where Tcl is actually used

Domain	What Tcl does there	Why it won
Electronic design automation	The command language of Synopsys, Cadence, Xilinx/AMD Vivado, Intel Quartus, Siemens tools: constraints, flows, reports	Easy to embed in a C++ tool; every vendor exposes its engine as Tcl commands, so one language drives the whole flow
Network equipment	Cisco IOS tclsh on the device, configuration and monitoring scripts	Small footprint, embeddable, no runtime to install
Test and automation	Expect drives interactive programs - ssh, telnet, installers, lab instruments	Expect exists only in Tcl, and remains the best tool for the job
Build and CI systems	Older but very large flows, plus tools such as DejaGnu (used to test GCC and GDB)	Stability: scripts written in 1998 still run
GUIs	Tk - the toolkit that also became the GUI layer of Python, Perl and Ruby	Fewer lines per widget than anything else
Embedded applications	A scriptable console inside a C or C++ program	Two function calls to embed; a clean C API (Chapter 33)
Databases and servers	SQLite began as a Tcl extension; AOLserver ran large web sites	Historical, but the code is still around

WHY A TCL BOOK IS WORTH READING IN AN EDA CAREER

If you write constraints, run synthesis, or maintain a tapeout flow, you write Tcl - whether or not you have ever learned it. The scripts that break at 2 a.m. before a deadline are usually broken by two things this book fixes in Chapters 2 and 3: **quoting that is nearly right**, and **a list that is really a string**. Nothing else in the language is as expensive to misunderstand.

1.3 Running Tcl

Way	Command	When
Interactive	<code>tclsh</code>	Experiments; it prints the result of every command, which is the fastest way to learn
Script file	<code>tclsh myscript.tcl arg1 arg2</code>	Normal use
Executable script	<code>#!/usr/bin/env tclsh</code> as the first line	Unix tools
Inside a tool	The tool's own shell: <code>dc_shell</code> , <code>vivado -mode tcl</code> , <code>quartus_sh -s</code>	EDA work - the interpreter is the same, plus the vendor's commands
With a GUI	<code>wish</code>	Tk applications (Chapter 27)
Embedded in C	<code>Tcl_CreateInterp()</code>	Chapter 33

```
# Interactively, tclsh prints every result - no puts needed.
% expr {3 * 7}
21
% set greeting "hi"
hi
% string toupper $greeting
HI
% info patchlevel
8.6.14
```

The interactive prompt is a read-eval-print loop over the same interpreter your scripts use, so anything you can type there works in a file, and vice versa.

1.4 The whole language, in one paragraph

THE MENTAL MODEL TO CARRY THROUGH THE BOOK

A Tcl script is a sequence of **commands**. A command is a list of **words** separated by whitespace and terminated by a newline or a semicolon. The **first word is the command name**, and the rest are its arguments - passed as strings, always. Before the command runs, the interpreter performs a fixed set of substitutions on the words (`$variable`, `[command]`, `\escape`), controlled by two grouping characters: **double quotes**, which allow substitution, and **braces**, which forbid it. That is the entire syntax. `if`, `while`, `proc` and `expr` are not keywords - they are commands that receive their arguments under exactly these rules.

Because everything is a command, the language is unusually uniform. `set` is a command that happens to be built in; so is `puts`; and so is any command you write yourself or that your EDA tool provides. The interpreter cannot tell them apart, and neither can a caller:

```
puts [info commands set]
puts [llength [info commands]]

# The command name is just the first word - it can come from a
# variable, which is why Tcl needs no separate "call" syntax.
set cmd puts
$cmd "called through a variable"
```

```
set
100
called through a variable
```

A bare `tclsh` starts with about a hundred commands; a Vivado or Design Compiler shell starts with several thousand, and they are indistinguishable from the built-in ones.

1.5 What Tcl is good at, honestly

Strength	Weakness
Embedding: two C calls put a full interpreter in your application	Performance: 10-100x slower than C for computation; use it as glue, not as a numerical engine
Stability: scripts from the 1990s still run	A small standard library compared with Python; you reach for Tcllib or write it
Uniform syntax with no keywords, so extension commands feel native	The same uniformity means errors are found at run time, not parse time
Excellent string, list and regular-expression handling	The 'everything is a string' model has performance traps (Chapter 3)
First-class event loop, channels and Expect	Quoting rules must be learned exactly; near-miss quoting is the classic bug
Safe interpreters: genuinely sandboxed execution (Chapter 32)	Object orientation arrived late (TclOO in 8.6) and older code uses several competing systems

Exercises

- 1 Start `tclsh` and find its version with `info patchlevel`. Then run `info commands` and count how many commands you have.
- 2 Write the three-line file-renaming script above and run it.
- 3 In your EDA tool of choice, run `info commands` and compare the count with plain `tclsh`. The difference is the tool's API.
- 4 Find one Tcl script already in use where you work, and note every construct in it you cannot yet explain. Chapters 2 and 3 will eliminate most of the list.

2. The Substitution Rules: All of Tcl's Syntax

Tcl's syntax is defined by eleven rules, traditionally called the **dodekalogue** (from the manual page [Tcl](#)). They are short enough to read in five minutes and complete enough that nothing else in the language is syntax. This chapter presents them with a worked example for each, because every confusing Tcl behaviour you will ever meet is one of these rules applied exactly.

2.1 How a command is evaluated

```

one line of script
|
1. SPLIT INTO WORDS      whitespace separates; quotes and braces
                        |      group; newline or ';' ends the command
                        v
2. SUBSTITUTE            $var, [command], \escape - ONCE, left
                        |      to right, in each word
                        v
3. INVOKE                first word = command name,
                        remaining words = arguments (strings)

```

The command itself decides what its arguments MEAN. 'expr' treats its argument as an expression; 'if' treats one as a script; 'puts' treats it as text. The parser has no idea and does not care.

Two ideas do most of the work: substitution happens **once**, and the **command decides the meaning** of its arguments.

2.2 The rules, with examples

#	Rule	In practice
1	Commands. A script is commands separated by newlines or semicolons.	<code>set a 1; set b 2</code> is two commands
2	Evaluation. Words are split, substituted once, then the first word names the command.	The core loop above
3	Words. Whitespace separates words unless quoted or braced.	<code>set x a b</code> is an error: three arguments to <code>set</code>
4	Double quotes. Group with substitution allowed; whitespace and newlines inside are literal.	<code>puts "x is \$x"</code>
5	Braces. Group with no substitution at all; nested braces must balance.	<code>proc p {} { ... }, expr {\$a+\$b}</code>
6	Command substitution. <code>[...]</code> runs the enclosed script and replaces the brackets with its result.	<code>set n [length \$items]</code>
7	Variable substitution. <code>\$name</code> , <code>\$name(index)</code> , <code>\${name}</code> .	<code>\${base}.txt</code> where a plain <code>\$base.txt</code> would be wrong
8	Backslash substitution. <code>\n</code> , <code>\t</code> , <code>\\$</code> , <code>\\</code> , <code>\<newline></code> folds a line.	Escaping a literal dollar or bracket
9	Comments. <code>#</code> starts a comment only where a command could start .	The rule that surprises everyone - see below
10	Order of substitution. Each word is scanned once, left to right; results are not rescanned.	Why <code>\$</code> inside a variable's value is not substituted again
11	Substitution and word boundaries. Substitution never creates new word boundaries.	A variable containing spaces stays one argument

2.3 Rules 4 and 5: the two grouping characters

```
set name world

puts "Hello, $name!"           ;# quotes: substitution happens
puts {Hello, $name!}          ;# braces: nothing is substituted
puts "1 + 2 = [expr {1 + 2}]" ;# brackets work inside quotes

set x 5
puts "\$x is $x, \${x}th is ${x}th"
```

```
Hello, world!
Hello, $name!
1 + 2 = 3
$x is 5, ${x}th is 5th
```

CHOOSING QUOTES OR BRACES, EVERY TIME

Use braces when you want the text passed through untouched: expression arguments (`expr {...}`), script arguments (`if`, `while`, `proc` bodies), and any literal containing `$` or `[`. **Use double quotes** when you want substitution: building a message, a filename, a command line. When in doubt, brace it - the failure mode of over-bracing is a visible error, while the failure mode of over-quoting is a script that works until someone's variable contains a bracket.

2.4 Rule 11: substitution never creates word boundaries

This is the rule that makes Tcl safe from most injection-style surprises, and the one that confuses people arriving from shell scripting.

```
set a "two words"
puts [llength $a]      ;# $a is ONE word to llength...
puts [llength [list a b c]]

# ...but the value it carries is a two-element list, because
# llength interprets the string it received as a list.
```

```
2
3
```

The distinction matters: `$a` was passed as a **single argument** whose **contents** happened to look like a two-element list. In a shell, `$a` would have been split into two arguments before the command saw it. In Tcl it never is - the splitting, when it happens, is done by the command, not by the parser.

2.5 Rule 9: where a comment may start

```
set y 1 # this is NOT a comment - it is arguments to set
```

```
wrong # args: should be "set varName ?newValue?"
```

`#` is only a comment at the start of a command. After `set y 1` the parser is still inside that command, so `#` is just another word.

```
set y 1 ;# a semicolon ends the command, so this IS a comment
puts $y
```

```
1
```

THE COMMENT-INSIDE-BRACES TRAP

Because braces suppress substitution but **not** brace counting, a comment containing an unbalanced brace breaks the enclosing script: `if ${x} { # don't do this` - the apostrophe is harmless, but a `{` or `}` in a comment inside a braced body is counted by the parser and will unbalance it. This produces the famously unhelpful error `missing close-brace`, pointing at a line far from the real problem. Keep braces balanced even inside comments.

2.6 Rule 10: substitution happens once

```
set x 3
set body {puts "x is $x"}    ;# braces: $x is NOT substituted now
eval $body                  ;# ...it is substituted when eval runs it
set x 4
eval $body
```

```
x is 3
x is 4
```

This single-pass property is what lets Tcl treat scripts as data safely. The body was stored as literal text; each `eval` substituted it once, at that moment, in the caller's scope. Chapter 18 builds on this for dynamic evaluation, and Chapter 32 explains why the same property makes injection possible when you build command strings from untrusted text.

2.7 Reading a line like the parser does

```
puts "outer [string toupper "inner [string reverse abc]"]"
```

```
outer INNER CBA
```

```
puts "outer [string toupper "inner [string reverse abc]"]"
|_____|
  one word (rule 4): quotes group it
    |_____|
      a command substitution (rule 6) - the quotes
      INSIDE brackets are a fresh parse, so they
      do not terminate the outer quoted word
        |_____|
          another nested command
```

Evaluation order: innermost bracket first (abc -> cba),
then the toupper (inner cba -> INNER CBA), then puts.

Brackets start a completely new parse, which is why nested quotes inside them are unambiguous. This is rule 6 and it is the reason Tcl needs no special nesting syntax.

Exercises

- 1 Predict the output of each of these, then run them: `puts {$x}`, `puts "$x"`, `puts [set x]`, with `set x 7` first.
- 2 Explain why `expr $a + $b` and `expr {$a + $b}` differ when `$a` is `"1 + 1"`. Then run both.
- 3 Write a line where a variable's value contains a space and prove it is passed as one argument.
- 4 Break a script with an unbalanced brace inside a comment, and note how far the reported error line is from the real one.
- 5 Write out, word by word, what the parser does with `set msg "Found [llength $files] files in $dir"`.

3. Everything Is a String (and What That Costs)

Tcl's second organising principle is that **every value is a string**. A number is a string. A list is a string. A script is a string. A procedure body is a string. This is what makes the language so uniform - and it is the source of both its most elegant idioms and its worst performance surprises.

3.1 The principle, demonstrated

```
set n 42
puts "len=[string length $n] plus1=[expr {$n+1}] rev=[string reverse $n]"

set l {a b c}
puts "as string: '$l'  string length: [string length $l]  llength: [llength $l]"
```

```
len=2 plus1=43 rev=24
as string: 'a b c'  string length: 5  llength: 3
```

The same value is a number to **expr**, a string to **string length**, and a list to **llength**. Nothing about the value decides this - the **command** does.

VALUES HAVE NO TYPE; COMMANDS HAVE EXPECTATIONS

Ask 'what type is this variable?' and Tcl has no answer. Ask 'can this string be interpreted as a list / an integer / a script?' and it does. This is why there are no declarations, why any value can be passed to any command, and why errors such as **expected integer but got "3.5"** appear at the moment of use rather than at assignment.

3.2 A list is a string with quoting rules

```
set weird [list "two words" {a b} "quote\"inside"]
puts $weird
puts [llength $weird]
puts [lindex $weird 0]
```

```
{two words} {a b} quote\"inside
3
two words
```

list did not build a data structure in the way another language would: it built a **string** in which each element is quoted so that reading it back as a list recovers the originals exactly. That round-trip property is the whole definition of a Tcl list, and it is why **list** is the correct way to build one:

```
# WRONG: string concatenation loses the element boundaries
set bad "item1 item2"
append bad " two words"
puts "bad: [llength $bad] elements"

# RIGHT: list operations preserve them
set good [list item1 item2]
lappend good "two words"
puts "good: [llength $good] elements, third = [lindex $good 2]"
```

```
bad: 4 elements
good: 3 elements, third = two words
```

This is the single most common defect in real Tcl scripts, and it appears the day a filename or a design name contains a space.

3.3 Under the surface: Tcl_Obj and dual representations

If every value were literally re-parsed as a string on every access, Tcl would be unusably slow. Since version 8.0 it has not been: each value is a **Tcl_Obj** that can hold two representations at once - the string, and a cached internal form appropriate to how it was last used.

```

Tcl_Obj
+-----+
| refCount          |
| string rep:  "1 2 3 4 5"      | <- may be absent (invalid)
| internal rep: list of 5 objs  | <- or int, double, dict,
+-----+          bytecode, regexp, ...

set s "1 2 3 4 5"      -> string rep only
llength $s             -> builds the LIST internal rep, caches it
string length $s       -> needs the string rep; it still exists
lindex $s 2            -> reuses the cached list rep: fast

```

Both representations are kept in step: whichever is generated, the other stays valid until something changes the value.

3.4 Shimmering, and why it costs

Trouble begins when a value is used alternately as two different types in a loop. Each switch discards one representation and regenerates the other - this is called **shimmering**, and it turns an O(1) operation into an O(n) one.

```

# Pathological: forces a list <-> string conversion every iteration
set items [lrepeat 20000 element]
set t1 [time {
    foreach x $items {
        set n [llength $items]      ;# uses the list rep
        set c [string length $items] ;# forces the string rep
    }
} 1]

# Same work, no shimmering: compute each once, outside the loop
set t2 [time {
    set n [llength $items]
    set c [string length $items]
    foreach x $items { }
} 1]
puts "shimmering: $t1"
puts "hoisted   : $t2"

```

```

shimmering: 37113776 microseconds per iteration
hoisted   : 7214 microseconds per iteration

```

Measured on tclsh 8.6.14: 37.1 seconds against 7.2 milliseconds - **5,145x slower**. Nothing about the algorithm changed; only how many times the value was converted between representations. Each iteration regenerated a 20,000-element list from a string and then a string from the list.

FOUR RULES THAT AVOID SHIMMERING

(1) Decide what a value is and use it consistently - do not `string length` a list or `llength` a sentence. **(2)** Build lists with `list/lappend`, never with `append` or string concatenation. **(3)** Hoist conversions out of loops. **(4)** When you must have both views, keep two variables. Chapter 30 shows how to detect shimmering with `tcl::unsupported::representation`.

```

set v {1 2 3}
puts [tcl::unsupported::representation $v]
llength $v
puts [tcl::unsupported::representation $v]
expr {[string length $v]}
puts [tcl::unsupported::representation $v]

```

```

value is a pure string with a refcount of 4, object pointer at
0x55c1aa1a9550, string representation "1 2 3"
value is a list with a refcount of 4, object pointer at
0x55c1aa1a9550, internal representation 0x55c1aa1c74d0:(nil),
string representation "1 2 3"
value is a string with a refcount of 4, object pointer at
0x55c1aa1a9550, internal representation 0x55c1aa1e4270:(nil),
string representation "1 2 3"

```

Wrapped for width; the pointers differ per run. Watch the type change: **pure string** -> **list** -> **string**. That third line is a shimmer happening in front of you - **string length** threw away the list representation the **llength** had just built.

3.5 Numbers are strings that look like numbers

```

set a 007
puts "$a [expr {$a+0}] [string length $a]"

puts [expr {2**64}]      ;# arbitrary precision integers (8.5+)
puts [expr {10 / 3}]     ;# integer division
puts [expr {-7 / 2}]     ;# FLOOR division, not truncation
puts [expr {-7 % 2}]     ;# remainder has the divisor's sign

```

```

007 7 3
18446744073709551616
3
-4
1

```

TWO NUMERIC TRAPS

Leading zeros: in Tcl 8.4 and earlier, **007** was **octal** in an expression, so `expr {010 + 1}` gave 9. From 8.5 the rule changed to decimal, with **0o10** for octal - but old scripts and old habits persist, and dates or version numbers with leading zeros are the usual victims. **Division:** `-7 / 2` is **-4**, not **-3** - Tcl rounds towards negative infinity, unlike C. If you want truncation, use `int(-7.0/2)`.

3.6 Binary data

Strings hold characters, not bytes, so binary data needs a byte-array representation: **binary format** and **binary scan** build and parse it, and a channel must be configured **-translation binary** (Chapter 22). Getting this wrong silently corrupts data through encoding conversion, which is the third classic EIAS trap after quoting and shimmering.

```

set bytes [binary format H* "deadbeef"]
puts [string length $bytes]
binary scan $bytes H* hex
puts $hex
binary scan $bytes cu* asdecimal
puts $asdecimal

```

```

4
deadbeef
222 173 190 239

```

Exercises

- 1 Take a value and use it as a number, a list and a string in succession, printing `tcl::unsupported::representation` after each.
- 2 Reproduce the shimmering benchmark on your machine and record the ratio.
- 3 Build a list of filenames containing spaces with **append** and with **lappend**, and compare **llength**.
- 4 Predict, then check, the value of `expr {-9 / 4}` and `expr {-9 % 4}`.

- 5 Read 16 bytes of a binary file and print them as hex with `binary scan`.

4. Variables, Scope and Expressions

Variables in Tcl are names in a table, created on first assignment and holding strings. Expressions are a separate sub-language, evaluated only by the `expr` command and by the handful of commands that call it for you. Keeping those two ideas apart explains most of what follows.

4.1 Setting, reading and removing

```
set x 10                ;# create or assign; returns the value
puts $x
puts [set x]           ;# the long form of $x
set name "port_a"
set ${name}_width 8    ;# a computed variable NAME
puts $port_a_width

puts [info exists x]
unset x
puts [info exists x]
unset -nocomplain x    ;# no error if it is already gone
```

```
10
10
8
1
0
```

Command	Purpose
<code>set v ?value?</code>	Assign, or read with one argument
<code>unset ?-nocomplain? v...</code>	Remove variables
<code>info exists v</code>	Test existence without an error
<code>append v str...</code>	Efficient in-place string append
<code>incr v ?n?</code>	Integer increment in place - much faster than <code>set v [expr {\$v+1}]</code>
<code>lappend v item...</code>	Efficient in-place list append
<code>global, variable, upvar</code>	Scope control - below and in Chapter 12

4.2 Scope: local by default, and only two levels of it

Inside a procedure, variables are **local** unless you say otherwise, and there is no lexical nesting: a procedure sees its own locals, and anything else must be reached explicitly.

```
set counter 0          ;# a global

proc bump_wrong {} {
    set counter 99      ;# creates a LOCAL, hides nothing
}
proc bump_right {} {
    global counter      ;# link the global name into this scope
    incr counter
}
bump_wrong ; puts "after wrong: $counter"
bump_right ; puts "after right: $counter"
```

```
after wrong: 0
after right: 1
```

THE THREE WAYS TO REACH ANOTHER SCOPE

``global name`` links a global variable into the current procedure. ``variable name`` does the same for a namespace variable (Chapter 14) and is what you use inside packages. ``upvar 1 name local`` links a variable belonging to the **caller**, which is how a procedure takes an argument by reference (Chapter 12). There is no other scope in Tcl - no closures over enclosing procedures, no block scope.

4.3 expr: the expression sub-language

`expr` is a command whose single argument is an expression in a C-like syntax. Everything numeric in Tcl goes through it - including the condition of `if` and `while`, which call `expr` internally.

```
puts [expr {17 % 5}]
puts [expr {2**64}]
set n 17
puts [expr {$n > 10 ? "big" : "small"}]
puts [expr {sqrt(2)}]
puts [expr {int(3.7)}] ;# truncate
puts [expr {round(3.5)}]
puts [expr {max(3,9,2)}]
puts [expr {5 in {3 4 5}}]
puts [expr {1.0/3}]
```

```
2
18446744073709551616
big
1.4142135623730951
3
4
9
1
0.3333333333333333
```

ALWAYS BRACE THE EXPRESSION - IT IS NOT A STYLE PREFERENCE

`expr {$a + $b}` passes the expression to `expr` untouched, and the bytecode compiler compiles it once. `expr $a + $b` substitutes first, so `expr` receives an already-built string that it must parse afresh every time - **typically 10-30x slower** - and, far worse, if `$a` contains `1 + 1` or `[exec rm -rf /]` that text is now **part of the expression** and will be evaluated. Bracing is both the fast form and the safe form. The same applies to `if`, `while` and `for` conditions.

4.4 Comparison: the two families of operator

Operator	Compares	"10" op "10.0"	Use for
<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code>	Numerically if both look numeric, else as strings	true	Numbers
<code>eq</code> <code>ne</code>	Always as strings	false	Strings
<code>in</code> <code>ni</code>	List membership	-	Membership tests
<code>string equal</code> , <code>string compare</code>	Strings, with options	-	Case-insensitive or length-limited comparison

```
puts [expr {"10" == "10.0"}]
puts [expr {"10" eq "10.0"}]
puts [expr {"abc" < "abd"}]
puts [expr {0.1 + 0.2 == 0.3}]      ;# floating point!
puts [expr {abs(0.1 + 0.2 - 0.3) < 1e-9}]
```

```
1
0
1
0
1
```

The fourth line is the usual floating-point surprise, not a Tcl quirk: 0.1 has no exact binary representation in any language.

USE `EQ` FOR STRINGS, ALWAYS

`if {$mode == "fast"}` works until the day someone passes `0`, which compares numerically-equal to the string `"fast"` in some edge cases involving empty strings and leading zeros. `eq` has one meaning and no surprises. The same reasoning makes `ne` preferable to `!=` for strings, and `in` clearer than a chain of `||`.

4.5 Precision and formatting

```
puts [expr {1.0/3}]
puts [format %.4f [expr {1.0/3}]]
puts [format "%-10s %5.2f %3d" name 3.14159 42]
puts [format %08.3f 3.14159]
puts [format "%x %o %b %e" 255 255 255 12345.678]
```

```
0.3333333333333333
0.3333
name      3.14  42
0003.142
ff 377 11111111 1.234568e+04
```

Since Tcl 8.5, `tcl_precision` defaults to 0, meaning **the shortest decimal string that reads back as the same double**. That is why `1.0/3` prints 17 digits: fewer would not round-trip. Never set `tcl_precision` to 12 to make output prettier - use `format` at the point of display and keep full precision in the computation.

Exercises

- 1 Write a procedure that increments a global counter, and prove that omitting `global` silently does nothing.
- 2 Time `expr {$a+$b}` against `expr $a + $b` in a million-iteration loop and report the ratio.
- 3 Construct a case where `==` and `eq` disagree, and explain it.
- 4 Format a table of three columns with `format` so the columns line up regardless of value width.
- 5 Predict and then check: `expr {7/2}`, `expr {7/2.0}`, `expr {int(-7/2)}`, `expr {-7/2}`.

5. Control Flow Is Just Commands

`if`, `while`, `for`, `foreach` and `switch` are commands that take scripts as arguments. Understanding that is not pedantry: it explains why the braces must be where they are, why `else` cannot start a new line, and why you can write your own control structures (Chapter 12).

5.1 if, and why the brace must be on the same line

```
set n 7
if {$n > 10} {
    puts "big"
} elseif {$n > 5} {
    puts "medium"
} else {
    puts "small"
}
```

```
medium
```

THE ONE LAYOUT RULE YOU CANNOT BREAK

`if` is a command whose arguments are the condition and the body. Putting the opening brace on the next line ends the command at the newline, leaving `if` with too few arguments - **wrong # args: no expression after "if" argument**. The same applies to `} else {`: both braces must be on that line, because the command has not ended. This is not a style convention borrowed from C; it is forced by rule 1 of Chapter 2.

5.2 Loops

```
for {set i 0} {$i < 3} {incr i} { puts "i=$i" }

foreach {k v} {a 1 b 2 c 3} { puts "$k -> $v" }    ;# two at a time

foreach x {1 2 3} y {a b c} { puts "$x$y" }        ;# two lists at once

puts [lmap n {1 2 3 4} { expr {$n * $n} }]          ;# collect results (8.6+)
```

```
i=0
i=1
i=2
a -> 1
b -> 2
c -> 3
1a
2b
3c
1 4 9 16
```

Form	Use
<code>foreach var list body</code>	The normal loop - prefer it to <code>for</code> over a list
<code>foreach {a b} list body</code>	Consume several elements per iteration; ideal for key/value lists
<code>foreach a listA b listB body</code>	Walk two lists in parallel; the shorter is padded with empty strings
<code>lmap</code>	Like <code>foreach</code> but collects each body's result into a list (Tcl 8.6+)
<code>while {cond} body</code>	When the iteration count is not known

Form	Use
<code>for {init} {cond} {step} body</code>	C-style; mostly for numeric ranges
<code>break / continue</code>	Exit the innermost loop / skip to the next iteration

5.3 switch: pattern dispatch

```
foreach f {alpha.v beta.sv gamma.vhd delta.txt} {
    switch -glob -- $f {
        *.v      -                ;# fall through to the next body
        *.sv     { puts "$f: verilog" }
        *.vhd    { puts "$f: vhdL" }
        default  { puts "$f: ignored" }
    }
}
```

```
alpha.v: verilog
beta.sv: verilog
gamma.vhd: vhdL
delta.txt: ignored
```

Option	Matching
<code>-exact</code>	String equality (the default)
<code>-glob</code>	Shell-style wildcards: *, ?, [a-z]
<code>-regexp</code>	Full regular expressions (Chapter 10)
<code>--</code>	End of options - always use it before the value, or a value beginning with - is read as an option
A body of -	Fall through to the next pattern's body
<code>default</code>	Matches anything; must be last

`switch` also **returns** the value of the body it ran, so it can be used as an expression rather than a statement:

```
set severity [switch -- $level {
    error {expr {3}}
    warning {expr {2}}
    info {expr {1}}
    default {expr {0}}
}]
```

5.4 Writing your own control structure

Because a body is just a string containing a script, and `uplevel` runs a script in the caller's scope (Chapter 12), a new control structure is an ordinary procedure:

```
proc repeat {n body} {
    for {set i 0} {$i < $n} {incr i} {
        uplevel 1 $body ;# run it in the CALLER's scope
    }
}

set count 0
repeat 5 { incr count }
puts $count
```

5

`count` is the caller's variable, incremented by a loop the caller did not write. This is the mechanism behind every domain-specific command an EDA tool gives you.

5.5 Timing a loop

```
puts [time {expr {1+1}} 1000]
```

```
0.084 microseconds per iteration
```

`time script ?count?` is built in, and is the measurement tool used throughout this book (Chapter 30).

Exercises

- 1 Move an opening brace to the next line and read the error message carefully; explain it in terms of Chapter 2's rules.
- 2 Rewrite a `for` loop over a list as a `foreach`, then as an `lmap`.
- 3 Use `foreach {k v}` to iterate a key/value list, then do the same with a `dict for` (Chapter 8) and compare.
- 4 Write a `switch -glob` that classifies the file extensions used in your own project.
- 5 Implement `do ... while` as a procedure using `uplevel`.

PART II

Data

The four containers Tcl actually has - strings, lists, dictionaries and arrays - what each one really is underneath, which to reach for and when, how to keep each of them fast, and the regular-expression engine that ties them to the text you are usually processing.

6. Strings

Since every value is a string (Chapter 3), the `string` command is the most general tool in the language. It is an **ensemble**: a single command with many subcommands, which is the pattern Tcl uses everywhere and which you can build yourself in Chapter 14.

6.1 The essential subcommands

```
set s "  Design Compiler  "
puts "[string length $s][string trim $s][string toupper [string trim $s]]"
puts [string range "abcdefgh" 2 4]
puts [string index "abcdef" end-1]
puts [string first "Comp" $s]
puts [string map {a A e E} "beetle cage"]
puts [string repeat "-" 20]
puts [string match {*.v} "top.v"]
puts [string equal -nocase "ABC" "abc"]
```

```
19|Design Compiler|DESIGN COMPILER
cde
e
9
bEEtLE cAgE
-----
1
1
```

Subcommand	Does	Note
<code>string length s</code>	Characters (not bytes)	For bytes, use a byte array (Chapter 22)
<code>string index s i</code>	One character	<code>end</code> , <code>end-1</code> and <code>end-\$n</code> are valid indices everywhere in Tcl
<code>string range s i j</code>	Substring	Inclusive of both ends
<code>string first/last needle haystack ?start?</code>	Search	Returns -1 when absent - always compare against -1, not against false
<code>string map {from to ...} s</code>	Multiple simultaneous replacements	The fastest templating tool in Tcl ; pairs are applied left to right, each position once
<code>string match pattern s</code>	Glob matching	<code>*</code> , <code>?</code> , <code>[chars]</code> , <code>\\</code> to escape
<code>string trim/trimleft/trimright s ?chars?</code>	Strip characters	Default strips whitespace; the argument is a set of characters, not a prefix
<code>string equal ?-nocase? a b</code>	Comparison	Prefer over <code>==</code> for strings
<code>string compare a b</code>	-1 / 0 / 1	For sort comparators
<code>string is class ?-strict? s</code>	Classification	See the trap below
<code>string replace s i j ?new?</code>	Splice by index	
<code>string reverse s, string tolower/toupper/totitle</code>	Obvious	
<code>string cat a b ...</code>	Concatenate	Clearer and faster than repeated <code>append</code> in an expression

`STRING TRIM` TAKES A CHARACTER SET

`string trimright $path "/"` removes trailing slashes - fine. But `string trimright $filename ".log"` removes any trailing run of the characters `.`, `l`, `o` and `g`, so `catalog.log` becomes `cata`. To strip a suffix use `file rootname`, a regular expression, or an explicit length check. This bug is common enough to be worth grepping your codebase for.

6.2 Classification with `string is`

```
foreach v {42 3.14 abc "" " "} {
    puts "'$v': int=[string is integer -strict $v]\
    double=[string is double -strict $v]\
    alnum=[string is alnum -strict $v]\
    space=[string is space -strict $v]"
}
```

```
'42': int=1 double=1 alnum=1 space=0
'3.14': int=0 double=1 alnum=0 space=0
'abc': int=0 double=0 alnum=1 space=0
'': int=0 double=0 alnum=0 space=0
' ': int=0 double=0 alnum=0 space=1
```

ALWAYS PASS -STRICT

Without `-strict`, **every class returns true for the empty string**, because vacuously all of its characters satisfy the class. `string is integer $x` on an empty `$x` returns 1, and the validation you thought you wrote does nothing. Make `-strict` a habit; the only time you want the loose behaviour is when an empty value is genuinely acceptable.

6.3 format and scan

```
puts [format "%s has %d items totalling %.2f" cart 3 19.5]
puts [format "%-10s|%5.2f|%03d" name 3.14159 7]
puts [format "%x %o %b %e" 255 255 255 12345.678]

scan "top_module 42 3.14" "%s %d %f" name count value
puts "$name / $count / $value"
```

```
cart has 3 items totalling 19.50
name      | 3.14|007
ff 377 11111111 1.234568e+04
top_module / 42 / 3.14
```

`format` follows C's `printf` closely, with `%b` for binary added. `scan` is `sscanf`: it **assigns to variables** and returns how many conversions succeeded, so check that return value rather than assuming the parse worked. For anything with structure, a regular expression (Chapter 10) is usually clearer than `scan`.

6.4 Splitting, joining and building

```
puts [split "a,b,,c" ,]
puts [llength [split "a,b,,c" ,]]
puts [join {1 2 3} " + "]
puts [split "one two" ""]      ;# empty split char = every character
```

```
a b {} c
4
1 + 2 + 3
o n e { } t w o
```

`SPLIT` IS NOT `SPLIT ON WHITESPACE`

`split $line` splits on **any** of the default characters (space, tab, newline, carriage return, form feed) and produces empty elements for runs of them - `split "a b"` gives three elements, the middle one empty. If you want words, the value is usually already a valid list, so `foreach word $line` or `llength $line` does the right thing (Chapter 7). Use `split` when the separator is explicit: commas, colons, path separators.

6.5 Two ways to build text

```
# subst: like a double-quoted word, but applied to a value
set user tom
puts [subst {Hello $user, 2+2=[expr {2+2}]}]

# string map: no substitution at all - safe with untrusted templates
puts [string map [list %USER% tom %DAY% Friday] \
    "Hi %USER%, see you %DAY%"]
```

```
Hello tom, 2+2=4
Hi tom, see you Friday
```

`SUBST` RUNS CODE; `STRING MAP` DOES NOT

`subst` performs the substitutions of Chapter 2 on a value, including **command substitution** - so `subst $template` on a template that came from a user or a file will execute anything in brackets. Use `subst -nocommands -novariables` to restrict it, or prefer `string map`, which does pure textual replacement and cannot execute anything. This distinction is the templating half of Chapter 32's security discussion.

6.6 Unicode and encodings, briefly

Tcl strings are sequences of Unicode characters, not bytes. Conversion happens at the boundary - when reading or writing a channel - and is controlled by `fconfigure -encoding` (Chapter 22). Inside the interpreter you can name characters by code point, and `string length` counts characters:

```
set s "caf\u00e9"
puts "[string length $s] chars"
puts [string length [encoding convertto utf-8 $s]]
```

```
4 chars
5
```

Four characters, five bytes in UTF-8. Confusing the two is the usual cause of truncated non-ASCII output. **encoding system** reports the platform default used for filenames and for channels you have not configured - it is **not** necessarily UTF-8, and on the machine this book was built on it was **iso8859-1**.

Exercises

- 1 Write a procedure that strips a given suffix from a filename correctly, and show that `string trimright` gets it wrong.
- 2 Validate a user-supplied number with `string is`, first without and then with `-strict`, and explain the difference.
- 3 Build a report line with `format` so that three columns align for any input width.
- 4 Implement a template expander with `string map`, then break it by using `subst` on a template containing `[exec date]`.
- 5 Count the characters and the UTF-8 bytes of a string containing accented letters.

7. Lists

The list is Tcl's workhorse container: ordered, indexed from zero, and - crucially - it is also a string, formatted so that it can be read back exactly (Chapter 3). Nearly every command that takes 'several things' takes a list, and the commands below are the ones you will use every day.

7.1 Building and indexing

```
set l [list alpha beta gamma delta]
puts "[length $l] | [lindex $l 0] | [lindex $l end] | [lindex $l end-1]"
puts [lrange $l 1 2]
puts [lreverse $l]
puts [linsert $l 2 NEW]
puts [lreplace $l 1 2 X Y Z]
```

```
4 | alpha | delta | gamma
beta gamma
delta gamma beta alpha
alpha beta NEW gamma delta
alpha X Y Z delta
```

`LINSERT` AND `LREPLACE` RETURN NEW LISTS

They do not modify their argument. `lappend` and `lset` **do** modify the variable named in their first argument - which is why `lappend items x` takes a variable **name** while `linsert $items 0 x` takes a **value**. Mixing the two conventions up is a common early error, and the reason `set items [linsert $items 0 x]` is the correct shape for the second form.

7.2 Nested lists

```
set m {{1 2 3} {4 5 6} {7 8 9}}
puts [lindex $m 1 2]           ;# row 1, column 2 - one command
lset m 0 0 99                  ;# in-place assignment through indices
puts $m
```

```
6
{99 2 3} {4 5 6} {7 8 9}
```

Multi-index `lindex` and `lset` make nested lists a workable matrix type without any extra library.

7.3 Searching

```
set names {clk rst_n data_in data_out enable}
puts [lsearch $names data_in]
puts [lsearch -all $names data*]
puts [lsearch -all -inline $names data*]
puts [lsearch -exact $names data*]
puts [lsearch -inline -glob $names *_n]
```

```
2
2 3
data_in data_out
-1
rst_n
```


`LSEARCH` DEFAULTS TO GLOB; `SWITCH` DEFAULTS TO EXACT

The two most-used pattern commands in Tcl have **opposite defaults**. `lsearch $l data*` matches by wildcard - which is why line 2 above found two elements - while `switch -- $x data*` compares literally. Always state the matching style explicitly (`-exact`, `-glob`, `-regexp`) and you will never have to remember which is which.

Option	Effect
<code>-exact</code> / <code>-glob</code> / <code>-regexp</code>	Matching style (glob is the default)
<code>-all</code>	Return every match, not the first
<code>-inline</code>	Return the matching values instead of indices
<code>-not</code>	Invert the test
<code>-index n</code>	Match against element n of each sublist - searching a table by column
<code>-sorted</code> <code>-bisect</code>	Binary search of a sorted list: O(log n)
<code>-integer</code> / <code>-real</code> / <code>-dictionary</code> / <code>-nocase</code>	Comparison mode

7.4 Sorting

```
puts [lsort {banana Apple cherry}]
puts [lsort -nocase {banana Apple cherry}]
puts [lsort {10 9 100 1}]           ;# ASCII order by default!
puts [lsort -integer {10 9 100 1}]
puts [lsort -dictionary {item10 item9 item100 item1}]
puts [lsort -unique {b a c a b}]

set rows {{bob 30} {alice 25} {carol 35}}
puts [lsort -index 1 -integer $rows]
```

```
Apple banana cherry
Apple banana cherry
1 10 100 9
1 9 10 100
item1 item9 item10 item100
a b c
{alice 25} {bob 30} {carol 35}
```

`-DICTIONARY` IS WHAT YOU WANT FOR NAMES WITH NUMBERS IN THEM

It compares embedded digit runs numerically and is case-insensitive: `item9` sorts before `item10`, which plain string sorting gets backwards. For signal names, instance paths, file names and version strings - the things EDA scripts sort constantly - `-dictionary` is almost always the right choice.

For anything more complex, `-command` takes a comparison procedure returning a negative number, zero or a positive number:

```
proc by_length {a b} { expr {[string length $a] - [string length $b]} }
puts [lsort -command by_length {ccc a bb}]
```

```
a bb ccc
```

Note the cost: `-command` calls a Tcl procedure O(n log n) times. For large lists, prefer `-index` or `decorate-sort-undecorate`.

7.5 Destructuring, stacks and queues

```
lassign {1 2 3} a b c
puts "$a$b$c"

set rest [lassign {1 2 3 4 5} first second] ;# returns the remainder
puts "first=$first second=$second rest=$rest"

set stack {}
lappend stack x y z ;# push
set top [lindex $stack end] ;# peek
set stack [lrange $stack 0 end-1] ;# pop
puts "popped $top leaving $stack"
```

```
123
first=1 second=2 rest=3 4 5
popped z leaving x y
```

7.6 concat versus list - the difference that bites

```
puts [concat {a b} {c d}]
puts [list {a b} {c d}]
puts "[llength [concat {a b} {c d}]] vs [llength [list {a b} {c d}]]"
```

```
a b c d
{a b} {c d}
4 vs 2
```

concat flattens one level; **list** preserves structure. Use **concat** (or **{*}**) to splice, **list** to nest.

Tcl 8.5 added the expansion operator **{*}**, which splices a list into the command being built - the correct modern replacement for **eval** in most cases (Chapter 18):

```
set args {-nocase -all}
set names {Clk RST data}
puts [lsearch {*}$args $names *T*]

# without {*} the whole list would be ONE argument:
puts [catch {lsearch $args $names *T*} err]
puts $err
```

```
1 2
1
bad option "-nocase -all": must be -all, -ascii, -bisect,
  -decreasing, -dictionary, -exact, -glob, -increasing, -index,
  -inline, -integer, -nocase, -not, -real, -regexp, -sorted,
  -start, or -subindices
```

Wrapped for width. Without **{*}** the two options arrived as a single argument, which is exactly the class of bug **{*}** was introduced to remove.

7.7 Performance notes

Operation	Cost	Comment
lappend var x	Amortised O(1)	The correct way to build a list in a loop
set var [linsert \$var end x]	O(n) per call	Quadratic in a loop - a common accidental slowdown
lindex	O(1)	On a value with a list representation
lrange	O(k)	Copies the extracted range
llength	O(1)	Cached in the list representation
lsearch (linear)	O(n)	Use -sorted -bisect for O(log n), or a dict (Chapter 8) for O(1) lookup

Operation	Cost	Comment
<code>lsort</code>	$O(n \log n)$	<code>-command</code> multiplies by the cost of a procedure call
String-appending a list	$O(n)$ plus a shimmer	See Chapter 3 - never build lists with <code>append</code>

Exercises

- 1 Build a 100,000-element list with `lappend` and with `set l [linsert $l end ...]`, and time both.
- 2 Sort a list of instance names such as `u10 u9 u100` three ways: default, `-dictionary`, and `-command`. Explain each result.
- 3 Use `lsearch -index` to find a row in a list of {name value} pairs.
- 4 Convert a nested list into a flat one with `concat` and `{*}`, and explain why the results differ.
- 5 Write `lpop` as a procedure that removes and returns the last element of a list variable (Chapter 12 shows how to take the variable by name).

8. Dictionaries

A dict is an ordered mapping from keys to values, added in Tcl 8.5. Like everything else it is also a string - specifically, a list with an even number of elements - which means it can be passed around, stored and returned like any other value. That single property makes it the right default container for structured data, and the reason arrays (Chapter 9) are now mostly legacy.

8.1 The basics

```
set d [dict create name top_module width 32 signed 1]
puts $d
puts [dict get $d width]

dict set d width 64      ;# modifies the variable in place
dict set d owner alice
puts $d
puts "size=[dict size $d] keys=[dict keys $d]"
puts "[dict exists $d signed] [dict exists $d missing]"
dict unset d signed
puts $d
```

```
name top_module width 32 signed 1
32
name top_module width 64 signed 1 owner alice
size=4 keys=name width signed owner
1 0
name top_module width 64 owner alice
```

DICTS PRESERVE INSERTION ORDER; ARRAYS DO NOT

`dict keys` returns keys in the order they were first inserted, and that order is stable across `dict for`. An **array** has no order at all - `array get` returns whatever the hash table produces, which varies between runs and between Tcl versions. If your report needs deterministic output - and an EDA report always does - a dict gives it to you for free, while an array needs `lsort` at every use.

8.2 Iteration

```
dict for {k v} $d { puts " $k = $v" }
puts [dict values $d]
```

```
name = top_module
width = 64
owner = alice
top_module 64 alice
```

`dict for` is the idiomatic loop. Because a dict is also a valid list, `foreach {k v} $d {...}` works too - but `dict for` is clearer and, on a value already in the dict representation, faster.

8.3 Nested dicts: structured configuration

```
set cfg [dict create]
dict set cfg synth effort    high
dict set cfg synth retiming 1
dict set cfg place effort    medium
puts $cfg
puts [dict get $cfg synth effort]

dict for {stage opts} $cfg { puts "$stage: [dict get $opts effort]" }
```

```
synth {effort high retiming 1} place {effort medium}
high
synth: high
place: medium
```

`dict set` and `dict get` take any number of keys and create the intermediate levels automatically, so a nested configuration needs no setup code.

8.4 The accumulation commands

```
set counts [dict create a 1 b 2]
dict incr counts a 5           ;# add to a numeric value
dict incr counts c             ;# creates c = 1
dict lappend counts items x    ;# treat the value as a list
dict lappend counts items y
dict append counts a "!"       ;# treat the value as a string
puts $counts

puts [dict merge {a 1 b 2} {b 99 c 3}] ;# later wins
puts [dict filter {a 1 b 2 c 3} value 2]
puts [dict filter {alpha 1 beta 2} key a*]
```

```
a 6! b 2 c 1 items {x y}
a 1 b 99 c 3
b 2
alpha 1
```

Command	Does	Typical use
<code>dict create ?k v ...?</code>	Build one	Initialisation
<code>dict get d k ?k...?</code>	Read, error if absent	Reading a required field
<code>dict exists d k ?k...?</code>	Test	Guard before <code>dict get</code>
<code>dict set var k ?k...? v</code>	Write in place	Building nested structures
<code>dict unset var k</code>	Remove	
<code>dict keys/values/size d</code>	Inspect	
<code>dict for {k v} d body</code>	Iterate	The main loop
<code>dict incr/append/lappend var k ?v?</code>	Accumulate without a get/modify/set round trip	Counters, grouping, histograms
<code>dict merge d ...</code>	Combine, later values winning	Defaults plus overrides
<code>dict filter d key value script pat</code>	Select a subset	
<code>dict update var k v ... body</code>	Bind keys to variables for the body, writing back at the end	Editing several fields at once
<code>dict with var body</code>	Bind every key as a variable of the same name	Convenient and dangerous - see below
<code>dict remove/replace</code>	Non-destructive edits	

'DICT WITH' WRITES BACK EVERYTHING IN SCOPE

`dict with cfg { set effort high }` binds every key of `cfg` to a local variable, runs the body, then **copies every one of those variables back into the dict**. If the body creates a variable whose name happens to match nothing, it is added as a new key; if it unsets one, the key is removed. In a long body this couples the dict to every local name you use, so keep `dict with` bodies short, or prefer `dict update`, which binds only the keys you list.

8.5 Grouping and inverting - the two patterns you will write most

```

# Invert a mapping
set inv [dict create]
dict for {k v} {a 1 b 2} { dict set inv $v $k }
puts $inv

# Group a list of {key value} rows by key
set rows {{clk 1} {rst 0} {clk 2} {data 5}}
set groups [dict create]
foreach row $rows {
    lassign $row k v
    dict lappend groups $k $v
}
puts $groups

```

```

1 a 2 b
clk {1 2} rst 0 data 5

```

8.6 Dict or array?

	dict	array
Is a value	Yes - pass, return, nest, store in a list	No - it is a variable; only its name can be passed
Ordering	Insertion order, deterministic	Unordered
Nesting	Natural, any depth	Only by encoding keys such as <code>a(x,y)</code>
Iteration	<code>dict for</code>	<code>foreach {k v} [array get a]</code>
Traces	No	Yes - the main reason arrays survive
Very large data	Fine, but copy-on-write applies	Slightly better when updated in place millions of times
Use it when	Almost always	You need variable traces, or you are working in existing code that uses them

Exercises

- 1 Build a nested dict describing a design's clocks and their frequencies, and print it with `dict for` in a stable order.
- 2 Count word frequencies in a file using `dict incr`, then print the top ten by count.
- 3 Write the same grouping code with an array and with a dict; compare the line counts and the determinism of the output.
- 4 Use `dict merge` to layer default options under user options.
- 5 Demonstrate the `dict with` write-back surprise by creating an unrelated variable inside the body.

9. Arrays

Arrays are Tcl's original associative container, and they behave differently from everything else in the language: an array is not a value. It is a **collection of variables** sharing a name, which explains both its capabilities and its limitations.

9.1 Basics

```
array set cfg {width 32 depth 1024 name mem}
puts $cfg(width)
set cfg(depth) 2048
puts "[array size cfg] : [lsort [array names cfg]]"
puts "[array exists cfg] [info exists cfg(width)]"
puts [array names cfg d*]
puts [lsort -stride 2 [array get cfg]]
```

```
32
3 : depth name width
1 1
depth
depth 2048 name mem width 32
```

AN ARRAY IS NOT A VALUE - THIS IS THE WHOLE DIFFERENCE

You cannot pass an array to a procedure, return it, put it in a list, or nest one inside another. What you pass is its **name**, and the receiving procedure re-links it with **upvar** (Chapter 12). Every awkwardness of arrays follows from this, and every advantage - traces, in-place element updates, **env** - follows from it too.

```
proc takes_value {v} { return "got: $v" }
catch {takes_value $cfg} err
puts "ERR: $err"

proc takes_name {name} {
    upvar 1 $name a
    return "keys: [lsort [array names a]]"
}
puts [takes_name cfg]
```

```
ERR: can't read "cfg": variable is array
keys: depth name width
```

9.2 Converting between arrays, dicts and lists

```
set d [dict create {*}[array get cfg]]    ;# array -> dict
puts [dict get $d name]

array set cfg2 [dict create x 1 y 2]      ;# dict -> array
puts [lsort [array names cfg2]]
```

```
mem
x y
```

array get produces a flat key/value list, which is exactly a dict, so conversion in both directions is one command. This is the migration path for old code.

9.3 Traces: the reason arrays still exist

A **variable trace** fires a script when a variable is read, written or unset. It works on scalars too, but the array form - watching every element of a collection - is what has no dict equivalent.

```
array set watched {}
trace add variable watched write {apply {{name idx op} {
    upvar 1 $name a
    puts " TRACE: ${name}($idx) set to ${a($idx)}"
}}}

set watched(alpha) 1
set watched(beta) 2
```

```
TRACE: watched(alpha) set to 1
TRACE: watched(beta) set to 2
```

Trace type	Fires when	Used for
read	The variable is read	Computed or lazily-loaded values
write	It is assigned	Validation, logging, invalidating a cache, GUI updates (Chapter 27)
unset	It is removed or goes out of scope	Cleanup
array	An array-wide operation such as <code>array names</code>	Presenting a virtual array
trace add execution	A command runs (Chapter 20)	Profiling and debugging

TRACES MAKE BEHAVIOUR NON-LOCAL

A trace turns an innocuous `set` into arbitrary code execution somewhere else in the program, and errors inside a trace propagate back to the assignment. That is powerful for a GUI or a debugger and corrosive in ordinary application logic, where the reader of `set x 1` has no way of knowing that a hundred lines will run. Use traces deliberately, document them at the point the variable is declared, and prefer explicit calls where you can.

9.4 Special arrays you already have

Array	Contains
<code>env</code>	Environment variables - and writing to it changes the environment children inherit (Chapter 23)
<code>tcl_platform</code>	<code>os</code> , <code>osVersion</code> , <code>machine</code> , <code>platform</code> , <code>pointerSize</code> , <code>threaded</code> - the portability facts
<code>auto_index</code> , <code>auto_path</code>	The autoloader's index and search path (Chapter 15)
<code>errorCode/errorInfo</code>	Scalars, not arrays, but the same 'special global' family (Chapter 13)

```
puts $tcl_platform(platform)
puts $tcl_platform(os)
puts $tcl_platform(pointerSize)
puts [info exists env(HOME)]
```

```
unix
Linux
8
1
```

Exercises

- 1 Write a procedure that takes an array by name, doubles every value, and returns nothing.
- 2 Convert an array to a dict, sort it by key, and print it deterministically.
- 3 Add a write trace that rejects negative values by raising an error, and observe where the error surfaces.
- 4 Print your `tcl_platform` array and note which fields would let a script behave differently on Windows.

- 5 Take an existing script that uses arrays and convert one of them to a dict. Note what got simpler and what got harder.

10. Regular Expressions

Most Tcl in the world exists to read text that some other program produced: log files, timing reports, netlists, tool output. The regular-expression engine is therefore the most-used library in the language after `string`, and Tcl's is a full **Advanced Regular Expression** implementation - closer to Perl's than to `grep`'s.

10.1 The two commands

```
puts [regexp {[a-z_][a-z0-9_]*$} "clk_div2"]
puts [regexp {[a-z_][a-z0-9_]*$} "2fast"]

set line "slack (VIOLATED) : -0.253"
if {[regexp {slack \((\w+)\)\s*:\s*(-?\[d.\]+\)} $line -> status value}] {
    puts "status=$status value=$value"
}
```

```
1
0
status=VIOLATED value=-0.253
```

ALWAYS BRACE THE PATTERN

A regular expression is full of backslashes and dollar signs, and braces stop Tcl from touching any of them (Chapter 2, rule 5). `regexp {\d+} $s` passes `\d+` to the engine; `regexp "\d+" $s` passes `d+`, because Tcl consumed the backslash first. Every regular expression in this book is braced, and every one in your code should be. The single exception is when you genuinely need to interpolate a variable into the pattern - and then you should be wrapping it in `***=` or escaping it, as shown below.

The `->` in the example is not syntax: it is a variable name, chosen by convention to receive match 0 (the whole match) when you only care about the capture groups.

10.2 The options that do the real work

Option	Effect
<code>-all</code>	Match repeatedly; returns the count of matches
<code>-inline</code>	Return the matches as a list instead of assigning to variables
<code>-all -inline</code>	The most useful combination: a flat list of every match and every capture group
<code>-line</code>	<code>^</code> and <code>\$</code> match at line boundaries, and <code>.</code> stops matching newline - for multi-line reports
<code>-nocase</code>	Case-insensitive
<code>-indices</code>	Return character positions rather than text
<code>-expanded</code>	Ignore whitespace and <code>#</code> comments in the pattern - use it for anything long
<code>-start n</code>	Begin at an offset
<code>--</code>	End of options; use it whenever the pattern could begin with <code>-</code>

```
set text "u1/clk u2/rst u3/data"
puts [regexp -all -inline {u(\d+)/(\w+)} $text]
puts [regexp -all {u\d+} $text]

regexp -indices {clk} "the clk pin" pos
puts $pos

set report "WARN: a\nERROR: b\nWARN: c"
puts [regexp -all -inline -line {^ERROR: (.*)$} $report]
```

```
u1/clock 1 clk u2/rst 2 rst u3/data 3 data
3
4 6
{ERROR: b} b
```

With `-all` `-inline` the result is match, group 1, group 2, match, group 1, ... - so `foreach {whole inst pin} $result` walks it naturally.

10.3 Substitution with `regsub`

```
puts [regsub {\s+} "a b" " "]           ;# first match only
puts [regsub -all {\s+} "a b c" " "]     ;# every match
puts [regsub -all {(\w+)@(\w+)} "bob@x carol@y" {\2:\1}]

# The 4-argument form assigns to a variable and returns the COUNT
set n [regsub -all {[aeiou]} "hello world" "" stripped]
puts "$n vowels removed -> $stripped"
```

```
a b
a b c
x:bob y:carol
3 vowels removed -> hll wrld
```

In the replacement text, `\1` to `\9` insert capture groups and `&` inserts the whole match. Brace the replacement too, or Tcl will substitute the backslashes before `regsub` sees them.

10.4 Greediness, and the trap it sets

```
puts [regexp -inline {<(.)>} "<a> and <b>"] ;# greedy
puts [regexp -inline {<(.*?)>} "<a> and <b>"] ;# non-greedy
```

```
{<a> and <b>} {a> and <b>}
<a> a
```

The greedy `+` swallowed everything up to the **last** `>`. Adding `?` makes each quantifier lazy. This one behaviour accounts for a large share of 'my regexp matches too much' problems.

A WHOLE-EXPRESSION GREEDINESS RULE YOU WILL NOT EXPECT

In Tcl's ARE the greediness of the **first** quantifier decides the overall matching preference for the entire expression, not just for that quantifier. That differs from Perl and is documented in the [re_syntax](#) manual page. When a mixed-greediness pattern behaves oddly, split it into two matches rather than reasoning about the interaction.

10.5 Syntax reference

Construct	Meaning
<code>.</code> <code>[abc]</code> <code>^[abc]</code> <code>[a-z]</code>	Any character; set; negated set; range
<code>\d</code> <code>\w</code> <code>\s</code>	Digit, word character, whitespace (and <code>\D</code> <code>\W</code> <code>\S</code> for their complements)
<code>[:alpha:]</code> <code>[:digit:]</code> <code>[:space:]</code>	POSIX classes - locale-aware and clearer in long patterns
<code>*</code> <code>+</code> <code>?</code> <code>{n,m}</code>	Quantifiers; append <code>?</code> for non-greedy
<code>^</code> <code>\$</code>	Start and end of string, or of line with <code>-line</code>
<code>\m</code> <code>\M</code> <code>\y</code> <code>\Y</code>	Start of word, end of word, any word boundary, non-boundary - Tcl spells these differently from Perl's <code>\b</code>
<code>(...)</code> , <code>(?:...)</code>	Capturing and non-capturing groups
<code>a b</code>	Alternation

Construct	Meaning
<code>(?=...) (?!...)</code>	Lookahead, positive and negative
<code>(?i)</code>	Inline case-insensitivity
<code>***=text</code>	Treat the rest as a literal string - the safe way to search for user-supplied text

```
puts [regsub -all {\mclk\M} "clk clkdiv myclk clk_x clk" "CLOCK"]
```

```
CLOCK clkdiv myclk clk_x CLOCK
```

`\m` and `\M` matched only the standalone occurrences - exactly what you want when renaming a signal and exactly what a bare `clk` would get wrong.

10.6 Long patterns: use -expanded

```
puts [regexp -expanded {
    ^ (\w+)      # the name
    \s* = \s*
    (\d+)        # the value
    $
} "width = 32" -> n v]
puts "$n $v"
```

```
1
width 32
```

Whitespace and comments in the pattern are ignored, so a twenty-character regular expression becomes readable. If you need a literal space in an expanded pattern, write `[ ]` or `\` .

10.7 Practical parsing

```
# The shape of almost every report parser you will write
set fh [open $report_file r]
set violations {}
while {[gets $fh line] >= 0} {
    if {[regexp {^ \s*(\S+)\s+slack\s+(\(VIOLATED\))\s+(-?\d[.]+\s*)} \
        $line -> endpoint slack]} {
        lappend violations [list $endpoint $slack]
    }
}
close $fh
set violations [lsort -index 1 -real $violations]
puts "worst: [lindex $violations 0]"
```

Read line by line, match with an anchored pattern, accumulate a list of lists, sort at the end. Do not try to write one regular expression for the whole file - line-oriented matching is faster, clearer and far easier to debug.

THREE HABITS THAT KEEP REGULAR EXPRESSIONS MAINTAINABLE

Anchor them. `^` and `$` turn a pattern that matches anywhere into one that matches a specific line format, and catch malformed input instead of silently accepting it. **Test them in isolation** in `tclsh` before embedding them, using a handful of lines that should and should not match. **Comment them with ``-expanded``** once they exceed about forty characters - the next person to read the script will be you, in eighteen months, at 2 a.m.

DO NOT BUILD A PATTERN BY INTERPOLATING UNTRUSTED TEXT

`regexp "$user_input" $line` lets the input change the pattern - at best a wrong match, at worst a pathological expression that takes exponential time (a regular-expression denial of service). If the text is data rather than a pattern, prefix it with `***=`, which tells the engine to treat the whole remainder literally, or use `string first` / `string match` instead.

Exercises

- 1 Write a pattern that validates a Verilog identifier and test it against five valid and five invalid names.
- 2 Extract every instance path and pin name from a netlist fragment with `-all -inline`.
- 3 Take a greedy pattern that matches too much and fix it two ways: with a non-greedy quantifier and with a negated character class.
- 4 Rename a signal in a file with `regsub` using `\m/\M`, and show that the naive pattern corrupts `myclk`.
- 5 Rewrite your longest existing regular expression with `-expanded` and comments.

PART III

Structure

How a script grows into a program: procedures and the return-code machinery underneath them, reaching into other scopes with `upvar` and `uplevel`, error handling that cleans up after itself, namespaces and ensembles, and packaging code so other people can load it.

11. Procedures

`proc` is a command that takes a name, an argument list and a body, and creates a new command. The new command is indistinguishable from a built-in one - which is the property that lets a library, or an EDA tool, extend the language rather than sit beside it.

11.1 Arguments: defaults and variadics

```
proc greet {name {greeting Hello}} { return "$greeting, $name!" }
puts [greet World]
puts [greet World Hi]

proc sum {args} {
    set t 0
    foreach n $args { incr t $n }
    return $t
}
puts "[sum 1 2 3 4] [sum]"

proc report {title args} { return "$title: [llength $args] items ($args)" }
puts [report Files a b c]
```

```
Hello, World!
Hi, World!
10 0
Files: 3 items (a b c)
```

Form	Meaning
<code>{a b c}</code>	Three required arguments
<code>{a {b default}}</code>	<code>b</code> is optional with a default
<code>{a args}</code>	<code>args</code> collects all remaining arguments as a list - only as the last parameter , and only under that exact name
<code>{}</code>	No arguments
Defaults are evaluated at definition time	<code>proc p {{when [clock seconds]}}</code> captures the time the proc was defined, not called - almost never what you want

11.2 Return values and the implicit result

```
proc last_value {} { set x 5; expr {$x * 2} } ;# no explicit return
puts [last_value]
```

```
10
```

A procedure returns the result of its last command if it does not `return`. Relying on this is idiomatic in short procedures and confusing in long ones - use `return` explicitly when the body has more than a few lines.

EVERY COMMAND RETURNS A ****CODE**** AS WELL AS A VALUE

The code is one of `ok` (0), `error` (1), `return` (2), `break` (3), `continue` (4), or an application-defined integer. `return` sets it: `return -code error "message"` raises an error, and `return -code break` makes the **caller's** loop break. This mechanism is what makes `break`, `continue`, `error` and `return` ordinary commands rather than syntax, and it is the foundation of Chapter 13's error handling and Chapter 12's custom control structures.

```
proc validate {n} {
    if {[string is integer -strict $n]} {
        return -code error "not an integer: $n"
    }
    return $n
}
puts [catch {validate abc} msg]
puts $msg
```

```
1
not an integer: abc
```

11.3 Introspection: procedures are inspectable

```
puts [info args greet]
info default greet greeting v
puts $v
puts [info body greet]
puts [info procs gr*]
```

```
name greeting
Hello
    return "$greeting, $name!"
greet
```

The body comes back as the literal text you wrote. Chapter 20 builds on this to write procedures that inspect and generate other procedures.

11.4 Recursion, and its limit

```
proc fact {n} { expr {$n <= 1 ? 1 : $n * [fact [expr {$n-1}]] } }
puts [fact 20]
puts [fact 30]      ;# arbitrary precision - no overflow
```

```
2432902008176640000
2652528598121910586363084800000000
```

Recursion depth is limited by `interp recursionlimit` (1,000 by default) rather than by the C stack, so a runaway recursion produces a clean error instead of a crash. For deep recursion, either raise the limit deliberately or use `tailcall` (Chapter 18), which replaces the current frame instead of adding to it.

11.5 Anonymous procedures with `apply`

```
set f {[x y] {expr {$x + $y}}}      ;# an argument list and a body
puts [apply $f 3 4]

puts [lmap v {1 2 3} {apply {[n] {expr {$n*$n}}} $v}]
```

```
7
1 4 9
```

`apply` takes a two- or three-element list - arguments, body, and optionally a namespace - and calls it without giving it a name. This is Tcl's lambda, and it is what you pass to `lsort -command`, to traces, and to any callback.

11.6 Renaming and deleting commands

```
proc original {} { return "orig" }
rename original renamed
puts [renamed]
rename renamed ""      ;# renaming to empty DELETES the command
puts [llength [info procs renamed]]
```



```
orig  
0
```

THE WRAPPING IDIOM EVERY LARGE TCL CODEBASE USES

`rename` works on **any** command, including built-ins. To log every call to `exec`, rename it out of the way and define a replacement that logs and then calls the original: `rename exec _real_exec` then `proc exec {args} { log $args; uplevel 1 [list _real_exec {*} $args] }`. This is how test harnesses stub out tool commands, how EDA wrappers add logging to vendor commands, and how Chapter 32's safe interpreters hide dangerous ones. It is powerful and it is invisible - document it loudly wherever you do it.

Exercises

- 1 Write a procedure with one required argument, one optional argument and `args`, and call it four different ways.
- 2 Demonstrate that a default argument is evaluated at definition time.
- 3 Write `assert` as a procedure using `return -code error`, then use it.
- 4 Use `info body` and `info args` to print the definition of a procedure at run time.
- 5 Wrap a built-in command with `rename` so that it logs its arguments, then restore it.

12. upvar, uplevel and Traces

Tcl has no pointers and no references, but it has something more general: a procedure can reach into another **stack frame** by name. **upvar** links a variable; **uplevel** runs a script. Between them they provide call-by-reference, custom control structures, and most of what an object system would otherwise be needed for.

12.1 upvar: taking an argument by name

```
proc increment {varName {by 1}} {
    upvar 1 $varName v      ;# link the CALLER's variable to local v
    incr v $by
}
set count 10
increment count 5
puts $count

proc lpop {listVar} {
    upvar 1 $listVar l
    set last [lindex $l end]
    set l [lrange $l 0 end-1]
    return $last
}
set stack {a b c}
puts "[lpop stack] / $stack"
```

```
15
c / a b
```

Note the call: **increment count 5**, not **increment \$count 5**. You pass the **name**; the procedure does the dereferencing. Every Tcl command that modifies a variable - **incr**, **lappend**, **append**, **dict set** - follows this convention.

Level argument	Refers to
1 (or omitted)	The immediate caller - the normal case
2	The caller's caller
#0	The global frame
#1	Absolute frame 1, counting from the global frame

Arrays are passed the same way, which is the standard answer to 'how do I pass an array to a procedure?' (Chapter 9):

```
proc dump {arrName} {
    upvar 1 $arrName a
    foreach k [lsort [array names a]] { puts " $k = $a($k)" }
}
array set cfg {b 2 a 1}
dump cfg
```

```
a = 1
b = 2
```

12.2 uplevel: running a script in another frame

```
proc do_twice {body} {
    uplevel 1 $body
    uplevel 1 $body
}
set n 0
do_twice { incr n }      ;# n is the CALLER's variable
puts $n
```

2

THIS IS HOW EVERY DOMAIN-SPECIFIC COMMAND IS BUILT

When a tool gives you `create_clock -period 10 [get_ports clk]` or a test framework gives you `test name -body {...} -result ok`, what you are seeing is a procedure that takes scripts and configuration as arguments and runs them with `uplevel` or `eval`. Tcl needs no macro system because a command that receives an unevaluated script is already a macro.

USE `LIST` TO BUILD THE SCRIPT YOU UPLEVEL

`uplevel 1 "set x $value"` breaks the moment `$value` contains a space, a bracket or a quote - it is string-building a command, which is the injection bug of Chapter 32. Write `uplevel 1 [list set x $value]` instead: `list` quotes each element exactly so that it arrives as one word. The rule is general - **build commands with `list`, never with string concatenation.**

12.3 Knowing where you are: `info level`

```
proc outer {} { set marker OUTER; middle }
proc middle {} { inner }
proc inner {} {
    puts "  this call: [info level 0]"
    puts "  my caller: [info level -1]"
    puts "  depth:      [info level]"
}
outer
```

```
this call: inner
my caller: middle
depth:      3
```

`info level 0` returns the current call **with its arguments**, which is how a procedure can log or re-raise exactly how it was invoked.

12.4 Traces on execution

Chapter 9 introduced variable traces. The execution trace is the other half: it fires when a **command** is entered or left, and it is the basis of profilers, debuggers and coverage tools (Chapter 30).

```
proc target {x} { return [expr {$x*2}] }

trace add execution target enter {apply {{cmd op} {
    puts "  ENTER: $cmd"
}}}
trace add execution target leave {apply {{cmd code result op} {
    puts "  LEAVE: -> $result"
}}}

puts [target 21]
```

```
ENTER: target 21
LEAVE: -> 42
42
```

Trace	Fires	Callback receives
<code>enter</code>	Before the command body runs	The full command with arguments, and the operation
<code>leave</code>	After it returns	Command, return code, result, operation
<code>enterstep/leavestep</code>	Around every command inside the traced procedure	The same - this is how a single-step debugger is built

Exercises

- 1 Write `swap a b` that exchanges the values of two caller variables.
- 2 Write `with_timing body` that runs a script in the caller's scope and prints how long it took.
- 3 Show that `uplevel 1 "set x $v"` breaks for a value containing a space, and that the `list` form does not.
- 4 Use `info level 0` inside a procedure to log every call with its arguments.
- 5 Add `enterstep` tracing to a procedure and watch every command it executes.

13. Errors, Exceptions and Cleanup

Tcl's error handling is built on the return codes of Chapter 11: an error is simply a command returning code 1, and every mechanism in this chapter is a way of producing, catching or annotating that. Tcl 8.6 added `try/trap/finally` and `throw`, which make the result far more readable than the `catch` idioms that came before.

13.1 catch: the low-level primitive

```
set rc [catch {expr {1/0}} msg]
puts "rc=$rc msg=$msg"
puts "errorCode=$::errorCode"

# The three-argument form captures the full options dictionary
set rc [catch {open /nonexistent/file r} msg opts]
puts $msg
puts "code=[dict get $opts -code] errorcode=[dict get $opts -errorcode]"
```

```
rc=1 msg=divide by zero
errorCode=ARITH DIVZERO {divide by zero}
couldn't open "/nonexistent/file": no such file or directory
code=1 errorcode=POSIX ENOENT {no such file or directory}
```

TWO THINGS DESCRIBE EVERY ERROR

The **message** is for humans and may be reworded at any time. The **error code** - `$::errorCode`, or `-errorcode` in the options dict - is a machine-readable list whose first element names the class: `POSIX`, `ARITH`, `TCL`, or anything your own code chooses. **Branch on the error code, never on the text of the message.** Code that does `if {[string match "*no such file*" $msg]}` breaks when the wording changes or the locale differs.

13.2 try: the modern form

```
try {
    error "boom" "" {MYAPP BADTHING 42}
} trap {MYAPP BADTHING} {msg opts} {
    puts "trapped: $msg / [dict get $opts -errorcode]"
} on error {msg} {
    puts "generic: $msg"
} finally {
    puts "finally always runs"
}
```

```
trapped: boom / MYAPP BADTHING 42
finally always runs
```

Clause	Meaning
<code>trap {PREFIX} {msg opts} body</code>	Handle errors whose errorCode begins with this prefix - the structured, preferred form
<code>on error {msg opts} body</code>	Handle any error
<code>on ok {res} body</code>	Run when no error occurred
<code>on return break continue {...}</code>	Handle the other return codes
<code>finally body</code>	Always runs - on success, on error, and on <code>return</code> out of the body

```
proc f {} {
    try {
        return "from body"
    } finally {
        puts "  cleanup ran"
    }
}
puts [f]
```

```
cleanup ran
from body
```

The **finally** ran even though the body returned - which is exactly why it exists, and why **close \$fh** belongs there rather than after the body.

13.3 Raising errors well

```
proc parse_width {s} {
    if {[string is integer -strict $s]} {
        throw [list CONFIG BADWIDTH $s] "width must be an integer, got '$s'"
    }
    return $s
}

try { parse_width abc } trap {CONFIG BADWIDTH} {m o} {
    lassign [dict get $o -errorcode] _ _ bad
    puts "bad width value was: $bad"
}
```

```
bad width value was: abc
```

throw code message is **error message "" code** with the arguments in the sensible order. Putting the offending value **into the error code** lets the handler use it without parsing the message.

Command	Use
throw {CLASS SUBCLASS ...} msg	Raise a classified error (8.6+) - the default choice
error msg ?info? ?code?	The older form; still needed to re-raise with a specific stack trace
return -code error -errorcode {...} msg	Raise from a procedure without adding this frame to the trace
catch {...} msg opts +return -options \$opts \$msg	Re-raise unchanged after inspecting - preserves the original stack trace exactly

13.4 Stack traces

```
proc a {} { b }
proc b {} { error "deep failure" }
catch {a} m
puts "message: $m"
foreach l [split $::errorInfo \n] { puts "  | $l" }
```

```

message: deep failure
| deep failure
|   while executing
| "error "deep failure" "
|   (procedure "b" line 1)
|   invoked from within
| "b "
|   (procedure "a" line 1)
|   invoked from within
| "a"

```

`$::errorInfo` accumulates the trace as the error propagates. Print it when logging an unexpected failure - the message alone rarely identifies where the problem was.

13.5 Patterns that keep scripts honest

```

# 1. Acquire, use, release - with cleanup that cannot be skipped
set fh [open $path r]
try {
    process [read $fh]
} finally {
    close $fh
}

# 2. Convert an expected failure into a value
proc read_or_default {path default} {
    try {
        set fh [open $path r]
    } trap {POSIX ENOENT} {} {
        return $default
    }
    try { return [read $fh] } finally { close $fh }
}

# 3. Inspect, then re-raise with the original trace intact
if {[catch {risky_operation} msg opts]} {
    log_error $msg
    return -options $opts $msg
}

```

Error-handling rules for production scripts

- ☐ Every `catch` either handles the error or re-raises it - never swallows it silently.
- ☐ Branch on `-errorcode`, not on message text.
- ☐ Every resource acquired inside a `try` is released in its `finally`.
- ☐ `catch {...}` around a whole script body is a bug, not a safety net: it hides the line that failed.
- ☐ A script that fails exits with a non-zero status (`exit 1`), so the flow that called it notices.
- ☐ Unexpected errors log `$::errorInfo`, not just the message.
- ☐ Background errors (Chapter 24) have a handler installed - otherwise they vanish.

Exercises

- 1 Convert a `catch`-based error handler in existing code into `try/trap/finally` and compare readability.
- 2 Define your own error class with `throw` and handle it selectively while letting others propagate.
- 3 Prove that `finally` runs when the body executes `return`, `break` and `error`.
- 4 Write a procedure that logs and re-raises an error with its original stack trace intact, and verify with `$::errorInfo`.
- 5 Find a `catch` in your codebase that discards the error, and decide what it should do instead.

14. Namespaces and Ensembles

A namespace is a container for commands and variables with a hierarchical name. It is how a library avoids colliding with your script, how a package keeps its state private, and - through **ensembles** - how Tcl's own multi-word commands such as `string` and `dict` are built.

14.1 Creating and using

```
namespace eval ::timing {
    variable unit ns           ;# a namespace variable
    variable count 0

    proc report {value} {
        variable unit         ;# link it into this procedure
        variable count
        incr count
        return "$value $unit (report #$count)"
    }
    namespace export report    ;# eligible for import
}

puts [::timing::report 1.5]
puts [timing::report 2.5]     ;# relative names work too
puts [set ::timing::unit]
```

```
1.5 ns (report #1)
2.5 ns (report #2)
ns
```

`VARIABLE`, NOT `GLOBAL`

Inside a procedure that belongs to a namespace, `global x` reaches the **global** namespace `::x`, not the enclosing one. To reach the namespace's own variable you must write `variable x`. Forgetting this creates a second, unrelated global - and the symptom is state that silently does not persist, which is tedious to debug. Make `variable` the reflex whenever a procedure lives inside `namespace eval`.

14.2 Names, resolution and navigation

Name	Resolves to
<code>::foo::bar</code>	Absolute - unambiguous, always correct
<code>foo::bar</code>	Relative to the current namespace, then the global one
<code>bar</code>	Current namespace, then global
<code>namespace current</code>	The namespace a command is executing in
<code>namespace parent</code> , <code>namespace children</code>	Navigate the tree
<code>namespace which cmd</code>	Where a name actually resolves - the debugging command for name confusion
<code>namespace path {::a ::b}</code>	Additional namespaces to search (8.5+), avoiding imports


```
namespace eval ::a::b { proc where {} { return [namespace current] } }
puts [::a::b::where]
puts [namespace children ::a]
puts [namespace parent ::a::b]

namespace import ::timing::report
puts [report 3.5]
puts [namespace which report]
```

```
::a::b
::a::b
::a
3.5 ns (report #3)
::report
```

`namespace import` copies the command into the current namespace - note that `namespace which` now reports it as `::report`. Import sparingly: it re-creates the collisions namespaces exist to prevent.

14.3 Ensembles: building string-style commands

```
namespace eval ::tool {
    namespace export run status
    namespace ensemble create      ;# THIS makes ::tool a command

    proc run {args} { return "running $args" }
    proc status {} { return "idle" }
}

puts [tool run -fast]
puts [tool status]
puts [catch {tool bogus} e]
puts $e
```

```
running -fast
idle
1
unknown or ambiguous subcommand "bogus": must be run, or status
```

ENSEMBLES GIVE YOU A PROFESSIONAL API FOR ONE LINE OF CODE

`namespace ensemble create` turns the exported procedures of a namespace into subcommands, complete with unambiguous-prefix matching and a generated error message listing the valid options. Your library then looks exactly like `string`, `dict` and `file` - which is what users of an EDA package expect. `namespace ensemble configure` lets you map subcommand names, add options and set a default subcommand.

14.4 Namespaces and state

```
namespace eval ::cfg {
    variable settings [dict create debug 0]

    proc set_debug {v} {
        variable settings
        dict set settings debug $v
    }
    proc get {} { variable settings; return $settings }
}
::cfg::set_debug 1
puts [::cfg::get]
```

debug 1

This is the standard shape of a Tcl module: a namespace holding state in `variable`s, with exported procedures as the only way to touch it. It is object orientation with a single instance, and for most libraries it is all that is needed - Chapter 16 covers the case where you need many instances.

Exercises

- 1 Convert a group of related global procedures in your code into a namespace with exports.
- 2 Show what happens when you use `global` instead of `variable` inside a namespaced procedure.
- 3 Turn that namespace into an ensemble and call it as a single command with subcommands.
- 4 Use `namespace which` to resolve a name that exists in two namespaces.
- 5 Set `namespace path` so that a namespace can call another's commands without qualification or import, and explain when that is preferable to `namespace import`.

15. Packages, Modules and Libraries

A package is a named, versioned unit of code that a script asks for by name. The mechanism is small - two commands and an index file - and it is how every library you will use, from Tcllib to your own team's utilities, is delivered.

15.1 The three commands

Command	Who calls it	Meaning
<code>package require name</code> <code>?version?</code>	The user of the package	Load it, or fail with a clear error. Returns the version loaded
<code>package provide name</code> <code>version</code>	The package itself	Declare that this file provides that version
<code>package ifneeded name</code> <code>version script</code>	The index file	'If someone requires this, run this script to load it' - so nothing is sourced until it is needed

15.2 A complete package, on disk

```
# lib/timing/timing.tcl
package require Tcl 8.6
package provide timing 1.2

namespace eval ::timing {
    namespace export report
    namespace ensemble create
    variable unit ns
    proc report {value} { variable unit; return "$value $unit" }
}
```

```
# lib/timing/pkgIndex.tcl
package ifneeded timing 1.2 [list source [file join $dir timing.tcl]]
```

`$dir` is set by the package loader to the directory containing the index, so the file works wherever the package is installed. Use `list` here for the same reason as everywhere else: paths contain spaces.

```
# main.tcl
lappend auto_path [file join [file dirname [info script]] lib]
set v [package require timing]
puts "loaded timing $v"
puts [timing report 3.5]
puts "versions available: [package versions timing]"
```

```
loaded timing 1.2
3.5 ns
versions available: 1.2
```

Run for real: the package was found by searching `auto_path`, its `pkgIndex.tcl` was evaluated, and `timing.tcl` was sourced only when `package require` asked for it.

HOW `PACKAGE REQUIRE` ACTUALLY FINDS THINGS

It searches every directory in `$auto_path` (and their immediate subdirectories) for `pkgIndex.tcl` files, evaluates them all - which only registers `package ifneeded` scripts, loading nothing - and then runs the script for the best matching version. So: to make your package findable, either install it under a directory already on `auto_path`, or `lappend auto_path` your own directory before requiring it, as the example does. When a require fails, print `$auto_path` first; nine times in ten the directory is simply not on it.

15.3 Versions

Request	Accepts
<code>package require foo</code>	The highest available version
<code>package require foo 1.2</code>	1.2 or any later 1.x - not 2.0. Tcl's rule is that a change of major version means incompatibility
<code>package require -exact foo 1.2</code>	Exactly 1.2
<code>package require foo 1.2-2.0</code>	A range (8.5+)
Version numbering	<code>major.minor.patch</code> ; use <code>a</code> and <code>b</code> for alpha and beta as in <code>1.3a1</code>

15.4 Tcl Modules: the simpler alternative

A **Tcl Module** (`.tm`) is a single file whose name encodes the package and version - `timing-1.2.tm` - placed on the module path. There is no index file and no `package ifneeded`: the loader derives everything from the filename. For a pure-Tcl package in one file, this is the least-effort distribution format.

```
# timing-1.2.tm -- the whole package, one file, no index
package require Tcl 8.6
namespace eval ::timing { ... }

# and to make the directory searchable:
::tcl::tm::path add /opt/mytools/tcl-modules
package require timing 1.2
```

15.5 What is already available

Source	Contains
Built into Tcl	<code>http</code> , <code>msgcat</code> (localisation), <code>platform</code> , <code>tcltest</code> , <code>zlib</code> , <code>TclOO</code> , <code>opt</code>
Tcllib	The standard library: <code>json</code> , <code>csv</code> , <code>base64</code> , <code>md5/sha256</code> , <code>struct::*</code> (queues, trees, graphs, matrices), <code>cmdline</code> , <code>fileutil</code> , <code>textutil</code> , <code>math::*</code> , <code>uri</code> , <code>ftp</code> , <code>smtp</code> , <code>snit</code>
Tklib	Widgets and plotting for Tk (Chapter 27)
<code>tls</code>	TLS/SSL channels (Chapter 25)
<code>tdbc</code> , <code>sqlite3</code>	Database access
<code>Thread</code>	Threading (Chapter 31)
<code>Expect</code>	Interactive automation (Chapter 26)
<code>tcllibc</code> , <code>critcl</code>	Compiled accelerators (Chapter 33)

```
puts [lsort [package names]]
```

```
Tcl TclOO http msgcat opt platform tcl::tommath tcltest timing zlib
```

What a plain `tclsh` has after loading our example package. In a vendor tool shell the same command lists that vendor's packages, which is the fastest way to discover what a tool exposes.

15.6 Structuring your own library

A package that other people can use

- ☐ One namespace, matching the package name, holding all state.
- ☐ `package require` for every dependency, with a minimum version, at the top of the file.
- ☐ `package provide` exactly once, with a version you increment on every release.
- ☐ Only intended API commands are exported; everything else is private by convention (a leading underscore) or by not exporting it.

- ☐ An ensemble front end if the API has more than three or four commands.
- ☐ No side effects at load time beyond defining commands - never open files, connect to servers or print at `source` time.
- ☐ A `tests/` directory using `tcldtest` (Chapter 29) that runs from the package directory.
- ☐ A version history and a statement of which Tcl version is required.

`SOURCE` IS NOT A PACKAGE SYSTEM

`source ../lib/utils.tcl` works until the script is run from a different directory, or two scripts source the same file twice and redefine its procedures, or two versions of a utility exist in one flow. `package require` solves all three - it is path-independent, it loads once, and it versions. In large EDA flows, where scripts source scripts that source scripts, this difference is the difference between a maintainable flow and a fragile one.

Exercises

- 1 Turn a file of utility procedures you already have into a package with a `pkgIndex.tcl`, and load it with `package require`.
- 2 Package the same code as a `.tm` module and compare the effort.
- 3 Break the load path deliberately and read the error; then print `$auto_path` to see why.
- 4 Install Tcllib if you can and use `csv` and `json` to convert a data file.
- 5 Add a version check that requires at least Tcl 8.6, and verify it fails cleanly on 8.5.

PART IV

Advanced Language

Object orientation with TclOO and the systems that preceded it; dynamic evaluation done safely; coroutines, which give Tcl generators and cooperative concurrency in one command; and the introspection that makes the whole interpreter inspectable from inside.

16. TclOO: Object Orientation

Tcl went twenty years without a built-in object system, and the community produced several - incr Tcl, snit, XOTcl. Tcl 8.6 settled the question with **TclOO**, which is now part of the core and is the foundation the others are built on. It is small, dynamic and class-based, and objects are commands.

16.1 A class, an object, a lifetime

```
oo::class create Counter {
    variable count name           ;# instance variables

    constructor {{start 0} {label counter}} {
        set count $start
        set name $label
    }
    method bump {{by 1}} { incr count $by; return $count }
    method value {} { return $count }
    method describe {} { return "$name = $count" }
    destructor { puts "  destroying $name" }
}

set c [Counter new 10 clocks]      ;# "new" auto-names the object
puts [$c bump]
puts [$c bump 5]
puts [$c describe]
$c destroy
```

```
11
16
clocks = 16
  destroying clocks
```

AN OBJECT IS A COMMAND; A CLASS IS A COMMAND THAT MAKES THEM

`Counter new` returns the **name of a new command**, and `$c bump 5` is an ordinary command invocation whose first argument selects a method. That is why objects compose with everything else in Tcl - you can `rename` them, alias them into another interpreter (Chapter 32), trace them (Chapter 12), or store them in a dict. Use `ClassName create name ...` instead of `new` when you want to choose the command name yourself.

16.2 Inheritance and `next`

```
oo::class create Base {
    method greet {} { return "base" }
    method both {} { return "both: [my greet]" }    ;# "my" = self-call
}
oo::class create Derived {
    superclass Base
    method greet {} { return "derived (parent says [next])" }
}

set d [Derived new]
puts [$d greet]
puts [$d both]
```

```
derived (parent says base)
both: derived (parent says base)
```

`next` calls the next implementation in the method resolution order - the equivalent of `super`, but it also works through mixins and filters. `my` invokes a method on the current object, including private ones.

16.3 Mixins and filters: composition without inheritance

```
oo::class create Loggable {
    method log {msg} { return "[LOG\] $msg" }
}
oo::define Derived mixin Loggable      ;# add behaviour to an EXISTING class
puts [$d log "mixed in"]
```

```
[LOG] mixed in
```

Feature	What it does	Use for
superclass	Classical single or multiple inheritance	Is-a relationships
mixin	Splice a class into the resolution order of a class or one object	Cross-cutting behaviour: logging, serialisation, debugging
filter	A method that wraps every other method call	Tracing, access control, argument validation
oo::define / oo::objdefine	Modify a class - or a single object - after creation	Dynamic behaviour, monkey-patching in tests
self	The current object's name; self class , self method , self target	Chaining, introspection, delegation
oo::object , oo::class	The root object and the root class - oo::class is itself an object	Metaclasses: classes that create classes

16.4 Introspection

```
puts [info object class $d]
puts [lsort [info class methods Derived]]
puts [info class superclasses Derived]
puts [info object isa object $d]
```

```
::Derived
greet
::Base
1
```

16.5 Class-level methods and instance registries

```
oo::class create Registry {
    self method instances {} { return [info class instances Registry] }
    constructor {} {}
}
Registry create r1
Registry create r2
puts [llength [Registry instances]]
```

```
2
```

self method defines a method on the **class object** - the equivalent of a static method.

WHEN TO USE OBJECTS IN TCL, AND WHEN NOT TO

Use a namespace (Chapter 14) when there is exactly one of something: a configuration store, a logger, a tool wrapper. **Use TclOO** when you need many independent instances with their own state - connections, parsers, widgets, design objects - or when polymorphism genuinely simplifies the code. Do not convert working procedural code to objects for its own sake; Tcl's dict and namespace facilities already cover most of what small classes are used for elsewhere.

Exercises

- 1 Write a `Stack` class with `push`, `pop`, `peek` and `size`, and a destructor that reports how many items were left.
- 2 Subclass it as `BoundedStack` that raises an error past a limit, using `next` for the normal path.
- 3 Add a `Logging` mixin to one instance only, with `oo::objdefine`, and show the other instance is unaffected.
- 4 Use `info class methods` and `info object class` to write a procedure that describes any object.
- 5 Convert a namespace-based module of yours into a class and note what improved and what did not.

17. The Other Object Systems

You will meet three other object systems in existing code, especially in EDA and in Tcllib. Knowing what each looks like is enough to read and maintain it; for new code, TcIOO is the right default.

System	Character	You will meet it in
incr Tcl (itcl)	The oldest; C++-flavoured with <code>class</code> , <code>public/protected/private</code> , <code>inherit</code> . Shipped with many vendor tools	Long-lived EDA infrastructure and older commercial code
snit	Pure Tcl, from Tcllib. Delegation-based rather than inheritance-based: a snit type wraps and forwards to components, and it excels at building Tk megawidgets	Tcllib, Tk applications
XOTcl / NX	Highly dynamic, research-grade metaprogramming: per-object mixins, filters, method introspection everywhere	Academic code, some web frameworks
TcIOO	In the core since 8.6; the others can be, and are, layered on top of it	New code, and modern Tcllib

```
# itcl, for recognition when you meet it
itcl::class Adder {
    private variable total 0
    public method add {n} { incr total $n; return $total }
    public method value {} { return $total }
}

# snit, showing its delegation style
snit::type Server {
    option -port 8080
    component socket
    delegate method listen to socket
    constructor {args} { $self configurelist $args }
}
```

Both are illustrative: `itcl` and `snit` are separate packages and were not available in the interpreter used for this book, so unlike every other listing here these two were not executed.

THE MIGRATION RULE

Do not rewrite working `itcl` or `snit` code into TcIOO for its own sake - the risk is entirely on your side and the benefit is stylistic. Write **new** classes in TcIOO, keep the boundary between old and new explicit, and remember that all of them are ultimately commands, so the two styles interoperate without adapters.

Exercises

- 1 Find an object system in code you maintain and identify which one it is from its syntax alone.
- 2 Write the same small class in TcIOO and, if you have Tcllib, in snit; compare the line counts.
- 3 Explain the difference between inheritance and delegation to a colleague using those two implementations.

18. Dynamic Evaluation, Done Safely

Because a script is a string, Tcl can build and run code at run time. This is the language's greatest power and its main hazard: the same mechanism that lets an EDA tool define new commands from a specification also lets a filename containing a bracket execute arbitrary code.

18.1 `eval` and the expansion operator

```
set cmd [list puts "hello world"]
eval $cmd           ;# the old way
{*}$cmd            ;# the modern way (8.5+): expand, then invoke

set args {-nocase -all}
puts [lsearch {$}$args {Clk RST} *T*]
```

```
hello world
hello world
1
```

`{*}` REPLACED MOST USES OF `EVAL`

`eval` concatenates its arguments into a script and evaluates it - **re-parsing the text**, with all the quoting hazards that implies. `{*}$list` merely splices the list's elements in as separate words, with no re-parsing at all. If your goal is 'call this command with these arguments', use `{*}`. Reserve `eval` for the rare case where you genuinely have a **script** to run.

18.2 Why string-built commands break

```
set filename {report[1].txt}

set bad "puts \"file: $filename\""
catch {eval $bad} e
puts "string-built FAILED: $e"

eval [list puts "file: $filename"] ;# list quotes it correctly
```

```
string-built FAILED: invalid command name "1"
file: report[1].txt
```

The bracket in the filename was parsed as a command substitution. With `list`, every element is quoted so that it survives the re-parse as a single word.

```
set evil {x"; exec touch /tmp/pwned; puts "}
eval "puts \"$evil\"" ;# never do this
puts [file exists /tmp/pwned]
```

```
x
1
```

The injected `exec` ran. This is not a contrived example: it is what happens when a script builds a command from a design name, a report field or a user's input. Chapter 32 covers the defences.

Rules for dynamic code

- ☐ Build commands with `list` and invoke with `{*}` - never with string concatenation.
- ☐ `uplevel 1 [list cmd $arg]`, not `uplevel 1 "cmd $arg"`.
- ☐ Validate any value that will become a **command name** against an allowlist.
- ☐ Prefer `string map` to `subst` for templates (Chapter 6).
- ☐ If you must evaluate untrusted script, do it in a safe interpreter (Chapter 32).

- ☐ Treat `expr` with an unbraced argument as dynamic evaluation - because it is.

18.3 `subst` and its switches

```
set x 5
puts [subst {x=$x calc=[expr {$x*2}]]
puts [subst -nocommands {x=$x calc=[expr {$x*2}]]
puts [subst -novariables {x=$x calc=[expr {$x*2}]]
```

```
x=5 calc=10
x=5 calc=[expr {5*2}]
x=$x calc=10
```

`-nocommands` is the switch that makes `subst` safe for templates whose text you do not control.

18.4 Aliases, `tailcall` and friends

```
interp alias {} mylist {} list 1 2      ;# a partially-applied command
puts [mylist 3 4]

proc countdown {n} {
    if {$n <= 0} { return done }
    tailcall countdown [expr {$n-1}]    ;# replaces this frame
}
puts [countdown 50000]                 ;# no recursion limit hit
```

```
1 2 3 4
done
```

Without `tailcall`, 50,000 levels of recursion would exceed the default `interp recursionlimit` of 1,000.

Command	Does
<code>eval script...</code>	Concatenate and evaluate as a script
<code>{*}list</code>	Expand a list into words of the current command
<code>uplevel level script</code>	Evaluate in another stack frame (Chapter 12)
<code>namespace eval ns script</code>	Evaluate in a namespace
<code>apply {args body} ...</code>	Anonymous procedure (Chapter 11)
<code>interp alias {} new {} cmd ?prefix...?</code>	Define a command as another command plus fixed leading arguments
<code>tailcall cmd ...</code>	Replace the current frame with a call - proper tail calls
<code>coroutine, yield</code>	Chapter 19
<code>unknown</code>	The hook called when a command name is not found - how autoloading and <code>tool_command</code> shortcuts work

Exercises

- 1 Rewrite three `eval` calls in existing code as `{*}` and confirm the behaviour is identical.
- 2 Reproduce the bracket-in-filename failure, then fix it with `list`.
- 3 Write a template expander with `subst -nocommands` and prove that a bracketed payload in the template does not execute.
- 4 Use `interp alias` to make a shorthand for a long vendor command with fixed options.
- 5 Convert a deeply recursive procedure to use `tailcall` and show the recursion limit is no longer reached.

19. Coroutines

A coroutine is a procedure that can suspend itself and be resumed later, keeping its local variables and its position in the code. Tcl 8.6 added them in a single command, and they give you generators, iterators, state machines and cooperative concurrency without threads.

19.1 The mechanism

```
proc counter {start} {
    set i $start
    while 1 {
        yield $i          ;# suspend, handing $i back to the caller
        incr i            ;# resumes HERE on the next call
    }
}
```

```
puts "coroutine command returns: [coroutine c1 counter 10]"
puts "then: [c1] [c1] [c1]"
```

```
coroutine command returns: 10
then: 11 12 13
```

coroutine name proc args... creates a **command** called **name** and runs the body until its first **yield**, returning that value. Each later call to **name** resumes where it left off; the argument to that call becomes the result of the **yield**.

caller	coroutine c1
-----	-----
coroutine c1 counter 10 ---->	set i 10
	while 1 {
<---- returns 10 -----	yield \$i
[c1] -----	incr i (resumes here)
<---- returns 11 -----	yield \$i
[c1] -----	incr i
<---- returns 12 -----	yield \$i

The coroutine keeps its own stack frame, locals and position.

19.2 Generators over data

```
coroutine gen apply {{items} {
    yield          ;# the initial suspension
    foreach i $items { yield $i }
}} {a b c}

while {[info commands gen] ne ""} {
    set v [gen]
    if {[info commands gen] eq ""} break ;# body ended, command gone
    puts " got $v"
}
puts " generator finished"
```

```
got a
got b
got c
generator finished
```

When the body returns, the coroutine command **deletes itself** - which is how the loop knows to stop. Testing **info commands** is the standard termination idiom.

19.3 State machines without a state variable

```

proc parser {} {
    set state idle
    while 1 {
        set tok [yield $state]      ;# the caller sends a token
        switch -- $state {
            idle { if {$tok eq "start"} { set state running } }
            running { if {$tok eq "stop"} { set state idle } }
        }
    }
}
coroutine p parser
foreach t {start x stop start} { puts " $t -> [p $t]" }

```

```

start -> running
x -> running
stop -> idle
start -> running

```

Values flow both ways: **yield** returns whatever the resuming call passed, so a coroutine is a two-way channel, not just a producer.

19.4 Where coroutines really pay: event-driven code

The transforming use is in event-driven programs (Chapter 24). Without coroutines, waiting for input means splitting a procedure into callbacks that hold their state in globals; with them, the natural sequential code can suspend and resume:

```

# Callback style: state lives outside the logic
proc on_line {chan} {
    global phase buffer
    ...state machine spread across invocations...
}

# Coroutine style: the same logic reads top to bottom
coroutine session apply {{chan} {
    set greeting [yield]      ;# waits for the first line
    puts $chan "hello, $greeting"
    set name [yield]          ;# waits for the next
    puts $chan "welcome, $name"
}} $chan

fileevent $chan readable [list resume_with_line session $chan]

```

Chapter 25 develops this into a complete asynchronous server. This pattern - a coroutine per connection, resumed by **fileevent** - is the modern way to write concurrent Tcl.

Command	Meaning
coroutine name cmd args...	Create and start
yield ?value?	Suspend, returning value to the resumer
yieldto cmd args...	Suspend and transfer directly to another command or coroutine, without returning to the caller
info coroutine	The current coroutine's name, or empty
rename name {}	Destroy a suspended coroutine

TWO COROUTINE CONSTRAINTS WORTH KNOWING EARLY

A coroutine cannot **yield** from inside a command that does not support it - notably from within **catch**-style C code in older extensions, and not across a C stack frame in general. And a suspended coroutine holds its entire frame, so a leaked coroutine leaks memory; delete abandoned ones with **rename name {}**.

Exercises

- 1 Write a coroutine that yields the Fibonacci sequence and take the first twenty values.
- 2 Turn a file reader into a generator that yields one record at a time, and consume it with `foreach`-like code.
- 3 Build a two-state protocol parser as a coroutine and drive it with a list of tokens.
- 4 Use `yieldto` to alternate between two coroutines and print the interleaving.
- 5 Take a callback-based piece of code you have and rewrite it as a coroutine; compare how much state disappeared.

20. Introspection and Metaprogramming

Everything in a Tcl interpreter can be inspected from inside it: which commands exist, what a procedure's body is, which variables are in scope, how deep the call stack is, and which line of which file is executing. That is what makes debuggers, profilers, test frameworks and EDA tool wrappers writable in Tcl itself.

20.1 The `info` command

Query	Returns
<code>info exists v / info vars pat</code>	Existence; variables visible here
<code>info globals pat / info locals pat</code>	By scope
<code>info commands pat / info procs pat</code>	All commands / only Tcl procedures
<code>info args p / info body p / info default p arg var</code>	A procedure's definition
<code>info level / info level n</code>	Stack depth; the call at that level, with arguments
<code>info frame / info frame n</code>	File and line number of a frame - what a stack trace is built from
<code>info script</code>	The file currently being sourced
<code>info nameofexecutable</code>	The interpreter binary
<code>info patchlevel / info tclversion</code>	Version
<code>info complete s</code>	Whether a string is a syntactically complete script - how a REPL knows to keep reading
<code>info functions</code>	Math functions available to <code>expr</code>
<code>info hostname / info library</code>	Environment

```
proc sample {a {b 2} args} { return ok }
puts "args: [info args sample]"
puts "body: [string trim [info body sample]]"
puts "vars: [lsort [info vars ::tcl_p*]]"
puts "script: [file tail [info script]]"

puts [info complete "set x 1"]
puts [info complete "if {1} {}"]
```

```
args: a b args
body: return ok
vars: ::tcl_patchLevel ::tcl_pkgPath ::tcl_platform
script: v21.tcl
1
0
```

20.2 Generating procedures

```
foreach {name op} {add + sub - mul *} {
    proc $name {a b} [format {return [expr {$a %s $b}]] $op]
}
puts "[add 3 4] [sub 10 3] [mul 6 7]"
puts [info body add]
```



```
7 7 42
return [expr {$a + $b}]
```

`proc` takes its name and body as ordinary strings, so generating families of commands is a `foreach` loop. This is how EDA wrappers produce one command per tool object type, and how ORM and RPC stubs are built.

GENERATE WITH 'FORMAT' AND BRACES, NOT WITH QUOTES

The body above is built with `format` from a **braced** template, so `$a` and `$b` survive into the generated procedure and only `%s` is substituted. Building it with double quotes would substitute `$a` at generation time - almost certainly an empty string - and produce a procedure that silently computes nonsense. When generating code, decide for every `$` whether it belongs to the generator or to the generated code, and brace accordingly.

20.3 Wrapping and memoising existing commands

```
proc slow {n} { after 1; return [expr {$n*$n}] }

proc memoize {name} {
    rename $name _memo_$name
    set body [format {
        set key [list %s $args]
        if {[info exists ::memo_cache($key)]} { return $::memo_cache($key) }
        set r [_memo_%s {*} $args]
        set ::memo_cache($key) $r
        return $r
    } $name $name]
    proc $name {args} $body
}

memoize slow
puts [slow 12]
puts [slow 12]           ;# second call is served from the cache
puts [array size ::memo_cache]
```

```
144
144
1
```

Rename, generate a wrapper, install it under the original name - the same three steps as the logging wrapper of Chapter 11, applied to caching.

20.4 The `unknown` handler

When a command name does not resolve, Tcl calls `unknown` with the whole command. That hook implements autoloading, command abbreviation in the interactive shell, and - in many EDA environments - transparent access to tool objects.

```
rename unknown _original_unknown
proc unknown {args} {
    set cmd [lindex $args 0]
    if {[string match "get_*" $cmd]} {
        return "stub for $cmd with [lrange $args 1 end]"
    }
    uplevel 1 [list _original_unknown {*} $args] ;# chain to the real one
}

puts [get_ports clk]
catch {no_such_command} e
puts $e
```

```
stub for get_ports with clk
invalid command name "no_such_command"
```

The unhandled case was passed to the original handler, so autoloading and normal error reporting still work. **Always chain**: an **unknown** that does not is a debugging nightmare.

20.5 Knowing where you are: `info frame`

```
proc where {} { return [dict get [info frame -1] line] }
puts "called from line [where]"
```

```
called from line 44
```

`info frame` returns a dict with `type`, `file`, `line`, `cmd` and more. It is how a logger reports the caller's file and line without being told, and how test frameworks point at the failing assertion.

```
# A logger that reports its own call site
proc log {level msg} {
    set f [info frame -1]
    set where [expr {[dict exists $f file]
        ? "[file tail [dict get $f file]]:[dict get $f line]"
        : "line [dict get $f line]"}]
    puts stderr "\[$level\] $where: $msg"
}
```

INTROSPECTION IS FOR TOOLS, NOT FOR LOGIC

Using `info` to build a debugger, a test runner, a profiler or a documentation generator is exactly what it is for. Using it in application logic - a procedure that behaves differently depending on who called it - produces code that cannot be reasoned about locally and breaks the moment it is refactored. If you find yourself inspecting the caller to decide what to do, pass an argument instead.

Exercises

- 1 Write a procedure that prints the full definition of any procedure, reconstructing it from `info args`, `info default` and `info body`.
- 2 Generate a family of `get_*` accessor procedures from a list of field names.
- 3 Add memoisation to a genuinely slow function in your code and measure the difference.
- 4 Install an `unknown` handler that suggests the closest matching command name, and chain to the original.
- 5 Write a `log` procedure that reports the file and line of its caller using `info frame`.

PART V

The Outside World

Files and the channel abstraction underneath them, buffering and encodings, running other programs, the event loop that makes everything asynchronous, sockets and HTTP, and Expect - the reason many people learn Tcl in the first place.

21. Files and Directories

File I/O in Tcl goes through **channels**, an abstraction shared by files, pipes, sockets and in-memory transforms. This chapter covers the file case; Chapter 22 covers what a channel is underneath.

21.1 Reading and writing

```
set fh [open report.txt w]
puts $fh "line one"
puts $fh "line two"
puts -nonewline $fh "no newline"
close $fh

# Whole file at once - simplest, and fine up to a few hundred MB
set fh [open report.txt r]
set content [read $fh]
close $fh
puts "read [string length $content] chars"

# Line by line - constant memory, and what report parsing wants
set fh [open report.txt r]
set n 0
while {[gets $fh line] >= 0} { incr n; puts "  $n: $line" }
close $fh
```

```
read 28 chars
  1: line one
  2: line two
  3: no newline
```

`GETS` RETURNS THE LENGTH, AND -1 AT END OF FILE

`while {[gets $fh line] >= 0}` is the correct loop: `gets` returns the number of characters read and **-1** at end of file. Testing `while {[eof $fh]}` instead is the classic bug - `eof` only becomes true **after** a read has failed, so that loop processes one spurious empty line at the end of every file. If you see a stray blank record in a report, this is why.

Mode	Meaning
<code>r</code>	Read; the file must exist (the default)
<code>w</code>	Write, truncating or creating
<code>a</code>	Append, creating if needed
<code>r+ w+ a+</code>	The same, plus the opposite direction
<code>rb wb (8.6+)</code>	Binary - equivalent to <code>-translation binary</code>
<code>open \$f w 0600</code>	A third argument sets Unix permissions

21.2 The `file` command

```
puts "exists=[file exists report.txt] size=[file size report.txt]"
puts "ext=[file extension report.txt] root=[file rootname report.txt]"
puts "tail=[file tail /a/b/c.txt] dirname=[file dirname /a/b/c.txt]"
puts "join=[file join /a b c.txt]"
puts "type=[file type report.txt] isdir=[file isdirectory .]"
```

```
exists=1 size=28
ext=.txt root=report
tail=c.txt dirname=/a/b
join=/a/b/c.txt
type=file isdir=1
```

Subcommand	Purpose
<code>exists isfile isdirectory readable writable executable</code>	Tests
<code>size mtime atime type stat attributes</code>	Metadata
<code>dirname tail rootname extension split join normalize</code>	Path manipulation - use these instead of string surgery, and your script works on Windows too
<code>mkdir delete copy rename link</code>	Manipulation
<code>nativeName</code>	Convert to the platform's own form
<code>tempfile (8.6+)</code>	Create and open a temporary file safely

NEVER BUILD PATHS WITH STRING CONCATENATION

`set p "$dir/$name"` breaks on Windows, doubles separators when `$dir` already ends in one, and mangles a `$dir` of `~`. `file join $dir $name` handles all three. Likewise `file dirname/file tail` instead of `string last /`, and `file rootname` instead of the `string trimright` trap from Chapter 6.

21.3 Finding files

```
puts [lsort [glob *.log]]
puts "[glob -nocomplain *.none]" ;# empty, no error
puts [lsort [glob -types f *]] ;# files only
puts [glob -directory /etc -tails host*]
```

```
a.log b.log
''
a.log b.log bin.dat c.txt report.txt t.txt
hosts host.conf hostname
```

Always pass `-nocomplain` unless a missing match is genuinely an error: bare `glob` raises one when nothing matches, which is rarely what a script wants. For recursive walks, use `fileutil::find` from Tcllib or write the recursion explicitly.

21.4 Writing files safely

```
# The atomic-replace idiom: never leave a half-written file behind
proc write_atomic {path content} {
    set tmp ${path}.tmp.[pid]
    set fh [open $tmp w]
    try {
        puts -nonewline $fh $content
    } finally {
        close $fh
    }
    file rename -force $tmp $path ;# atomic on the same filesystem
}
```

A tool killed halfway through writing a report should leave the old report intact, not a truncated one. Write to a temporary name in the same directory, then rename - the rename is atomic, so readers see either the old file or the new one.

File-handling rules

- ☐ Every `open` has a matching `close`, in a `finally` (Chapter 13).
- ☐ Use `file join` and the `file` subcommands for all path work.

- ☐ `glob -nocomplain`, and check the result is non-empty.
- ☐ Read line by line unless you know the file is small.
- ☐ Write via a temporary file and `file rename` when the output matters.
- ☐ Set the encoding explicitly for text you did not create (Chapter 22).
- ☐ Check `file writable` before a long computation whose result you cannot save.

Exercises

- 1 Write a procedure that returns a file's contents, closing the handle even on error.
- 2 Demonstrate the `eof`-based loop bug by processing a file that does not end in a newline.
- 3 Write a recursive directory walker that returns every file matching a pattern.
- 4 Implement `write_atomic` and prove it by killing the script between the write and the rename.
- 5 Rewrite a path-manipulation routine that uses `string` operations to use `file` subcommands instead.

22. Channels: Buffering, Encodings and Transforms

A channel is Tcl's uniform interface to a byte or character stream. Files, pipes, sockets, serial ports and in-memory transformations all present the same API, and all are configured with the same command - which is why learning `fconfigure` once pays off everywhere.

22.1 `fconfigure`: the five options that matter

```
set fh [open t.txt w]
puts [fconfigure $fh]
```

```
-blocking 1 -buffering full -buffer-size 4096 -encoding iso8859-1
-eofchar {} -translation lf
```

Wrapped for width. Note `-encoding iso8859-1`: that is this machine's system encoding, **not** UTF-8, and it is applied to every file you open without saying otherwise.

Option	Values	Why it matters
<code>-buffering</code>	<code>full</code> / <code>line</code> / <code>none</code>	<code>full</code> means output may not appear until the buffer fills or you <code>flush</code> - the usual reason a log file looks empty while a script runs. Use <code>line</code> for logs and interactive output
<code>-translation</code>	<code>auto</code> / <code>lf</code> / <code>crlf</code> / <code>cr</code> / <code>binary</code>	End-of-line conversion. <code>`binary`</code> is mandatory for non-text data or bytes will be silently rewritten
<code>-encoding</code>	<code>utf-8</code> , <code>iso8859-1</code> , <code>binary</code> , ...	Character-set conversion. Set it explicitly for any file whose encoding you know
<code>-blocking</code>	<code>1</code> / <code>0</code>	Non-blocking mode, for use with the event loop (Chapter 24)
<code>-eofchar</code>	A character	Legacy DOS text handling; usually left empty

THE THREE SYMPTOMS OF A MIS-CONFIGURED CHANNEL

Nothing in the log file until the program exits: buffering is `full`; set `-buffering line` or `flush` after each write. **Binary data corrupted, usually every 0x0D byte:** translation is not `binary`. **Accented characters mangled, or lengths wrong:** the encoding is not what the file actually uses. All three are one `fconfigure` call away from being fixed, and all three are invisible until they are not.

22.2 Binary channels

```
set fh [open bin.dat wb] ;# b = binary
puts -nonewline $fh [binary format H* "00ff10"]
close $fh
```

```
set fh [open bin.dat rb]
set data [read $fh]
close $fh
binary scan $data H* hex
puts "bytes=[string length $data] hex=$hex"
```

```
bytes=3 hex=00ff10
```

22.3 Positioning

```

set fh [open t.txt r]
puts "tell=[tell $fh]"
seek $fh 2
puts "after seek 2: [read $fh 2], tell=[tell $fh]"
seek $fh 0 end
puts "size via seek: [tell $fh], eof=[eof $fh]"
close $fh

```

```

tell=0
after seek 2: ta, tell=4
size via seek: 5, eof=0

```

22.4 Stacked transformations

A channel can have transformations pushed onto it, so compression or encryption becomes transparent to the code that reads and writes:

```

set fh [open z.gz wb]
zlib push gzip $fh                ;# compress on the way out
puts $fh "compress me, compress me, compress me"
close $fh

set fh [open z.gz rb]
zlib push gunzip $fh              ;# decompress on the way in
puts [string trim [read $fh]]
close $fh
puts "compressed size: [file size z.gz]"

```

```

compress me, compress me, compress me
compressed size: 37

```

The reading and writing code is unchanged - **zlib push** inserted itself into the channel. The **tls** package adds encryption the same way (Chapter 25).

22.5 The **chan** command and reflected channels

Tcl 8.5 grouped the channel commands under one ensemble - **chan configure**, **chan gets**, **chan puts**, **chan close**, **chan event** - and added **chan create**, which lets you implement a **channel in Tcl**. Anything that reads or writes then works against your object without knowing it is not a file:

```

# Sketch of a reflected channel: a handler object with the required
# methods (initialize, finalize, read, write, watch, ...).
set ch [chan create {read write} myHandler]
puts $ch "this goes to myHandler write method"

```

Uses: in-memory buffers for tests, instrumented channels that count bytes, protocol adaptors, and mock files in unit tests (Chapter 29). This example is illustrative - a full handler needs several methods and is longer than the space here allows.

```
puts [chan names]
```

```
stdin stdout stderr
```

The three standard channels exist in every interpreter and are configured exactly like any other - including **fconfigure stdout -buffering none** when you need output to appear immediately.

Exercises

- 1 Write a log file with **-buffering full** and watch it stay empty; switch to **line** and watch it fill.
- 2 Copy a binary file with and without **-translation binary** and compare the results byte for byte.
- 3 Read a UTF-8 file with the wrong **-encoding** and observe what happens to **string length**.
- 4 Write a gzip-compressed log with **zlib push** and read it back.
- 5 Measure the effect of **-buffersize** on the time to write a million short lines.

23. Running Other Programs

Tcl is a glue language, and `exec` is where most of the glue is applied. It is more capable than it first appears - and its error behaviour surprises everyone once.

23.1 `exec` basics

```
puts [exec echo hello world]
puts [exec sh -c "echo out; echo err >&2" 2>@1]    ;# merge stderr

set rc [catch {exec false} msg opts]
puts "rc=$rc errorcode=[dict get $opts -errorcode]"
```

```
hello world
out
err
rc=1 errorcode=CHILDSTATUS 1016 1
```

`exec` raises an error when the child exits non-zero - and also when it merely writes to stderr. The error code is `CHILDSTATUS pid exitcode`, which is how you recover the real exit status.

THE EXEC WRAPPER EVERY SERIOUS SCRIPT NEEDS

Because `exec` conflates 'wrote to stderr' with 'failed', and hides the exit code inside an error, production scripts wrap it once:

```
proc run {args} {
    set rc [catch {exec {*} $args} out opts]
    if {$rc == 0} { return [list 0 $out] }
    set ec [dict get $opts -errorcode]
    if {[lindex $ec 0] eq "CHILDSTATUS"} {
        return [list [lindex $ec 2] $out]    ;# real exit status
    }
    return [list -1 $out]                  ;# could not run at all
}

puts [run true]
puts [run sh -c "exit 3"]
```

```
0 {}
3 {child process exited abnormally}
```

23.2 Redirection and pipelines

Syntax	Meaning
<code>exec cmd > file</code>	Redirect stdout to a file
<code>exec cmd >> file</code>	Append
<code>exec cmd 2> file / 2>@1</code>	stderr to a file / merged into stdout
<code>exec cmd < file</code>	stdin from a file
<code>exec cmd << \$data</code>	stdin from a Tcl value - very useful
<code>exec a b c</code>	A pipeline
<code>exec cmd &</code>	Run in the background; returns the pid
<code>exec >@ \$chan, <@ \$chan</code>	Connect a child's stream to an existing Tcl channel
<code>open " cmd" r+</code>	A two-way pipe as a channel - write to the child, read its output

```
puts [exec printf "b\na\nc\n" | sort]

set p [open "|sort" r+]
puts $p "zebra"
puts $p "apple"
close $p w           ;# close only the WRITE side: sends EOF
puts [read $p]
close $p
```

```
a
b
c
apple
zebra
```

`close $p w` is the half-close that tells the child no more input is coming. Forgetting it is the classic cause of a script that hangs forever waiting for a pipeline's output.

23.3 Arguments, quoting and safety

```
set args {-n hello}
puts [exec echo {*} $args] ;# each element is one argument
```

```
hello
```

'EXEC' DOES NOT USE A SHELL - AND THAT IS A FEATURE

`exec ls $file` passes `$file` as **one argument**, spaces and all, with no shell involved: no globbing, no `$` expansion, no injection. The moment you write `exec sh -c "ls $file"` you have handed the value to a shell parser and re-created every shell-injection bug. Keep arguments as list elements and use `{*}`; use `sh -c` only when you genuinely need shell features, and then quote with care.

23.4 The environment and the process

```
puts "HOME set: [info exists env(HOME)]"
puts "argv0: [file tail $argv0], argc=$argc"
```

```
HOME set: 1
argv0: v25.tcl, argc=2
```

`env` is an array (Chapter 9) whose changes are inherited by children. `argv` is the list of arguments, `argv0` the script name, and `argc` their count.

Task	How
Change directory	<code>cd \$dir</code> , <code>pwd</code> - process-wide, so avoid it in libraries; prefer absolute paths
Set a child's environment	<code>set env(VAR) value</code> before <code>exec</code>
Get the process id	<code>pid</code> , or <code>pid \$channel</code> for a pipeline
Exit with a status	<code>exit 1</code> - and make sure your script does this on failure
Portable command lookup	<code>auto_execok tool</code> returns the executable path or empty
Time limits	There is no built-in timeout: run the child in the background, and use <code>after</code> (Chapter 24) to kill it - or use Expect (Chapter 26)

Exercises

- 1 Write the `run` wrapper above and use it to call a command that exits 2 while printing to both streams.
- 2 Show that `exec` raises an error for a command that writes to `stderr` but succeeds, and handle it correctly.
- 3 Build a two-way pipe to a filter program and prove the half-close is necessary.

- 4 Run a command with a filename containing a space, first with `exec cmd $f` and then with `exec sh -c "cmd $f"`, and compare.
- 5 Implement a timeout that kills a background child after five seconds.

24. The Event Loop

Tcl has had an event loop since before it was fashionable, and it is the same one whether you are writing a GUI, a server, or a script that supervises three subprocesses. Understanding it is the prerequisite for Chapters 25, 26 and 27 - and for any program that must do more than one thing at a time without threads.

24.1 The model

```

+-----+
|               THE EVENT LOOP               |
|               |                             |
|   while (events pending) {                 |
|       take the next ready event             |
|       run its callback TO COMPLETION        |
|   }                                         |
+-----+
|               |                             |
|   ^           ^           ^           ^     |
|   |           |           |           |     |
| timers        channels    sockets    GUI events |
| (after)       (fileevent) (accept/read) (bind)  |

```

ONE thread. Callbacks never overlap. A callback that takes a second blocks EVERYTHING for a second - the whole discipline of event-driven programming follows from that single fact.

24.2 Timers: *after*

```

# Two different commands share the name:
after 100                                ;# BLOCKS for 100 ms
after 100 { puts "later" }               ;# SCHEDULES a callback, returns at once

set done 0
after 50 { set ::done 1 ; puts " timer fired" }
puts " before vwait, done=$done"
vwait ::done                             ;# run the event loop until done changes
puts " after vwait, done=$done"

```

```

before vwait, done=0
timer fired
after vwait, done=1

```

Without *vwait*, *update* or a GUI, no callback ever runs: scheduling an event is not the same as processing one. This is the single most common confusion for newcomers to event-driven Tcl.

```

set id [after 10000 { puts "never" }]
puts "pending: [llength [after info]]"
after cancel $id
puts "pending after cancel: [llength [after info]]"

after idle { puts " idle callback" } ;# runs when nothing else is ready
update

```

```

pending: 1
pending after cancel: 0
idle callback

```

24.3 Channel events: *fileevent*

```

set chan [open "|sh -c {echo one; sleep 0.1; echo two}" r]
fconfigure $chan -blocking 0 -buffering line

fileevent $chan readable [list apply {{ch} {
    if {[gets $ch line] >= 0} {
        puts "  read: $line"
    } elseif {[eof $ch]} {
        catch {close $ch}
        set ::finished 1
    }
}} $chan]

vwait ::finished

```

```

read: one
read: two

```

The script stayed responsive during the child's 100 ms sleep. Note the three-way test: data, end of file, or **neither** - in non-blocking mode **gets** can return -1 simply because a whole line has not arrived yet, and treating that as end of file is a classic bug.

THE NON-BLOCKING `GETS` CONTRACT

With **-blocking 0**, **gets** returns -1 in two different situations, distinguished by **eof** and **fblocked**: **`eof` true** means the channel really ended; **`fblocked` true** means the data is incomplete and you should return and wait for the next event. Always check both, in that order.

24.4 Repeating work without blocking

```

proc every {ms body} {
    uplevel #0 $body
    after $ms [list every $ms $body]    ;# reschedule itself
}

set ::ticks 0
every 20 { incr ::ticks; if {$::ticks >= 3} { set ::stop 1 } }
vwait ::stop
puts "  ticks=$::ticks"

```

```
ticks=3
```

A self-rescheduling **after** is Tcl's periodic timer. To stop one, keep the id returned by **after** and **after cancel** it - or set a flag the body checks, as here.

24.5 **vwait**, **update** and the traps

Command	Does	Danger
vwait <i>varName</i>	Run the event loop until that variable is written	Re-entrant : events processed inside it can call code that calls vwait again, nesting the loop. Deeply nested vwaits are a classic source of confusing bugs
update	Process all currently pending events, then return	Also re-entrant, and it can run your own callbacks in the middle of a procedure - use sparingly
update <i>idletasks</i>	Process only idle events (typically GUI redraws)	Much safer than plain update ; this is what you want to refresh a progress bar
after 0 <i>script</i>	Run as soon as the loop is next idle	The standard way to defer work out of the current callback
tkwait	Tk's variant, waiting on a window or variable	Same re-entrancy caveat

NEVER DO SLOW WORK INSIDE A CALLBACK

A callback that parses a 200 MB file freezes every timer, every socket and the entire user interface until it finishes. Split long work into chunks that reschedule themselves with `after 0`, or move it into a coroutine (Chapter 19) that yields regularly, or into a thread (Chapter 31). The rule is absolute in GUI code and almost absolute in servers.

24.6 Background errors

An error inside a callback has no caller to propagate to, so Tcl hands it to the **background error handler** - by default `bgererror`, which prints it. If you never define one and never see it, errors in your event-driven code can vanish silently. Install one that logs:

```
proc bgererror {msg} {  
    puts stderr "BACKGROUND ERROR: $msg"  
    puts stderr $::errorInfo  
}  
  
# or, in modern code:  
interp bgererror {} [list my_error_handler]
```

Exercises

- 1 Schedule three timers with different delays and confirm they fire in time order, not in scheduling order.
- 2 Write a non-blocking reader for a command's output that also handles the `fblocked` case.
- 3 Build a progress indicator that updates with `update idletasks` during a long loop.
- 4 Deliberately block the event loop for two seconds inside a callback and observe the effect on a concurrent timer.
- 5 Install a `bgererror` handler and make a callback fail; confirm the error is reported with its stack trace.

25. Networking: Sockets, HTTP and TLS

Sockets in Tcl are channels (Chapter 22) with an event-driven acceptance mechanism, which means everything you know about `gets`, `puts`, `fconfigure` and `fileevent` applies unchanged. A working server is about fifteen lines.

25.1 A server and a client

```
proc accept {chan addr port} {           ;# called for each connection
    fconfigure $chan -buffering line
    fileevent $chan readable [list serve $chan]
}
proc serve {chan} {
    if {[gets $chan line] < 0} {
        if {[eof $chan]} { close $chan; set ::done 1 }
        return
    }
    puts $chan "echo: $line"
}

set srv [socket -server accept 0]       ;# port 0 = pick a free one
set port [lindex [fconfigure $srv -sockname] 2]
puts "  listening on port $port"

set cli [socket localhost $port]       ;# a client, same process
fconfigure $cli -buffering line
puts $cli "hello server"
update                                ;# let the server run
puts "  reply: [gets $cli]"
puts $cli "second"
update
puts "  reply: [gets $cli]"
close $cli
vwait ::done
close $srv
```

```
listening on port 45499
reply: echo: hello server
reply: echo: second
```

Client and server in one process, which is only possible because the server is event-driven. The `update` calls give the event loop a chance to run the accept and read callbacks - in a real client you would simply be in the event loop already.

Form	Meaning
<code>socket host port</code>	Blocking client connection
<code>socket -async host port</code>	Non-blocking connect; use <code>fileevent writable</code> to learn when it succeeded or failed
<code>socket -server cmd port</code>	Listen; <code>cmd</code> is called with channel, address and port for each connection
<code>socket -server cmd -myaddr 127.0.0.1 port</code>	Bind to localhost only - the difference between a debug port and an open service
<code>fconfigure \$sock -sockname / -peername</code>	Local and remote address, host and port
<code>fconfigure \$sock -blocking 0</code>	Essential for any server that must not stall on one slow client

25.2 The `http` package

```
package require http
puts " http version [package provide http]"
puts " formatQuery: [http::formatQuery name {a b} x 1]"
```

```
http version 2.9.8
formatQuery: name=a%20b&x=1
```

```
# Synchronous
set tok [http::geturl "http://example.com/api" -timeout 5000]
if {[http::status $tok] eq "ok" && [http::ncode $tok] == 200} {
    set body [http::data $tok]
}
http::cleanup $tok           ;# ALWAYS - it frees the token

# Asynchronous: -command makes it event-driven
http::geturl $url -command [list on_response] -timeout 5000
proc on_response {tok} {
    try {
        if {[http::status $tok] ne "ok"} { ... }
        process [http::data $tok]
    } finally {
        http::cleanup $tok
    }
}

# POST with a body and headers
http::geturl $url \
    -query [http::formatQuery key value] \
    -headers {Content-Type application/json} \
    -method POST
```

Two rules: always pass **-timeout** (the default is to wait forever), and always **http::cleanup** the token, in a **finally** - a leaked token keeps its socket and its buffers alive.

25.3 TLS

```
package require tls
http::register https 443 [list ::tls::socket -require 1 \
                           -cadir /etc/ssl/certs]
set tok [http::geturl "https://example.com/" -timeout 10000]

# Or directly, as a channel transform:
set sock [tls::socket -require 1 example.com 443]
```

The **tls** package (a separate install) registers itself as the handler for **https** and otherwise behaves exactly like a normal socket. **-require 1** is not the default - without it, certificates are not verified and the connection is encrypted but not authenticated.

25.4 Writing a robust network client

What separates a demo from production

- ☐ A timeout on every operation - connect, read and total.
- ☐ Non-blocking channels plus **fileevent**, so one slow peer cannot stall everything.
- ☐ Explicit **-encoding** and **-translation** on the channel; protocols specify their bytes.
- ☐ A framing rule (length prefix or line terminator) and a maximum message size, so a malformed peer cannot exhaust memory.
- ☐ Retry with exponential backoff and jitter, and a bounded number of attempts.
- ☐ **close** in a **finally**, and a **catch** around it - closing a broken socket can itself error.
- ☐ TLS with certificate verification for anything crossing a network you do not own.

- ☐ A background error handler (Chapter 24), so failures in callbacks are seen.

Exercises

- 1 Run the echo server and connect to it with `telnet` or `nc` from another terminal.
- 2 Extend it to handle several clients at once and prove it by connecting twice.
- 3 Add a per-connection idle timeout using `after`, and cancel it on each message.
- 4 Fetch a URL synchronously and asynchronously, and compare how the program behaves while waiting.
- 5 Write a length-prefixed message protocol over a socket and test it against a peer that sends a truncated message.

26. Expect: Automating Interactive Programs

Some programs cannot be driven with pipes: they demand a terminal, prompt for passwords, and expect a human. **Expect** solves that by allocating a pseudo-terminal, running the program in it, and letting your script wait for patterns and send responses. It was written by Don Libes in 1990, it is still the best tool for the job, and it exists only for Tcl.

26.1 The three commands

Command	Does
<code>spawn program args...</code>	Start a program under a pseudo-terminal; sets <code>\$spawn_id</code>
<code>expect ?options? pat body ...</code>	Wait until output matches one of the patterns, then run its body
<code>send string</code>	Type into the program (use <code>\r</code> , not <code>\n</code>)
<code>interact</code>	Hand the terminal back to the human
<code>close / wait</code>	Close the connection / reap the exit status

26.2 A complete, working session

```
package require Expect
log_user 0                      ;# do not echo the dialogue
set timeout 5

spawn sh
set PROMPT "TCLBOOK> "
send "PS1='$PROMPT'; export PS1\r"
expect -exact $PROMPT           ;# the echo of our own command
expect -exact $PROMPT           ;# the first real prompt

proc cmd {command} {
    global PROMPT
    send -- "$command\r"
    expect -exact "$command\r\n" ;# consume the terminal echo
    expect {
        -exact $PROMPT {
            return [string trim $expect_out(buffer) \
                [format "%s\r\n " $PROMPT]]
        }
        timeout { return TIMEOUT }
    }
}

puts " whoami -> [cmd whoami]"
puts " expr   -> [cmd {expr 6 \* 7}]"
puts " echo   -> [cmd {echo hello from the shell}]"
send "exit\r"
expect eof
puts " exit status = [lindex [wait] 3]"
```

```
whoami -> root
expr   -> 42
echo   -> hello from the shell
exit status = 0
```

Run for real with Expect 5.45.4. Every element of a production Expect script is here: a **unique prompt** you set yourself, a helper that sends a command and returns its output, explicit consumption of the terminal's echo, a timeout branch, and reaping the exit status.

THE THREE RULES THAT MAKE EXPECT SCRIPTS RELIABLE

(1) Set your own prompt. Never try to match the user's `$PS1`, which may be coloured, multi-line, or contain the hostname. Set something unmistakable and match that. **(2) Account for the echo.** A pseudo-terminal echoes what you type, so the first thing that comes back is your own command - if you do not consume it, your next pattern will match it and you will parse your own input as output. **(3) Every `expect` has a `timeout` branch.** Without one, a silent tool hangs your automation forever, and the failure is invisible in a nightly run.

26.3 Patterns and the `expect_out` array

```
expect {
    -exact "Password: "      { send "$pw\r"; exp_continue }
    -re {ERROR: (.*)\r\n} { error "tool reported: $expect_out(1,string)" }
    "Are you sure? "        { send "yes\r"; exp_continue }
    -re {\$ \$}              { }                ;# prompt reached: done
    timeout                  { error "no response in $timeout s" }
    eof                      { error "program exited unexpectedly" }
}
```

`exp_continue` restarts the same `expect` without re-entering it - which is how one block handles a sequence of prompts in any order. `$expect_out(0,string)` is the whole match, `(1,string)` the first group, and `$expect_out(buffer)` everything consumed.

Feature	Meaning
<code>-exact</code>	Literal string (fastest and least surprising)
<code>-re</code>	Regular expression (Chapter 10)
<code>-gl</code>	Glob pattern - the default , which surprises people who assume regexp
<code>timeout, eof</code>	Special patterns; <code>set timeout -1</code> waits forever
<code>expect_before / expect_after</code>	Patterns checked before or after every <code>expect</code> - the right place for a global error or timeout handler
<code>exp_continue</code>	Keep waiting within the same <code>expect</code>
<code>log_user 0/1</code>	Suppress or show the dialogue
<code>exp_internal 1</code>	Debugging: print every character received and every pattern tried - the single most useful command when a script mysteriously times out

26.4 What Expect is used for

- **ssh and telnet automation** where keys are not an option: log in, run commands on network equipment, collect output.
- **Interactive installers and licence tools** that insist on a terminal.
- **EDA tool shells** driven from outside, when the tool has no batch mode for a particular step.
- **Lab instruments and serial consoles** over a terminal device.
- **Testing interactive programs** - Expect's original purpose, and still how many command-line tools are regression-tested.
- **`autoexpect`** records a manual session and writes a script from it: a good starting point, and never the finished article.

PASSWORDS, AND THE REASON TO AVOID EXPECT WHEN YOU CAN

A password in an Expect script is a password in a file, visible to anyone who can read it and often to `ps`. Prefer ssh keys, `sshpass`-free key agents, tool batch modes, or an API. When Expect genuinely is the only route: read the secret from a file with restrictive permissions or from a secrets manager at run time, never hard-code it, keep `log_user 0` so it does not reach a log, and audit who can read the script and its logs.

Exercises

- 1 Run the shell example above and adapt it to a program you use.
- 2 Add an `expect_after` block with a `timeout` handler and confirm it fires when a command hangs.
- 3 Turn on `exp_internal 1` and study what the pattern matcher actually sees, including the echo.
- 4 Write a script that logs into a device over ssh with a key, runs three commands, and returns their output as a dict.
- 5 Take an existing manual procedure in your team and automate it, including its failure paths - not just the happy one.

PART VI

Applied Tcl

Where the language is actually used: Tk user interfaces, the EDA flows that most Tcl in the world belongs to, testing with `tcptest`, finding and fixing slow code, threads, sandboxing untrusted script, and embedding the interpreter in a C application.

27. Tk: Graphical Interfaces

Tk is the toolkit that came with Tcl and then outgrew it - it became the standard GUI layer for Python, Perl and Ruby as well. Its appeal is density: a working dialogue is a dozen lines, and the same script runs on Linux, macOS and Windows.

THE EXAMPLES IN THIS CHAPTER WERE NOT EXECUTED

Every other listing in this book was run and its output captured. Tk needs a display, and the machine that built this book has none, so the code here is written from the documented API but is - uniquely in this book - **not verified by execution**. Treat it as a guide to the shape of a Tk program rather than as tested code.

27.1 The three ideas

```
package require Tk

# 1. Widgets are commands, named by their place in a hierarchy.
ttk::label .greeting -text "Design name:"
ttk::entry .name -textvariable design_name -width 30
ttk::button .go -text "Run" -command run_flow

# 2. A geometry manager decides where they go.
grid .greeting .name .go -padx 4 -pady 4

# 3. The event loop (Chapter 24) runs everything.
# wish enters it automatically; tclsh needs "vwait forever".
```

Concept	Detail
Widget paths	<code>.</code> is the main window; <code>.f.b</code> is a button inside frame <code>.f</code> . The path is the command name
ttk:: versus classic	Always use the themed <code>ttk::</code> widgets - they look native. The classic ones remain for <code>text</code> , <code>canvas</code> and <code>listbox</code> , which have no themed equivalent
-textvariable	Binds a widget to a global variable : change the variable and the display updates, through the variable traces of Chapter 9
-command	A script run when the widget is activated - built with <code>list</code> (Chapter 18), not string concatenation
bind	Attach a script to an event: <code>bind .c <Button-1> {...}</code> , with <code>%x %y %K</code> substitutions

27.2 Geometry managers

Manager	Model	Use
<code>grid</code>	Rows and columns with weights and spans	The default choice for forms and dialogues
<code>pack</code>	Fill from an edge inwards	Toolbars, single-direction stacks, quick layouts
<code>place</code>	Absolute or relative coordinates	Rarely - custom or overlaid widgets only

NEVER MIX `GRID` AND `PACK` IN THE SAME CONTAINER

Each manager negotiates size with its parent, and two of them negotiating at once produces either a window that will not size or an infinite layout loop. Different containers may use different managers; the same one may not.

27.3 A complete small application

```
package require Tk
```

```

set ::status "ready"
set ::logfile ""

ttk::frame .top -padding 8
ttk::label .top.l -text "Log file:"
ttk::entry .top.e -textvariable ::logfile -width 40
ttk::button .top.browse -text "Browse..." -command choose_file
ttk::button .top.run -text "Analyse" -command analyse
grid .top.l .top.e .top.browse .top.run -sticky w -padx 2

text .out -height 20 -width 80 -yscrollcommand {.sb set}
ttk::scrollbar .sb -command {.out yview}
ttk::label .status -textvariable ::status -relief sunken -anchor w

grid .top      -row 0 -column 0 -columnspan 2 -sticky ew
grid .out      -row 1 -column 0 -sticky nsew
grid .sb       -row 1 -column 1 -sticky ns
grid .status   -row 2 -column 0 -columnspan 2 -sticky ew
grid rowconfigure . 1 -weight 1      ;# the text area grows
grid columnconfigure . 0 -weight 1

proc choose_file {} {
    set f [tk_getOpenFile -filetypes {{Logs {.log}} {All *}}]
    if {$f ne ""} { set ::logfile $f }
}

proc analyse {} {
    if {[file readable $::logfile]} {
        tk_messageBox -icon error -message "Cannot read $::logfile"
        return
    }
    set ::status "analysing..."
    update idletasks          ;# let the label repaint
    .out delete 1.0 end
    set fh [open $::logfile r]
    try {
        while {[gets $fh line] >= 0} {
            if {[string match "ERROR*" $line]} { .out insert end "$line\n" }
        }
    } finally { close $fh }
    set ::status "done"
}

```

Sixty lines for a real tool: file chooser, scrolling output, status bar, error dialogue and a resizable layout. This is why Tk endures for internal engineering tools even where it would not be chosen for a consumer product.

27.4 Keeping the interface alive

THE GUI RULE THAT FOLLOWS FROM CHAPTER 24

Tk runs in the event loop, so **a long computation inside a callback freezes the window** - no repaint, no button response, and on some platforms an 'application not responding' banner. The three cures: call `update idletasks` periodically to repaint (never plain `update`, which re-enters your callbacks); break the work into chunks that reschedule with `after 0`; or push it into a coroutine (Chapter 19) or a thread (Chapter 31) and update the GUI from the result.

27.5 The canvas, and where Tk still wins

The `canvas` widget draws lines, rectangles, polygons, text and images as **retained objects** you can move, tag, bind events to and delete individually. For schematic viewers, waveform displays, floorplan sketches and graph

layouts - the visualisation an engineering team actually needs - it remains one of the fastest routes from idea to working tool in any language.

```
canvas .c -width 400 -height 300 -background white
set id [.c create rectangle 20 20 120 80 -fill lightblue -tags block]
.c create text 70 50 -text "U1" -tags block
.c bind block <Button-1> {puts "clicked block %x %y"}
.c move $id 10 5
.c itemconfigure $id -fill lightgreen
```

Exercises

- 1 Build a window with an entry, a button and a label, where pressing the button copies the entry's text to the label.
- 2 Lay the same widgets out with **grid** and with **pack**, and make the window resize sensibly with **-weight**.
- 3 Add a long computation to a button callback and observe the freeze; then fix it with **after 0** chunking.
- 4 Draw a simple block diagram on a canvas and make the blocks draggable with **bind**.
- 5 Add a file dialogue and an error message box to a script you already have.

28. Tcl in EDA Flows

More Tcl runs in chip design than anywhere else. Every major tool - Synopsys Design Compiler and ICC, Cadence Genus and Innovus, AMD Vivado, Intel Quartus, Siemens Questa - embeds a Tcl interpreter and exposes its engine as commands. This chapter is what changes when your interpreter lives inside somebody else's tool.

28.1 What the tool adds, and what it does not

Aspect	In <code>tclsh</code>	In a tool shell
The language	Chapters 1-26 exactly	Identical - it is the same interpreter
Commands	About 100	Thousands: <code>get_cells</code> , <code>create_clock</code> , <code>report_timing</code> , <code>set_property</code> ...
Collections	Lists and dicts	Often an opaque collection object that is not a Tcl list - see below
Errors	<code>error</code> , <code>try</code>	Tool commands may print a message and return an empty result instead of raising
Exit status	<code>exit 1</code>	Tool-specific; some tools exit 0 even after errors unless told otherwise
Version	Yours	Whatever the vendor shipped, often an older 8.5 or 8.6 - check <code>info patchlevel</code> before using 8.6 features

TOOL COLLECTIONS ARE NOT LISTS

`get_cells *` in most tools returns a **collection**, a handle to an internal object set. `llength` on it may return 1, `foreach` may iterate once, and `lsort` may silently do nothing. Use the tool's own iterator (`foreach_in_collection`, `sizeof_collection`, `collection_to_list` or equivalent) and convert deliberately when you want Tcl semantics. Assuming a collection is a list is the most common bug in EDA scripting, and it usually fails silently rather than loudly.

28.2 The shape of a robust flow script

```
#!/usr/bin/env tclsh
# synth.tcl - run synthesis. Usage: dc_shell -f synth.tcl -x "set TOP cpu"

#--- 1. Parameters, overridable from the command line or environment
if {[info exists TOP]} { set TOP [expr {[info exists env(TOP)]
                                     ? $env(TOP) : "top"}] }
if {[info exists EFFORT]} { set EFFORT medium }
set RUNDIR [file normalize ./run_$TOP]

#--- 2. Fail loudly and early
proc die {msg} { puts stderr "ERROR: $msg"; exit 1 }
proc need_file {p} { if {[file readable $p]} { die "cannot read $p" } }

#--- 3. Log everything, with timestamps
proc log {msg} {
    puts "\[[clock format [clock seconds] -format %H:%M:%S]\] $msg"
    flush stdout
}

#--- 4. Record exactly what was run, so a result can be reproduced
log "top=$TOP effort=$EFFORT rundir=$RUNDIR"
log "tool=[info nameofexecutable] tcl=[info patchlevel]"
```

```

file mkdir $RUNDIR

#--- 5. Do the work, checking each stage
foreach f $RTL_FILES { need_file $f; read_verilog $f }
if {[catch {elaborate $TOP} err]} { die "elaboration failed: $err" }

#--- 6. Check results programmatically, not by eye
set wns [get_worst_slack]
if {$wns < 0} {
    log "TIMING FAILED: WNS = $wns"
    write_reports $RUNDIR
    exit 2
}
log "PASSED: WNS = $wns"
exit 0

```

Six habits, none of them exotic: parameters with defaults, a **die** that exits non-zero, timestamped and flushed logging, a record of the environment, error checks on every stage, and a **programmatic** pass/fail decision with a meaningful exit status.

28.3 The failures that cost tapeout schedules

Failure	Cause	Fix
A script 'succeeds' after the tool errored	Tool command printed an error and returned normally; the script carried on	Check results explicitly; exit 1 on failure; make the flow's wrapper check exit status
Works for one designer, fails for another	Relative paths, cd , or an assumption about the current directory	file normalize everything at the start; never cd in a library
Breaks when a design name contains a space or a bracket	String-built commands (Chapters 3 and 18)	Build with list and {*} ; brace every expr
Different results on re-run	Iteration over an array or a hash-ordered collection	Sort before iterating - lsort or -dictionary (Chapter 7)
Silent truncation of a report	Buffered output and an exit that skipped the flush	-buffering line on log channels, or flush before exit (Chapter 22)
A 10,000-line script nobody dares change	One file, no procedures, globals everywhere	Namespaces and packages (Chapters 14-15); a tests/ directory (Chapter 29)

28.4 Making flow scripts testable

The central difficulty is that the tool commands only exist inside the tool. The answer is the seam of Chapter 23: **separate the logic from the tool calls**, so the logic can be tested in plain **tclsh** with stubs.

```

# flowlib/timing.tcl - pure logic, no tool commands
package provide flowlib 1.0
namespace eval flowlib {
    proc classify_slack {wns setup_margin} {
        if {$wns >= $setup_margin} { return pass }
        if {$wns >= 0} { return marginal }
        return fail
    }
}

# tests/timing.test - runs in tclsh, no tool needed (Chapter 29)
test classify-1.1 {clear pass} -body {
    flowlib::classify_slack 0.5 0.1
} -result pass

# synth.tcl - the thin layer that talks to the tool
set wns [get_attribute [get_timing_paths] slack]

```

```
switch [flowlib::classify_slack $wns 0.05] {
    pass      { log "timing OK" }
    marginal  { log "WARNING: margin is thin" }
    fail      { die "timing not met: $wns" }
}
```

The decision logic - which is where the bugs are - now has tests that run in a second on any machine, with no licence checked out.

THREE THINGS WORTH DOING ON YOUR NEXT FLOW SCRIPT

Print the environment at the top: tool version, Tcl version, git commit of the scripts, hostname, date. Half of all 'it worked yesterday' investigations end the moment that block exists. **Make every script exit non-zero on failure,** and make the wrapper that calls it check. **Put the constants in one file** - clock names, corner names, paths - and source it, so a change is one edit rather than a grep.

Exercises

- 1 In your tool shell, run `info patchlevel` and `llength [info commands]`, and compare with plain `tclsh`.
- 2 Take a collection from a tool command and prove whether `llength` reports what you expect.
- 3 Add the environment-recording block to one flow script.
- 4 Extract one decision from a flow script into a pure procedure and write two `tcptest` cases for it.
- 5 Audit one script for string-built commands and unbraced `expr`, and fix what you find.

29. Testing with tcltest

`tcltest` ships with Tcl, so there is nothing to install and no excuse. It gives you named tests with setup and cleanup, expected results and expected errors, constraints for skipping, and a summary that a CI system can act on.

29.1 A test file that runs

```
package require tcltest
namespace import ::tcltest::*
configure -verbose {body pass skip error}

proc slugify {s} {
    string tolower [regsub -all {[^a-zA-Z0-9]+} [string trim $s] "-"]
}

test slugify-1.1 {basic slug} -body {
    slugify "Hello World"
} -result "hello-world"

test slugify-1.2 {punctuation collapses} -body {
    slugify "A -- B"
} -result "a-b"

test slugify-1.3 {deliberately failing} -body {
    slugify "X"
} -result "wrong"

test slugify-2.1 {error case} -body {
    error "boom"
} -returnCodes error -result "boom"

test slugify-3.1 {with setup and cleanup} -setup {
    set tmp [makeFile "content" sample.txt]
} -body {
    file exists $tmp
} -cleanup {
    removeFile sample.txt
} -result 1

cleanupTests
```

```
++++ slugify-1.1 PASSED
++++ slugify-1.2 PASSED

==== slugify-1.3 deliberately failing FAILED
==== Contents of test case:
    slugify "X"
---- Result was:
X
---- Result should have been (exact matching):
wrong
==== slugify-1.3 FAILED

++++ slugify-2.1 PASSED
++++ slugify-3.1 PASSED
v35.tcl: Total 5 Passed 4 Skipped 0 Failed 1
```

The failure report shows the test's body, what it produced and what was expected - which is the whole point of using a framework rather than a script full of `ifs`.

29.2 The options

Option	Meaning
<code>-body script</code>	The code under test; its result is compared
<code>-result value</code>	The expected result
<code>-match exact glob regexp</code>	How to compare (exact is the default)
<code>-returnCodes ok error ...</code>	Expect a particular return code - how you test that something fails correctly
<code>-errorCode pattern</code>	Expect a particular error code (Chapter 13)
<code>-setup / -cleanup</code>	Run before and after the body; cleanup runs even if the body fails
<code>-constraints name</code>	Skip unless the constraint is set - for platform-specific or slow tests
<code>-output / -errorOutput</code>	Compare what the body wrote to stdout/stderr

```
# Constraints let one suite cover several environments
testConstraint unix      [expr {$tcl_platform(platform) eq "unix"}]
testConstraint hasNetwork [expr {[info exists env(RUN_NET_TESTS)}]]

test net-1.1 {fetch a URL} -constraints hasNetwork -body {
    ...
} -result ok

# Helper files, cleaned up automatically by cleanupTests
set f [makeFile "line1\nline2\n" data.txt]
set d [makeDirectory scratch]
```

29.3 Structuring a suite

```
myproject/
  lib/
    mypkg/
      mypkg.tcl      the package (Chapter 15)
      pkgIndex.tcl
    tests/
      all.tcl        the runner
      parse.test     one file per module
      report.test

tests/all.tcl:
package require tcltest
::tcltest::configure -testdir [file dirname [info script]]
lappend auto_path [file join [file dirname [info script]] .. lib]
::tcltest::runAllTests

Run with: tclsh tests/all.tcl
In CI:    tclsh tests/all.tcl && echo PASS || exit 1
```

`runAllTests` finds every `*.test` file, runs it in a fresh interpreter, and exits non-zero if anything failed - which is exactly what a CI job needs.

29.4 Testing code that talks to the world

- **Separate the logic from the I/O**, as in Chapter 28. A pure function that takes a report's text and returns a dict is testable; one that opens the file and parses it is not, without a file.
- **Stub commands by renaming them** (Chapter 11): `rename exec _real_exec; proc exec {args} {...}` in `-setup`, and restore in `-cleanup`. This is how you test a script that shells out, without shelling out.
- **Use `makeFile` and `makeDirectory`** for tests that genuinely need the filesystem; `cleanupTests` removes them.

- **Test the failure paths.** `-returnCodes error` with an `-errorCode` pattern is how you assert that bad input is rejected - and that is where the bugs are.
- **Keep tests fast.** A suite that takes ten minutes is run once a week; one that takes ten seconds is run on every save.

THE TWO TESTS WORTH WRITING FIRST

For any script you already have: **one test that runs it end to end on a small input and compares the output**, which catches the catastrophic breakage; and **one test per bug you fix**, written before the fix, which stops the bug returning. That is a working regression suite for an afternoon's effort, and it is what makes a large flow script safe to refactor.

Exercises

- 1 Add a `tests/` directory to an existing script with three tests, and run them with `runAllTests`.
- 2 Write a test that asserts an error is raised, using `-returnCodes` and `-errorCode`.
- 3 Stub out a command that calls an external tool, and test the logic around it.
- 4 Add a constraint so that slow tests only run when an environment variable is set.
- 5 Take the last bug you fixed and write the test that would have caught it.

30. Debugging and Performance

Tcl gives you an unusual amount of visibility into a running program - every command, variable and stack frame is inspectable (Chapter 20) - and one built-in measurement command that settles most performance arguments in a minute.

30.1 Debugging without a debugger

Technique	How
Print the state	<code>puts</code> to <code>stderr</code> , so it does not pollute a report on <code>stdout</code>
Stack trace	<code>\$::errorInfo</code> after a <code>catch</code> (Chapter 13)
Where am I?	<code>info level 0, info frame -1</code> (Chapter 20)
Watch a variable	<code>trace add variable v write ...</code> (Chapter 9)
Watch a command	<code>trace add execution cmd enterstep ...</code> - a poor man's single-step (Chapter 12)
Interactive break	Insert a mini-REPL: read from <code>stdin</code> and <code>uplevel</code> what you type
Full debugger	TclPro Debugger's descendants, or the <code>tcldebug</code> package; most engineers never need one

```
# A breakpoint you can drop into any script
proc bp {{prompt "bp> "}} {
    while 1 {
        puts -nonewline stderr $prompt; flush stderr
        if {[gets stdin line] < 0} break
        if {$line in {c continue}} break
        catch {uplevel 1 $line} result
        if {$result ne ""} { puts stderr $result }
    }
}
```

Nine lines that give you an inspection prompt in the caller's scope: you can print variables, call procedures and modify state, then type `c` to continue.

30.2 Measuring, not guessing

```
set a 5; set b 7
puts " braced:    [time {expr {$a+$b}} 20000]"
puts " unbraced: [time {expr $a+$b} 20000]"
```

```
braced:    0.2212 microseconds per iteration
unbraced:  1.08805 microseconds per iteration
```

Measured: the unbraced form is **4.9x slower**, because it re-parses the expression on every call (Chapter 4). This is the cheapest performance fix in Tcl and it is a search-and-replace.

```
proc naive_join {n} {
    set s ""
    for {set i 0} {$i<$n} {incr i} { append s "x" }
    return [string length $s]
}
proc list_join {n} {
    set l {}
    for {set i 0} {$i<$n} {incr i} { lappend l "x" }
    return [string length [join $l ""]]
}
puts " append:      [time {naive_join 10000} 5]"
puts " lappend+join: [time {list_join 10000} 5]"
```

```
append:      627.0 microseconds per iteration
lappend+join: 880.2 microseconds per iteration
```

A useful corrective to folklore: for building a **string**, **append** on a variable is already amortised and beats collecting a list and joining it. The list form is for building **lists**. Measure the specific case rather than applying a remembered rule.

30.3 What is actually slow in Tcl

Cost	Why	Remedy
Shimmering	A value converted between representations in a loop (Chapter 3)	Use one type consistently; hoist conversions out of loops. This is usually the biggest single win
Unbraced <code>expr</code>, <code>if</code>, <code>while</code>	Re-parsed every iteration	Brace them - measured 4.9x above
<code>set x [expr {\$x+1}]</code>	Builds a new object each time	<code>incr x</code>
String-building a list	Quadratic re-parsing	<code>lappend</code>
<code>lsort -command</code>	A Tcl procedure call per comparison	<code>-index</code> , <code>-integer</code> , <code>-dictionary</code> , or <code>decorate-sort-undecorate</code>
Procedure call overhead	Roughly a microsecond each	Inline the hottest few; do not inline everything
Global variable access in a loop	Namespace lookup each time	Copy into a local first
Regular expressions rebuilt each iteration	Compilation is cached by pattern - but not if the pattern is built dynamically	Keep patterns literal and braced
Doing it in Tcl at all	It is an interpreter	Push the inner loop into <code>exec</code> , a C extension (Chapter 33), or the tool's own commands

LOOKING AT THE BYTECODE

`::tcl::unsupported::disassemble proc name` prints the bytecode a procedure compiles to. You will rarely need it, but it settles two questions definitively: whether an `expr` was compiled or left to run time, and whether a loop body was compiled once or is being re-parsed. Together with `tcl::unsupported::representation` (Chapter 3) it is the whole of Tcl's low-level performance toolkit.

30.4 A profiling harness in ten lines

```
# Wrap the procedures you care about and accumulate their time
proc profile {name} {
    rename $name _prof_$name
    proc $name {args} [format {
        set t0 [clock microseconds]
        try { return [_prof_%s {*}]$args } finally {
            incr ::prof(%s) [expr {[clock microseconds]-$t0}]
            incr ::profn(%s)
        }
    }]
    } $name $name $name]
}

proc profile_report {} {
    foreach {k v} [array get ::prof] {
        puts [format "%-20s %8d us  %6d calls" $k $v $::profn($k)]
    }
}
```

The rename-and-wrap idiom of Chapter 11 again. For serious work, `trace add execution ... enterstep` gives statement-level data, and Tcllib's `profiler` package does this properly.

Exercises

- 1 Time the braced and unbraced forms of an expression in your own code and report the ratio.

- 2 Find a loop that shimmers using `tcl::unsupported::representation`, fix it, and measure the improvement.
- 3 Add the `bp` breakpoint procedure to a script and inspect a variable mid-run.
- 4 Profile a slow script with the harness above and identify the top three procedures.
- 5 Disassemble a procedure with and without a braced `expr` and compare the bytecode.

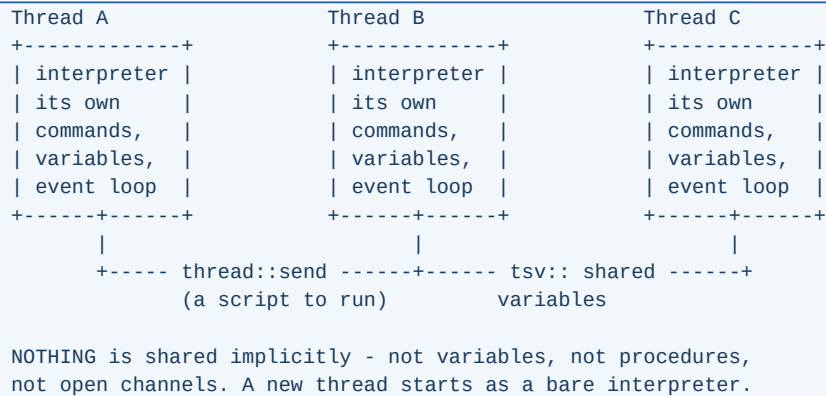
31. Threads

Tcl's threading model is unusual and, once understood, unusually easy to reason about: **each thread has its own interpreter and shares nothing**. There is no global interpreter lock because there is no shared interpreter state to lock. Communication is by message passing or through explicitly shared variables.

NOT EXECUTED

The `Thread` package was not available in the interpreter used to build this book, so - like the Tk chapter - the listings here are written from the documented API rather than captured from a run.

31.1 The model



31.2 Creating threads and sending work

```
package require Thread

# A worker with its own script; it runs an event loop and waits
set worker [thread::create {
    proc square {n} { return [expr {$n*$n}] }
    thread::wait                ;# serve requests until released
}]

# Synchronous: block until the worker answers
set answer [thread::send $worker {square 12}]

# Asynchronous: the result lands in a variable, event-loop style
thread::send -async $worker {square 20} ::result_var
vwait ::result_var

thread::release $worker
```

Command	Does
<code>thread::create ?script?</code>	Start a thread; returns its id
<code>thread::send ?-async? id script ?varName?</code>	Run a script in that thread; synchronously, or asynchronously with the result delivered into a variable
<code>thread::wait</code>	Enter the thread's event loop (a worker's main loop)
<code>thread::release id</code>	Ask a thread to exit
<code>thread::id, thread::names</code>	Introspection
<code>tsv::set/get/lappend/incr array key ?value?</code>	Thread-shared variables: a separate store with atomic operations

Command	Does
<code>tpool::create/post/wait/get</code>	A thread pool - usually what you actually want
<code>thread::mutex, thread::cond</code>	Explicit locking, if you must

31.3 A thread pool: the pattern to reach for

```
package require Thread

set pool [tpool::create -minworkers 2 -maxworkers 8 -initcmd {
    package require mypkg          ;# each worker loads what it needs
}]

set jobs {}
foreach file $files {
    lappend jobs [tpool::post $pool [list analyse_file $file]]
}
foreach job $jobs {
    tpool::wait $pool $job
    lappend results [tpool::get $pool $job]
}
tpool::release $pool
```

The `-initcmd` runs once per worker - each interpreter is empty, so every package and procedure the job needs must be loaded there. Forgetting that is the first mistake everyone makes with Tcl threads.

31.4 When threads are the right answer

Situation	Better tool
Waiting for I/O - files, sockets, subprocesses	The event loop (Chapter 24) - simpler, no shared-state hazards, and usually faster
Keeping a GUI responsive during a long computation	A thread, or after -chunking (Chapter 27)
Genuinely parallel CPU work on several cores	Threads or a thread pool - this is the case they exist for
Isolating an unreliable or blocking library	A thread, so a hang does not take the main interpreter with it
Parallel invocation of external tools	Often just <code>exec ... &</code> plus the event loop; threads add little

SHARE NOTHING, AND THE HARD PROBLEMS DO NOT ARISE

Because interpreters are isolated, the data races that dominate threaded C and Java are largely absent: you cannot accidentally share a variable. What you can still get wrong is **deadlock** - a synchronous `thread::send` to a thread that is itself waiting on you - and **cost**: every message is serialised as a string, so passing megabytes between threads is expensive. Design for a few large jobs rather than many small messages.

Exercises

- 1 Create a worker thread, send it a computation, and confirm that a procedure defined in the main thread is not visible there.
- 2 Build a thread pool that processes a list of files and collects the results in order.
- 3 Measure the cost of `thread::send` with a 1 KB argument and with a 10 MB one, and explain the difference.
- 4 Construct a deadlock with two synchronous sends, then fix it with `-async`.
- 5 Take a script that runs four tools sequentially and parallelise it - first with the event loop, then with threads - and compare the complexity of each.

32. Safe Interpreters and Security

Tcl was designed to be embedded, which meant it needed a way to run somebody else's script without giving it the machine. **Safe interpreters** are that mechanism, and they are unusually complete: not a language subset, but a full interpreter with the dangerous commands removed and a controlled way to add capabilities back.

32.1 What a safe interpreter cannot do

```
set safe [interp create -safe]
puts "  arithmetic works:  [$safe eval {expr {2+2}}]"
puts "  open a file:      [catch {$safe eval {open /etc/passwd r}} e]"
puts "  error: $e"
puts "  exec:              [catch {$safe eval {exec ls}} e2]"
puts "  error: $e2"
```

```
arithmetic works:  4
open a file:       1
error: invalid command name "open"
exec:              1
error: invalid command name "exec"
```

```
puts [lsort [$safe hidden]]
```

```
cd encoding exec exit fconfigure file glob load open pwd socket
source unload (plus tcl:file:* and tcl:encoding:* helpers)
```

Abbreviated. The commands are **hidden**, not deleted: the parent can still invoke them with `interp invokehidden`, which is how you implement a controlled version of `open` that only permits certain directories.

32.2 Giving capability back, deliberately

```
proc safe_log {which msg} { return "logged by $which: $msg" }
$safe alias log safe_log SAFE      ;# the alias adds a fixed argument
puts [$safe eval {log "hello from the sandbox"}]
```

```
logged by SAFE: hello from the sandbox
```

An **alias** is a command inside the safe interpreter that runs a procedure in the **parent**. The fixed leading arguments let the parent know which sandbox called it. This is the whole extension model: expose exactly the operations you intend, validated on the parent side.

32.3 Resource limits

```
$safe limit time -seconds [expr {[clock seconds]+1}]
set rc [catch {$safe eval {while 1 {}}} err]
puts "  runaway loop stopped: rc=$rc err=$err"
```

```
runaway loop stopped: rc=1 err=time limit exceeded
```

`interp limit` bounds wall-clock time and recursion depth, with a callback when the limit is hit. Together with hidden commands this makes it practical to run a user's expression, plug-in or configuration script without trusting it.

Mechanism	Purpose
<code>interp create -safe</code>	A sandboxed child interpreter
<code>interp hide</code> / <code>interp expose</code>	Move commands in and out of reach
<code>interp invokehidden</code>	The parent calls a hidden command on the child's behalf
<code>\$i alias name cmd ?args?</code>	Expose a parent procedure as a child command
<code>interp limit \$i time command</code>	Bound run time and depth
<code>::safe::interpCreate</code>	The Safe Base: a safe interpreter plus a virtualised, restricted <code>source</code> and <code>load</code> so it can use packages from directories you nominate

32.4 The threats in ordinary scripts

Risk	How it happens	Defence
Command injection	Building a command by string concatenation from data (Chapters 3, 18)	<code>list + {*}</code> ; never <code>eval "cmd \$data"</code>
Expression injection	<code>expr \$userinput</code>	Always brace: <code>expr {...}</code>
Template execution	<code>subst</code> on text you did not write	<code>string map</code> , or <code>subst -nocommands -novariables</code>
Shell injection	<code>exec sh -c "tool \$arg"</code>	<code>exec tool \$arg</code> - no shell is involved (Chapter 23)
Path traversal	Joining a user-supplied name onto a directory	<code>file normalize</code> , then verify the result is still inside the intended root
Regex denial of service	A pattern built from input	<code>***=</code> literal prefix, or <code>string match</code> (Chapter 10)
Secrets in scripts	Passwords in Expect scripts and config files (Chapter 26)	Read at run time from a protected file or a secrets service; <code>log_user 0</code>
Untrusted <code>`source`</code>	Sourcing a file from a shared area	A safe interpreter, or verify the file's provenance

THE ONE-LINE SUMMARY OF TCL SECURITY

Data must never become code by accident. Every vulnerability above is the same mistake: a value that should have been an argument was concatenated into something that gets parsed - a command, an expression, a template, a shell line, a pattern. Tcl gives you exact tools to keep the boundary - `list`, `{*}`, braces, `string map` - and they cost nothing.

Exercises

- 1 Create a safe interpreter and try to read a file, run a command and exit from inside it.
- 2 Give it a controlled `read_config` alias that only allows paths under one directory, and try to escape with `...`
- 3 Set a time limit and prove a runaway loop is stopped.
- 4 Find one place in your own scripts where data is concatenated into a command, and fix it with `list`.
- 5 Write a plug-in loader that runs third-party scripts in a safe interpreter with three aliases.

33. Embedding and Extending Tcl in C

Tcl was built to be embedded, and the API shows it: two calls give a C program a complete scripting language, and one call adds a command written in C. This is how every EDA tool in Chapter 28 got its interpreter, and how you make a slow inner loop fast.

33.1 Embedding: a scriptable application in thirty lines

```
#include <tcl.h>
#include <stdio.h>

/* A command implemented in C, callable from the script */
static int AppStatus_Cmd(ClientData cd, Tcl_Interp *interp,
                        int objc, Tcl_Obj *const objv[])
{
    Tcl_SetObjResult(interp,
        Tcl_NewStringObj("application is healthy", -1));
    return TCL_OK;
}

int main(int argc, char **argv)
{
    Tcl_Interp *interp;
    Tcl_FindExecutable(argv[0]);          /* required first */
    interp = Tcl_CreateInterp();
    if (Tcl_Init(interp) != TCL_OK) { return 1; }

    Tcl_CreateObjCommand(interp, "app_status", AppStatus_Cmd, NULL, NULL);
    Tcl_SetVar(interp, "app_version", "3.1.4", TCL_GLOBAL_ONLY);

    if (Tcl_Eval(interp, "puts \"embedded Tcl [info patchlevel]\";"
        "puts \"version var: $app_version\";"
        "puts \"custom command: [app_status]\";"
        "expr {6*7}") != TCL_OK) { return 1; }
    printf("script result: %s\n", Tcl_GetStringResult(interp));
    Tcl_DeleteInterp(interp);
    return 0;
}
```

```
gcc -I/usr/include/tcl8.6 -o embed embed.c -ltcl8.6
./embed
```

```
embedded Tcl 8.6.14
version var: 3.1.4
custom command: application is healthy
script result: 42
```

Compiled and run while writing this chapter. That is the entire embedding story: create, initialise, add your commands and variables, evaluate, read the result.

33.2 Extending: a command written in C

```
#include <tcl.h>

static int Sum_Cmd(ClientData cd, Tcl_Interp *interp,
                  int objc, Tcl_Obj *const objv[])
{
    Tcl_WideInt total = 0, v;
    int i;
    if (objc < 2) {                          /* argument checking */
```

```

        Tcl_WrongNumArgs(interp, 1, objv, "number ?number ...?");
        return TCL_ERROR;
    }
    for (i = 1; i < objc; i++) {
        if (Tcl_GetWideIntFromObj(interp, objv[i], &v) != TCL_OK) {
            return TCL_ERROR;          /* message already set */
        }
        total += v;
    }
    Tcl_SetObjResult(interp, Tcl_NewWideIntObj(total));
    return TCL_OK;
}

int Demo_Init(Tcl_Interp *interp)          /* name matters: Demo_Init */
{
    if (Tcl_InitStubs(interp, "8.6", 0) == NULL) return TCL_ERROR;
    Tcl_CreateObjCommand(interp, "csum", Sum_Cmd, NULL, NULL);
    return Tcl_PkgProvide(interp, "demo", "1.0");
}

```

```

gcc -fPIC -shared -I/usr/include/tcl8.6 -DUSE_TCL_STUBS \
    -o libdemo.so hello.c -ltclstub8.6

```

```

# and from Tcl:
load ./libdemo.so demo
puts "csum 1 2 3 = [csum 1 2 3]"
puts "csum big    = [csum 1000000000 2000000000]"
puts "error case  = [catch {csum abc} e]/$e"
puts "wrong args  = [catch {csum} e2]/$e2"

```

```

csum 1 2 3 = 6
csum big    = 3000000000
error case  = 1/expected integer but got "abc"
wrong args  = 1/wrong # args: should be "csum number ?number ...?"

```

Also compiled and run for this chapter. Note how much you get for free: type conversion with a correct error message, and `Tcl_WrongNumArgs` producing the standard usage text.

33.3 The API in one page

Function	Purpose
<code>Tcl_CreateInterp</code> / <code>Tcl_DeleteInterp</code>	Interpreter lifetime
<code>Tcl_Init</code>	Initialise the standard library
<code>Tcl_Eval</code> / <code>Tcl_EvalObjv</code> / <code>Tcl_EvalFile</code>	Run a script, a pre-built command, or a file
<code>Tcl_CreateObjCommand</code>	Add a command; the <code>Obj</code> form is the modern one
<code>Tcl_SetObjResult</code> / <code>Tcl_GetStringResult</code>	Return a value / read one
<code>Tcl_NewIntObj</code> <code>Tcl_NewStringObj</code> <code>Tcl_NewListObj</code> <code>Tcl_NewDictObj</code>	Build values
<code>Tcl_GetIntFromObj</code> <code>Tcl_GetStringFromObj</code> <code>Tcl_ListObjGetElements</code>	Read values, with error messages
<code>Tcl_IncrRefCount</code> / <code>Tcl_DecrRefCount</code>	Reference counting - the one thing you must get right
<code>Tcl_SetVar</code> / <code>Tcl_GetVar</code> / <code>Tcl_LinkVar</code>	Variables; <code>Tcl_LinkVar</code> ties a Tcl variable to a C variable
<code>Tcl_CreateChannel</code> , <code>Tcl_CreateEventSource</code>	Add channel types and event sources

Function	Purpose
<code>Tcl_InitStubs</code>	Build against the stubs library so one binary works with any compatible Tcl

REFERENCE COUNTING IS THE WHOLE MEMORY MODEL

Every `Tcl_Obj` carries a reference count. A value you create has count zero and is owned by nobody; passing it to `Tcl_SetObjResult` or `Tcl_ListObjAppendElement` transfers ownership. If you keep a pointer to a value across a call, `Tcl_IncrRefCount` it and `Tcl-DecrRefCount` when done. **Almost every crash in a Tcl extension is a missing or extra `Tcl-DecrRefCount`**, and almost every leak is a missing one.

33.4 Easier routes to the same place

Tool	What it does	When
critcl	Embed C directly in a Tcl script; it compiles and caches on first use	The fastest way to accelerate one procedure - no build system at all
SWIG	Generate bindings from a C or C++ header	Wrapping an existing library with many functions
TEA	The standard autoconf-based extension build framework	Extensions you will distribute across platforms
exec a helper program	No API at all	Often the right answer: if the work is coarse-grained, a subprocess is simpler than an extension and cannot crash your interpreter

```
# critcl: C inside a Tcl file
package require critcl
critcl::cproc fastsum {int a int b} int { return a + b; }
puts [fastsum 20 22]
```

Illustrative - critcl was not installed in the interpreter used for this book. It compiles the function on first use and caches the result, so the script stays a script.

MEASURE BEFORE YOU WRITE C

The usual reason to reach for C is a slow inner loop - but Chapter 30's list of Tcl-level costs (shimmering, unbraced `expr`, string-built lists) accounts for most slow Tcl, and fixing those is hours rather than weeks. Write the C extension when profiling shows the time is genuinely in computation you cannot express efficiently in Tcl - image processing, numerical kernels, bit manipulation over megabytes - and not before.

Exercises

- 1 Compile and run the embedding example, then add a C command that returns the process's memory usage.
- 2 Build the `csum` extension and confirm the error messages come out as shown.
- 3 Add a command that takes a Tcl list and returns its sum, using `Tcl_ListObjGetElements`.
- 4 Use `Tcl_LinkVar` to expose a C variable and change it from a script.
- 5 Take a slow Tcl procedure, optimise it in Tcl first, then rewrite it in C, and compare the two speedups against the effort each cost.

PART VII

Practice

The idioms experienced Tcl programmers use and the ones they avoid, the anatomy of a script that runs unattended, versions and deployment, a cookbook of the tasks that come up constantly, and one complete project built from an empty file.

34. Idioms, Style and Pitfalls

This chapter collects the habits that separate Tcl that survives maintenance from Tcl that does not. Most of them are consequences of Chapters 2 and 3 - quoting and the string model - applied consistently.

34.1 The rules that prevent most bugs

Non-negotiable habits

- ☐ **Brace every ``expr``, ``if``, ``while`` and ``for`` condition.** Faster and safe (Chapters 4, 32).
- ☐ **Build commands and lists with ``list`` and ``lappend``, invoke with ``{*}``.** Never with string concatenation (Chapters 7, 18).
- ☐ **Brace every regular expression.** Otherwise Tcl eats the backslashes (Chapter 10).
- ☐ **Use ``eq`/`ne`` for strings and ``==`/`!=`` for numbers.**
- ☐ **Pass ``-strict`` to ``string is``.**
- ☐ **Pass ``-nocomplain`` to ``glob``, and ``--`` before any value that could start with a dash.**
- ☐ **Close what you open, in a ``finally``.**
- ☐ **Return a value; do not print it.** A procedure that `puts` cannot be reused or tested.
- ☐ **Name namespace variables with ``variable``, not ``global``.**
- ☐ **Exit non-zero when the script fails.**

34.2 Idioms worth knowing

```
# Default values for an options dict
proc configure {args} {
    set opts [dict merge {-verbose 0 -retries 3} $args]
    return [dict get $opts -retries]
}
puts [configure -retries 5]
puts [configure]

# Safe "get with default"
proc get {d key {default ""}} {
    expr [[dict exists $d $key] ? [dict get $d $key] : $default]
}
puts "[get {a 1} a] / [get {a 1} b MISSING]"

# Building a command that survives spaces
set cmd [list echo -n "my file.txt"]
puts "cmd = $cmd (llength [llength $cmd])"
```

```
5
3
1 / MISSING
cmd = echo -n {my file.txt} (llength 3)
```

```
# The K combinator: return the first argument, evaluate the second.
# Used to take a value from a variable while releasing the variable,
# so the value is not shared and can be modified in place.
proc K {a b} { return $a }

set big [lrepeat 5 x]
set first [K [lindex $big 0] [set big {}]]
puts "took '$first', list is now '[set big]'"
```

```
took 'x', list is now ''
```

In Tcl 8.6 `lassign $list` and the improved copy-on-write make `K` much less necessary than it was, but you will meet it in older code and in performance-critical library sources - now you know what it is.

34.3 Pitfalls, collected

Pitfall	Symptom	Chapter
Unbraced <code>expr</code> with user data	Injection, or 5x slower	4, 32
<code>string trimright \$f ".log"</code>	<code>catalog.log</code> becomes <code>cata</code>	6
<code>string is integer \$x</code> without <code>-strict</code>	Empty string passes validation	6
Building a list with <code>append</code>	Elements merge; <code>llength</code> wrong	3, 7
<code>lsearch</code> assumed exact	Wildcards match unexpectedly	7
<code>while {[eof \$f]}</code>	One spurious empty record at the end	21
<code>array get</code> iteration order	Non-deterministic reports	8, 9
Missing <code>-nocomplain</code> on <code>glob</code>	Error instead of empty result	21
<code>exec</code> treated as returning a status	Errors on stderr output; exit code hidden	23
Callback errors with no <code>bgerror</code>	Failures vanish silently	24
<code>catch</code> that discards the error	The real failure is invisible	13
Shimmering in a loop	Thousandfold slowdowns	3, 30
<code>global</code> in a namespaced procedure	State silently not shared	14
Comment with an unbalanced brace	<code>missing close-brace</code> far from the cause	2

34.4 Style

- **Indent four spaces; braces on the same line** as the command - Chapter 5 shows it is not optional for `if/else`.
- **One command per line.** Semicolons only for a trailing comment.
- **Namespace everything** that is not a script's own top level; prefix nothing with `my_`.
- **Name procedures with verbs** (`parse_report`, `write_config`) and variables with nouns; use `snake_case` consistently, since Tcl's own commands are lower case.
- **Comment why, not what.** `# collect before sorting: the tool returns arbitrary order` is worth ten lines of restated code.
- **Keep procedures short enough to see whole**, and give each one an explicit `return`.
- **Put the ``package require`` lines at the top**, and nothing executable at file scope in a library.

READING UNFAMILIAR TCL QUICKLY

Three questions answer most of it. **Where does substitution happen?** - scan for `"` versus `{`. **Is this value a list, a string or a dict?** - look at which commands touch it. **What is this command?** - `info body`, `namespace which`, or the tool's own help. A large flow script that looks impenetrable usually becomes clear once you have answered those three for its main loop.

Exercises

- 1 Audit one of your scripts against the ten non-negotiable habits and fix every violation.
- 2 Find an unbraced `expr` in existing code and construct an input that makes it misbehave.
- 3 Rewrite a procedure that prints its result so that it returns it instead, and update the callers.
- 4 Take the pitfalls table and grep your codebase for three of them.
- 5 Write down your team's Tcl style in one page, and make it a review checklist.

35. The Anatomy of a Production Script

A script that runs unattended - in a nightly regression, a CI job or a tapeout flow - needs more than working logic. This chapter is the skeleton, in the order the parts appear in the file.

35.1 The skeleton

```
#!/usr/bin/env tclsh
# analyse.tcl - summarise tool logs. See --help for usage.

#--- 1. Requirements, stated and checked -----
package require Tcl 8.6

#--- 2. Metadata, so a log identifies its own source -----
set VERSION "1.4.0"
set SCRIPT [file normalize [info script]]

#--- 3. Defaults, overridable by options and the environment -----
set opts [dict create \
    -verbose 0 \
    -output "" \
    -pattern {ERROR|FATAL} \
    -jobs 1]

#--- 4. Utilities: logging that goes to stderr, timestamped -----
proc log {level msg} {
    if {$level eq "debug" && ![dict get $::opts -verbose]} return
    set ts [clock format [clock seconds] -format "%H:%M:%S"]
    puts stderr [format "%s %-5s %s" $ts [string toupper $level] $msg]
}
proc die {msg {code 1}} { log error $msg; exit $code }

#--- 5. Argument parsing, with --help -----
proc usage {} {
    puts "usage: [file tail $::SCRIPT] \[options\] logfile..."
    puts "  -verbose      more detail on stderr"
    puts "  -output FILE  write the summary here (default stdout)"
    puts "  -pattern REGEXP lines to report (default ERROR|FATAL)"
    exit 0
}
set files {}
for {set i 0} {$i < [llength $::argv]} {incr i} {
    set a [lindex $::argv $i]
    switch -glob -- $a {
        --help - -h { usage }
        -verbose { dict set opts -verbose 1 }
        -output { dict set opts -output [lindex $::argv [incr i]] }
        -pattern { dict set opts -pattern [lindex $::argv [incr i]] }
        -- { lappend files {*}[lrange $::argv [expr {$i+1}] end]
            break }
        -* { die "unknown option: $a" 2 }
        default { lappend files $a }
    }
}
if {![llength $files]} { die "no input files (try --help)" 2 }
```

```
#--- 6. The work, as testable procedures -----
proc scan_file {path pattern} {
    set hits {}
```

```

    set fh [open $path r]
    try {
        set n 0
        while {[gets $fh line] >= 0} {
            incr n
            if {[regexp -- $pattern $line]} { lappend hits [list $n $line] }
        }
    } finally { close $fh }
    return $hits
}

#--- 7. main, with real error handling and an exit status -----
proc main {files opts} {
    set total 0
    set out stdout
    if {[dict get $opts -output] ne ""} {
        set out [open [dict get $opts -output] w]
    }
    try {
        foreach f $files {
            if {![file readable $f]} { log warn "skipping $f"; continue }
            set hits [scan_file $f [dict get $opts -pattern]]
            incr total [llength $hits]
            foreach h $hits { puts $out "[file tail $f]:[lindex $h 0]: [lindex $h 1]" }
        }
    } finally {
        if {$out ne "stdout"} { close $out }
    }
    log info "$total matches in [llength $files] files"
    return [expr {$total > 0 ? 1 : 0}]      ;# exit status carries meaning
}

if {[info exists ::argv0] && [file normalize $::argv0] eq $SCRIPT} {
    exit [main $files $opts]              ;# only when run, not sourced
}

```

The last three lines are the Tcl equivalent of Python's `if __name__ == "__main__":`: the file can be **sourced by a test** (Chapter 29) without running anything.

35.2 What each part buys

Part	Why it matters in an unattended run
Version and script path in the log	Answers 'which script produced this output?' three months later
Logging to stderr	Keeps stdout clean so the real output can be piped or redirected
Timestamps	Turns a log into a timeline when something hangs
--help and option validation	A typo in a cron entry fails immediately with a message, not silently with wrong defaults
Everything in procedures	Testable (Chapter 29), and the file can be sourced
try/finally around resources	Output files are closed and complete even on failure
A meaningful exit status	The calling flow, CI job or Makefile can branch on it
The argv0 guard	The same file is both a program and a library

TWO LINES THAT SAVE THE MOST DEBUGGING TIME

At the very top of `main`, log the **version, the arguments and the working directory**; at the very end, log the **result and the elapsed time**. Almost every question asked afterwards about an unattended run - what was it given, where did it run, how long did it take, what did it decide - is answered by those two lines, and neither costs anything.

Exercises

- 1 Take an existing script and add option parsing with `--help`.
- 2 Add the `argv0` guard and write a test file that sources the script and tests one of its procedures.
- 3 Make every failure path exit non-zero, and verify with `echo $?`.
- 4 Move all logging to stderr and confirm the script's stdout can be piped cleanly.
- 5 Add the two logging lines above and run the script under `time`.

36. Versions, Portability and Deployment

Tcl's compatibility record is unusually good - scripts from the 1990s mostly still run - but the version you get inside a vendor tool is rarely the newest, and the features you may use are decided by the oldest interpreter your script must support.

36.1 What arrived when

Version	Brought
8.0	The bytecode compiler and <code>Tcl_Obj</code> - the reason Tcl is not slow (Chapter 3)
8.4	<code>file</code> improvements, virtual filesystems, <code>Tcl_LinkVar</code> extensions
8.5	<code>`dict`, <code>{*}</code>, <code>lassign</code>, <code>apply</code>, arbitrary-precision integers, <code>chan</code>, decimal-by-default numbers, <code>lsearch -index</code></code>
8.6	<code>TclOO</code> , <code>coroutines</code> , <code>`try`/`throw`</code> , <code>lmap</code> , <code>zlib</code> , IPv6, non-recursive evaluation (deep recursion without a C-stack limit), <code>binary</code> improvements
8.7 / 9.0	Further <code>string/dict</code> subcommands (<code>dict getwithdefault</code>), better encodings and 64-bit string lengths, <code>TIP 445</code> value handling, and in 9.0 a cleaned-up C API with some source-level breakage for extensions

```
puts "patchlevel: [info patchlevel]"
puts "8.6+: [package vsatisfies [info patchlevel] 8.6]"
foreach c {dict try lmap coroutine} {
    puts "  has $c: [expr {[llength [info commands $c]] > 0}]"
}
```

```
patchlevel: 8.6.14
8.6+: 1
  has dict: 1
  has try: 1
  has lmap: 1
  has coroutine: 1
```

Two ways to test: `package vsatisfies` for a version rule, and `info commands` for the specific feature. Prefer the second - it keeps working when a feature is back-ported or provided by a package.

WRITING FOR AN OLDER TOOL INTERPRETER

If a vendor shell has 8.5, the practical losses are `try` (use `catch` with the options `dict`), `TclOO` (use namespaces, or a bundled pure-Tcl OO package), `coroutines` (restructure as callbacks), and `lmap` (use `foreach` with `lappend`). Everything else in this book applies unchanged. Put a `package require Tcl 8.5` at the top so the failure is a clear message rather than a mysterious error 200 lines in.

36.2 Portability

Concern	Do this
Paths	<code>file join</code> , <code>file normalize</code> , <code>file nativename</code> - never string concatenation (Chapter 21)
Line endings	<code>fconfigure -translation -auto</code> on input, <code>lf</code> or <code>crlf</code> deliberately on output (Chapter 22)
Encodings	Set <code>-encoding</code> explicitly; do not assume the system encoding is UTF-8 - on the machine that built this book it is <code>iso8859-1</code>
External commands	<code>auto_execok</code> , and check <code>\$tcl_platform(platform)</code> before assuming a Unix tool exists
Case sensitivity	Windows filenames are case-insensitive; Unix ones are not

Concern	Do this
Environment	<code>env</code> names differ (<code>HOME</code> versus <code>USERPROFILE</code>)
Threads	<code>\$tcl_platform(threaded)</code> says whether this build supports them (Chapter 31)

```
puts "$tcl_platform(platform) / $tcl_platform(os) /
      threaded=$tcl_platform(threaded)"
puts "system encoding: [encoding system]"
puts [clock format 0 -format "%Y-%m-%dT%H:%M:%SZ" -gmt 1]
```

```
unix / Linux / threaded=1
system encoding: iso8859-1
1970-01-01T00:00:00Z
```

Always pass `-gmt 1` (or an explicit `-timezone`) when a timestamp will be compared or stored; local time is a presentation choice, not a storage format.

36.3 Shipping a Tcl program

Method	What it produces	When
A directory of scripts plus <code>package require</code>	The normal case	Internal tools where Tcl is already installed
A <code>.tm</code> module or a package directory	One installable unit (Chapter 15)	Libraries
Starkit (<code>.kit</code>)	A single file containing scripts and packages in a virtual filesystem, run by a <code>tclkit</code> runtime	Distributing a tool without an installer
Starpack	A starkit plus the interpreter - a single executable with no dependencies	Giving a tool to someone who has no Tcl
A container image	Everything pinned	CI and reproducible flows

```
# Building a starkit/starpack with sdx (sketch)
sdx qwrap myapp.tcl           ;# wrap the script into myapp.kit
sdx unwrap myapp.kit          ;# open it as a directory
# ... add lib/ packages into myapp.vfs/lib ...
sdx wrap myapp.kit
sdx wrap myapp -runtime tclkit-linux-x86_64 ;# single executable
```

Illustrative - `sdx` is a separate tool and was not run here. The starkit format remains the simplest way to hand a colleague a Tcl program that just runs.

Before you ship

- ☐ `package require Tcl <minimum>` at the top of every entry point.
- ☐ No absolute paths to your own machine; everything relative to `[info script]`.
- ☐ Version number reported by `--version` and in the log.
- ☐ Tests pass on the **oldest** interpreter you support, not only yours.
- ☐ Encoding and line endings set explicitly wherever files are read or written.
- ☐ The script runs correctly when its path contains a space.
- ☐ Exit statuses documented, and non-zero on failure.

Exercises

- Print the Tcl version of every interpreter you have access to - your shell, your EDA tools, your CI image - and note the oldest.
- Take a script that uses `try` and rewrite it for 8.5 with `catch`.
- Run a script from a directory whose name contains a space and fix whatever breaks.
- Add a `-version` option that prints the script version and the interpreter version.

- 5 Package a small tool as a starkit if you have `sdx`, or as a `.tm` module if not.

37. Cookbook

The tasks that come up in nearly every script, with a tested recipe for each. All of these were run to produce the output shown.

37.1 Files

```
proc read_lines {path} {
    set fh [open $path r]
    try {
        return [split [string trimright [read $fh] "\n"] "\n"]
    } finally { close $fh }
}

proc write_file {path content} {
    set fh [open $path w]
    try { puts -nonewline $fh $content } finally { close $fh }
}
```

`string trimright` removes the final newline so the last element is not an empty string - the small detail that makes `llength` correct.

37.2 Parse a key = value configuration file

```
proc parse_config {path} {
    set d [dict create]
    foreach line [read_lines $path] {
        set line [string trim $line]
        if {$line eq "" || [string index $line 0] eq "#"} continue
        if {[regexp {[^=]+?)\s*=\s*(.*)$} $line -> k v]} {
            dict set d [string trim $k] [string trim $v]
        }
    }
    return $d
}

puts [parse_config cfg.txt]
```

```
width 32 name top depth 1024
```

Input: a file with a comment, a blank line, and both `width = 32` and `name=top` spacing styles.

37.3 Count and rank

```
set text "the quick brown fox jumps over the lazy dog the fox"
set freq [dict create]
foreach w [split $text] { dict incr freq $w }

set pairs {}
dict for {w n} $freq { lappend pairs [list $w $n] }
puts [lrange [lsort -index 1 -integer -decreasing $pairs] 0 2]
```

```
{the 3} {fox 2} {quick 1}
```

37.4 Unique, preserving order

```

proc uniq {l} {
    set seen {}; set out {}
    foreach x $l {
        if {[dict exists $seen $x]} { dict set seen $x 1; lappend out $x }
    }
    return $out
}
puts [uniq {b a b c a d}]

```

```
b a c d
```

`lsort -unique` also removes duplicates but sorts; this keeps the original order, which is usually what a report wants.

37.5 Walk a directory tree

```

proc find_files {dir pattern} {
    set out {}
    foreach f [glob -nocomplain -directory $dir *] {
        if {[file isdirectory $f]} {
            lappend out {[find_files $f $pattern]}
        } elseif {[string match $pattern [file tail $f]]} {
            lappend out $f
        }
    }
    return $out
}
puts [lsort [find_files . *.txt]]

```

```
./cfg.txt ./sub/deeper/two.txt ./sub/one.txt
```

Note the `} elseif {` on one line - putting it on the next produces `invalid command name "elseif"`, which is Chapter 5's rule biting in the wild. It bit while this recipe was being tested.

37.6 Retry with exponential backoff

```

proc retry {attempts script} {
    for {set i 1} {$i <= $attempts} {incr i} {
        if {[catch {uplevel 1 $script} result]} { return $result }
        if {$i < $attempts} { after [expr {50 * (1 < ($i-1))}] }
    }
    return -code error "failed after $attempts attempts: $result"
}

set ::n 0
puts [retry 5 {
    incr ::n
    if {$::n < 3} { error "not yet" }
    string cat "ok on attempt $::n"
}]

```

```
ok on attempt 3
```

`uplevel 1` runs the script in the caller's scope (Chapter 12), so the retried block can use the caller's variables.

37.7 A formatted table

```

set rows {{clk 100.0 pass} {rst 50.5 fail} {data_bus 12.25 pass}}
puts [format "%-12s %8s  %s" NAME FREQ STATUS]
foreach r $rows { puts [format "%-12s %8.2f  %s" {*} $r] }

```

NAME	FREQ	STATUS
clk	100.00	pass
rst	50.50	fail
data_bus	12.25	pass

`{*}$r` expands the row into `format`'s arguments - the idiom of Chapter 18 doing everyday work.

37.8 Temporary files, dates and randomness

```
set tf [file tempfile path]      ;# 8.6+: opens AND gives the name
puts $tf "temporary"
close $tf
puts "created: [file exists $path]"
file delete $path
```

```
expr {srand(42)}                ;# reproducible sequences for tests
puts "random: [expr {int(rand()*100)}] [expr {int(rand()*100)}]"
```

```
created: 1
random: 52 73
```

```
set now [clock seconds]
clock format $now -format "%Y-%m-%d %H:%M:%S" -gmt 1
clock scan "2023-11-14 22:13:20" -format "%Y-%m-%d %H:%M:%S" -gmt 1
clock add $now 1 day
clock format $now -format "%Y-%m-%dT%H:%M:%SZ" -gmt 1 ;# ISO 8601
```

Elapsed time in a script

```
set t0 [clock milliseconds]
```

... work ...

```
puts "took [expr {[clock milliseconds]-$t0}] ms"
```

37.9 Ten one-liners worth memorising

Task	Idiom
Read a whole file	<code>set data [read [set f [open \$p]]][close \$f]</code> - or the <code>read_lines</code> procedure above, which is clearer
Number of lines	<code>llength [split \$text \n]</code>
Strip a suffix	<code>file rootname \$name</code>
Join a path	<code>file join \$dir \$name</code>
Does a list contain x?	<code>expr {\$x in \$list}</code>
Remove an element	<code>set l [lsearch -all -inline -not -exact \$l \$x]</code>
Sum a list	<code>::tcl::mathop::+ {*} \$list</code>
Reverse a dict	<code>dict for {k v} \$d {dict set inv \$v \$k}</code>
Sort by a field	<code>lsort -index 1 -real \$rows</code>
Repeat a string	<code>string repeat - 40</code>

```
puts [::tcl::mathop::+ 1 2 3 4 5]
puts [::tcl::mathop::* {*} [list 2 3 7]]
puts [expr {"clk" in {clk rst data}}]
```

```
15
42
1
```

`::tcl::mathop` exposes every `expr` operator as a command, which makes `{*}` summation and `lmap`-style pipelines possible without writing a loop.

Exercises

- 1 Adapt `parse_config` to support sections in square brackets.
- 2 Extend `find_files` to skip directories matching a pattern.
- 3 Add a maximum total time to `retry`, not just a maximum number of attempts.
- 4 Write a recipe of your own for something your team does weekly, and put it in a package (Chapter 15).
- 5 Replace three ad-hoc loops in your code with `dict incr`, `lsort -index` and `{*}`.

38. A Complete Project, Stage by Stage

This chapter builds one small tool from an empty directory to a packaged, tested program with a working test suite - using the techniques of every previous chapter. It was built and run while this book was written, including the bug it hit on the way.

38.1 Stage 0 - The requirement

`logmine`: scan tool log files, classify each line as an error, a warning or neither, print a per-file summary with line numbers, and exit non-zero if any errors were found - so a flow can branch on it (Chapter 28). Options for verbose logging and for a quiet, summary-only mode.

38.2 Stage 1 - The layout

```
project/
  logmine.tcl           the command-line program
  lib/
    logmine/
      logmine.tcl       the package: all the logic
      pkgIndex.tcl
    tests/
      all.tcl           the runner
      logmine.test      the tests
  sample.log  clean.log  fixtures
```

The logic lives in a package (Chapter 15) so it can be tested without the command line, and the program is a thin shell around it (Chapters 28, 35).

38.3 Stage 2 - The package

```
package require Tcl 8.6
package provide logmine 1.0

namespace eval ::logmine {
  namespace export classify scan_text summarise format_report
  namespace ensemble create          ;# gives "logmine classify ..."

  variable PATTERNS {
    error  {(?i)^\s*(error|fatal)\y}
    warning {(?i)^\s*warn(ing)?\y}
  }

  proc classify {line} {
    variable PATTERNS
    dict for {sev re} $PATTERNS {
      if {[regexp -- $re $line]} { return $sev }
    }
    return info
  }

  proc scan_text {text} {
    set hits {} ; set n 0
    foreach line [split $text \n] {
      incr n
      set sev [classify $line]
      if {$sev ne "info"} {
        lappend hits [list $n $sev [string trim $line]]
      }
    }
    return $hits
  }
```

```

}

proc summarise {hits} {
    set counts [dict create error 0 warning 0]
    foreach h $hits { dict incr counts [lindex $h 1] }
    return $counts
}

proc format_report {file hits counts} {
    set out {}
    lappend out [format "%-30s errors:%-4d warnings:%-4d" \
                        [file tail $file] [dict get $counts error] \
                        [dict get $counts warning]]
    foreach h $hits {
        lassign $h line sev text
        lappend out [format " %5d %-8s %s" $line $sev $text]
    }
    return [join $out \n]
}
}

```

Four pure procedures - no files opened, nothing printed - plus an ensemble front end. Every one is testable in isolation.

38.4 Stage 3 - The tests, written alongside

```

package require tcltest
namespace import ::tcltest::*
lappend auto_path [file join [file dirname [info script]] .. lib]
package require logmine 1.0

test classify-1.1 {error line} -body {logmine classify "ERROR: bad"} -result error
test classify-1.2 {fatal line} -body {logmine classify " Fatal: x"} -result error
test classify-1.3 {warning} -body {logmine classify "WARN: hmm"} -result warning
test classify-1.4 {plain line} -body {logmine classify "elaborating"} -result info
test classify-1.5 {not at start} -body {logmine classify "no error here"} -result info

test scan-2.1 {finds line numbers} -body {
    logmine scan_text "ok\nERROR: bad\nWARN: hmm"
} -result {{2 error {ERROR: bad}} {3 warning {WARN: hmm}}}

test summarise-3.1 {counts} -body {
    logmine summarise [logmine scan_text "ERROR: a\nERROR: b\nWARN: c"]
} -result {error 2 warning 1}

cleanupTests

```

THE BUG THIS PROJECT ACTUALLY HIT

The first version of **PATTERNS** used `\b` for a word boundary - the Perl and **grep** spelling. **Tcl's regular expressions use `\y``** (`\m` and `\M` for the start and end of a word); `\b` means backspace. Every classification silently returned **info**, the tool reported zero errors on a file full of them, and it exited 0 - the worst possible failure for a flow that branches on the exit status. **Five of the seven tests caught it immediately**. That is the entire argument for Chapter 29 in one incident, and it is why the tests were written before the tool was run.

```
tclsh tests/all.tcl
```

```

all.tcl: Total 7 Passed 7 Skipped 0 Failed 0
Sourced 1 Test Files.

```

After the fix. Before it: **Total 7 Passed 2 Failed 5**.

38.5 Stage 4 - The command-line program

```

#!/usr/bin/env tclsh
package require Tcl 8.6
set SCRIPT [file normalize [info script]]
lappend auto_path [file join [file dirname $SCRIPT] lib]
package require logmine 1.0

set opts [dict create -verbose 0 -quiet 0]

proc log {level msg} {
    if {$level eq "debug" && ![dict get $::opts -verbose]} return
    puts stderr [format "%s %-5s %s" \
        [clock format [clock seconds] -format %H:%M:%S] \
        [string toupper $level] $msg]
}

proc die {msg {code 2}} { log error $msg; exit $code }

set files {}
for {set i 0} {$i < [llength $argv]} {incr i} {
    set a [lindex $argv $i]
    switch -glob -- $a {
        -h - --help { usage }
        -verbose     { dict set opts -verbose 1 }
        -quiet       { dict set opts -quiet 1 }
        -*           { die "unknown option: $a" }
        default      { lappend files $a }
    }
}

if {![llength $files]} { die "no input files (try --help)" }

proc main {files opts} {
    set total_errors 0
    foreach f $files {
        if {![file readable $f]} { log warn "skipping unreadable $f"; continue }
        log debug "scanning $f"
        set fh [open $f r]
        try { set text [read $fh] } finally { close $fh }
        set hits [logmine scan_text $text]
        set counts [logmine summarise $hits]
        incr total_errors [dict get $counts error]
        if {![dict get $opts -quiet]} {
            puts [logmine format_report $f $hits $counts]
        }
    }
    log info "total errors: $total_errors"
    return [expr {$total_errors > 0 ? 1 : 0}]
}

if {[file normalize $argv0] eq $SCRIPT} { exit [main $files $opts] }

```

38.6 Stage 5 - Running it

```
tclsh logmine.tcl sample.log clean.log ; echo "exit=$?"
```



```

sample.log                errors:2    warnings:1
    2 error    ERROR: cannot find cell U12
    4 warning  WARN: latch inferred at reg_q
    5 error    Fatal: timing not met
clean.log                 errors:0    warnings:0
17:02:12 INFO  total errors: 2
exit=1

```

Real output. Note the separation: the report is on **stdout** and can be redirected or piped; the log line is on **stderr**; and the **exit** status is 1, so a flow script can act on it.

38.7 What each chapter contributed

Element	Chapter
Braced regular expressions, <code>\y</code> instead of <code>\b</code>	10
<code>dict</code> for counts and options, <code>dict incr</code> for accumulation	8
<code>lappend</code> and <code>lassign</code> for the hit records	7
Namespace, <code>variable</code> , <code>namespace ensemble create</code>	14
<code>package provide</code> / <code>pkgIndex.tcl</code> / <code>auto_path</code>	15
<code>try/finally</code> around the open file	13, 21
<code>switch -glob --</code> option parsing, <code>format</code> output	5, 6
Logging to <code>stderr</code> , meaningful exit status, <code>argv0</code> guard	35
tcltest suite that caught the real bug	29

38.8 Where to take it next

- Add `-format json` output for a CI dashboard, using Tcllib's `json` package (Chapter 15).
- Make the patterns configurable from a file, parsed with the `parse_config` recipe (Chapter 37).
- Process large files line by line rather than with `read`, so memory stays constant (Chapter 21).
- Scan several files concurrently with a thread pool (Chapter 31) or by running the scanner as subprocesses under the event loop (Chapters 23-24).
- Wrap it in an ensemble of subcommands - `logmine scan`, `logmine watch` - and add a `watch` mode using `fileevent` on a growing log.
- Package it as a stargit so colleagues can run it without installing anything (Chapter 36).

IF YOU TAKE ONE HABIT FROM THIS BOOK

Write the four pure procedures **before** the program that calls them, and write their tests at the same time. The logic then has no dependency on files, options or a tool shell; it runs in a second on any machine; and when something breaks at 2 a.m. before a deadline, the failing test tells you which line to look at. Everything else in these thirty-eight chapters is detail by comparison.

Exercises

- 1 Build this project from the listings and run its tests.
- 2 Break one pattern deliberately and confirm which tests fail.
- 3 Add a `-format csv` option, with a test for it.
- 4 Add a `-max-errors N` option that exits 2 when exceeded, and test the exit status from a shell.
- 5 Take a script you already maintain and restructure it this way: pure package, thin program, test suite. Record how long it took and what you found.

APPENDICES

Appendices

A command reference organised by task, a glossary of every term used in this book, and a study roadmap with projects and resources.

Appendix A. Command Reference by Task

The eleven rules, condensed

- 1 A script is commands separated by newlines or semicolons.
- 2 A command is words; the first names the command.
- 3 Whitespace separates words unless quoted or braced.
- 4 "double quotes" group, WITH substitution.
- 5 {braces} group, with NO substitution.
- 6 [brackets] run a script and are replaced by its result.
- 7 \$name, \$name(idx), \${name} substitute a variable.
- 8 \ escapes: \n \t \\$ \[\ and \<newline>.
- 9 # is a comment ONLY where a command could begin.
- 10 Each word is substituted ONCE, left to right.
- 11 Substitution never creates new word boundaries.

Variables and scope

Task	Command
Assign / read	<code>set v value / \$v</code> or <code>[set v]</code>
Remove	<code>unset ?-nocomplain? v</code>
Exists?	<code>info exists v</code>
Increment	<code>incr v ?n?</code>
Append string / list item	<code>append v str / lappend v item</code>
Reach the global scope	<code>global v</code> or <code>::\$v</code>
Reach a namespace variable	<code>variable v</code>
Reach the caller's variable	<code>upvar 1 name local</code>
Run a script in the caller	<code>uplevel 1 \$script</code>

Strings

Task	Command
Length / index / substring	<code>string length / string index / string range</code>
Search	<code>string first, string last</code> (-1 when absent)
Replace many at once	<code>string map {from to ...} s</code>
Glob match / compare	<code>string match pat s / string equal ?-nocase?</code>
Trim	<code>string trim trimleft trimright s ?chars?</code> (a set of characters)
Classify	<code>string is class -strict s</code>
Case	<code>string tolower toupper totitle</code>
Format / parse	<code>format spec args / scan s spec vars</code>

Task	Command
Split / join	<code>split s ?chars? / join list ?sep?</code>
Substitute in a value	<code>subst ?-nocommands? ?-novariables? s</code>

Lists

Task	Command
Build	<code>list a b c, lrepeat n v</code>
Length / element / range	<code>llength / lindex l i ?i...? / lrange l i j</code>
Append / insert / replace	<code>lappend var ... / linsert l i ... / lreplace l i j ...</code>
Assign in place	<code>lset var i ?i...? value</code>
Destructure	<code>lassign l a b c</code> (returns the remainder)
Search	<code>lsearch ?-exact -glob -regexp? ?-all? ?-inline? ?-index n? l pat</code> - glob is the default
Sort	<code>lsort ?-integer -real -dictionary -nocase? ?-unique? ?-index n? ?-command p? ?-decreasing?</code>
Transform	<code>lmap v \$l {...}</code> (8.6+)
Flatten / expand	<code>concat / {*} \$list</code>
Reverse	<code>lreverse</code>

Dicts and arrays

Task	dict	array
Create	<code>dict create k v ...</code>	<code>array set a {k v ...}</code>
Read	<code>dict get \$d k ?k...?</code>	<code>\$a(k)</code>
Write	<code>dict set var k ?k...? v</code>	<code>set a(k) v</code>
Exists	<code>dict exists \$d k</code>	<code>info exists a(k)</code>
Remove	<code>dict unset var k</code>	<code>unset a(k)</code>
Keys / size	<code>dict keys \$d / dict size \$d</code>	<code>array names a ?pat? / array size a</code>
Iterate	<code>dict for {k v} \$d {...}</code>	<code>foreach {k v} [array get a] {...}</code>
Accumulate	<code>dict incr append lappend var k ?v?</code>	-
Combine / select	<code>dict merge, dict filter</code>	-
Convert	<code>array get a</code> is a dict	<code>array set a \$dict</code>

Control, procedures and errors

Task	Command
Conditional	<code>if {cond} {...} elseif {cond} {...} else {...}</code> - braces on the same line
Loops	<code>while, for, foreach ?vars list...?, lmap</code>
Dispatch	<code>switch ?-exact -glob -regexp? -- \$v {pat {body} ...}</code> - exact is the default
Define / delete a command	<code>proc name {args} body / rename name {}</code>

Task	Command
Anonymous	<code>apply {{args} body} ...</code>
Return a code	<code>return -code error break continue ?value?</code>
Catch	<code>catch {script} ?msgVar? ?optsVar?</code>
Structured handling	<code>try {...} trap {CODE} {m o} {...} on error {m} {...} finally {...}</code>
Raise	<code>throw {CLASS DETAIL} message or error msg ?info? ?code?</code>
Diagnostics	<code>\$::errorInfo, \$::errorCode, info frame</code>

Files, channels and processes

Task	Command
Open / close	<code>open path ?mode? ?perms? /close \$ch</code>
Read	<code>read \$ch ?n?, gets \$ch ?var? (>= 0 while data remains)</code>
Write	<code>puts ?-nonewline? \$ch s, flush \$ch</code>
Configure	<code>fconfigure \$ch -buffering -translation -encoding -blocking</code>
Position	<code>seek, tell, eof</code>
Paths	<code>file join dirname tail rootname extension normalize</code>
Metadata / manipulate	<code>file exists size mtime type / file mkdir delete copy rename</code>
Find	<code>glob -nocomplain ?-directory d? ?-types f? pat</code>
Run a program	<code>exec ?options? cmd args (errors on non-zero and on stderr output)</code>
Pipe as a channel	<code>open " cmd" r+; close \$ch w half-closes</code>
Compression	<code>zlib push gzip gunzip \$ch</code>

Events, networking and concurrency

Task	Command
Timer	<code>after ms ?script?, after cancel id, after info</code>
Channel readiness	<code>fileevent \$ch readable writable script</code>
Run the loop	<code>vwait varName, update ?idletasks?</code>
Sockets	<code>socket ?-async? host port, socket -server cmd port</code>
HTTP	<code>package require http; http::geturl url -timeout ms ?-command cb?; always `http::cleanup`</code>
Coroutines	<code>coroutine name cmd args, yield, yieldto, info coroutine</code>
Threads	<code>package require Thread; thread::create send release, tpool::*, tsv::*</code>
Sandbox	<code>interp create -safe, \$i alias, interp limit, \$i hidden</code>

Regular expressions

```

regexp ?-all? ?-inline? ?-line? ?-nocase? ?-indices? ?-expanded?
    ?--? {pattern} string ?matchVar? ?subVar ...?
regsub ?-all? ?-nocase? ?--? {pattern} string {replacement} ?var?

. any      [abc] set      [^abc] negated    a|b alternation
\d \w \s   digit word space      \D \W \S   their complements
* + ? {n,m} quantifiers; add ? for non-greedy
^ $        start/end (of each LINE with -line)
\m \M \y   start of word / end of word / any word boundary
            (NOT \b - in Tcl that means backspace)
(...) group      (?:...) non-capturing      (?i) case-insensitive
***=text        treat the rest as a LITERAL string
Replacement: \1..\9 groups, & whole match

```

Introspection and measurement

Question	Command
What commands exist?	<code>info commands ?pat?, info procs</code>
What is this procedure?	<code>info args body default p</code>
Where am I?	<code>info level ?n?, info frame ?n?, namespace current</code>
Where does this name resolve?	<code>namespace which name</code>
What version?	<code>info patchlevel, package require, package vsatisfies</code>
How long does it take?	<code>time {script} ?count?</code>
What type is this value?	<code>tcl::unsupported::representation \$v</code>
What bytecode?	<code>tcl::unsupported::disassemble proc name</code>
Watch a variable / command	<code>trace add variable execution ...</code>

Appendix B. Glossary

Term	Definition
Alias	A command in one interpreter that runs a procedure in another - the controlled way to give a safe interpreter a capability.
ARE	Advanced Regular Expression - Tcl's regexp flavour, closer to Perl than to POSIX, with <code>\y</code> rather than <code>\b</code> for word boundaries.
Array	A collection of variables sharing a name, indexed by string. Not a value: it cannot be passed or returned, only named.
Bytecode	The compiled form of a script, produced on first execution; the reason braced expressions and bodies are fast.
Channel	The uniform interface to a stream - file, pipe, socket or transform - configured with <code>fconfigure</code> .
Coroutine	A procedure that can suspend with <code>yield</code> and be resumed later, keeping its frame; Tcl's generator and cooperative concurrency mechanism.
Dict	An ordered key/value mapping that is also a valid list; the default structured container since 8.5.
Dodekalogue	The eleven-rule summary of Tcl syntax in the <code>Tcl</code> manual page - the whole language's grammar.
EIAS	'Everything Is A String': every value has a string representation, and commands decide how to interpret it.
Ensemble	A command whose first argument selects a subcommand, built from a namespace with <code>namespace ensemble create</code> - how <code>string</code> , <code>dict</code> and <code>file</code> work.
errorCode	A machine-readable list describing an error's class; branch on this, never on the message text.
Event loop	The single-threaded dispatcher that runs timer, channel, socket and GUI callbacks to completion, one at a time.
Expect	The extension that drives interactive programs through a pseudo-terminal with <code>spawn</code> , <code>expect</code> and <code>send</code> .
fileevent	A callback registered for channel readability or writability - the basis of non-blocking I/O.
Interpreter	A complete, independent Tcl execution context; programs may create children, including safe ones.
K combinator	<code>proc K {a b} {return \$a}</code> - returns its first argument while evaluating the second, historically used to avoid copying a shared value.
Namespace	A hierarchical container for commands and variables; the unit of modularity below a package.
Package	A named, versioned unit of code loaded by <code>package require</code> through a <code>pkgIndex.tcl</code> or a <code>.tm</code> module.
Safe interpreter	A child interpreter with the dangerous commands hidden, capable of running untrusted script.
Shimmering	Repeated conversion of a value between internal representations - the main cause of unexpectedly slow Tcl.
Starkit / Starpack	A single-file distribution of a Tcl application; a starpack also bundles the interpreter.
Substitution	The one-pass replacement of <code>\$var</code> , <code>[command]</code> and <code>\escape</code> performed on each word before a command runs.
Tcl_Obj	The C structure holding a value's string representation and its cached internal representation.
Tcllib	The community standard library: json, csv, struct, cmdline, fileutil, math and much more.
TclOO	The object system built into Tcl 8.6: classes, methods, mixins and filters, where objects are commands.
tcltest	The test framework shipped with Tcl.

Term	Definition
Thread-shared variable (tsv)	The explicitly shared store used to pass data between Tcl threads, which otherwise share nothing.
Trace	A script fired when a variable is read, written or unset, or when a command is entered or left.
upvar / uplevel	Link a caller's variable into this scope / evaluate a script in the caller's scope - Tcl's call-by-reference and its macro mechanism.
vwait	Enter the event loop until a named variable is written.
yield	Suspend a coroutine, returning a value to whoever resumed it.
{*} (expansion)	Splices a list into the words of the command being built (8.5+); the safe replacement for most uses of <code>eval</code> .

Appendix C. Roadmap, Projects and Resources

A 10-week study plan

Week	Read	Build
1	Chapters 1-5	A script that renames files by pattern; type every example in <code>tclsh</code>
2	Chapters 6-7	A log filter using <code>string</code> and lists, with <code>-dictionary</code> sorting
3	Chapters 8-10	A key/value config parser and a report parser with regular expressions
4	Chapters 11-13	Refactor both into procedures with proper error handling
5	Chapters 14-15	Turn them into a package with an ensemble API
6	Chapters 21-23	A tool wrapper that runs a program, captures output and reports its exit status correctly
7	Chapters 24-25	An event-driven monitor that watches a growing log file
8	Chapters 29-30	A <code>tcldtest</code> suite for everything so far; profile and fix the slowest part
9	Chapters 16-20	Rewrite one module with <code>TclOO</code> ; add a coroutine-based generator
10	Chapters 34-38	Build the Chapter 38 project, then restructure a real script of your own the same way

Projects worth building

- **A log analyser** (Chapter 38) - the canonical first tool.
- **A report differ**: parse two tool reports and print what changed, sorted by significance.
- **A flow driver**: run three programs in sequence with logging, timing and correct exit statuses.
- **A watcher**: `fileevent` on a growing file, printing new matching lines as they appear (Chapter 24).
- **An Expect wrapper** for a tool with no batch mode (Chapter 26).
- **A Tk front end** for a script you run constantly (Chapter 27).
- **A C extension** that speeds up the one function profiling says matters (Chapter 33).
- **A plug-in host**: run user-written scripts in safe interpreters with three aliases (Chapter 32).

Where to look things up

Resource	For
<code>man n <command></code> , or the manual pages at <code>tcl.tk</code>	The reference. Tcl's manual pages are precise and complete; <code>man n regexp</code> and <code>man n re_syntax</code> in particular
The Tcl manual page (<code>man n Tcl</code>)	The eleven rules
The Tcler's Wiki (<code>wiki.tcl-lang.org</code>)	Idioms, recipes and long-running discussions of every corner
Tcllib and Tklib documentation	What already exists before you write it
TIPs (Tcl Improvement Proposals)	Why a feature works the way it does, and what is coming
<code>comp.lang.tcl</code> archives and the Tcl'ers mailing list	Deep history and expert answers
Your tool's own command reference	In EDA work this is most of your day-to-day lookup (Chapter 28)

How to keep improving

- **Read the manual page for a command you already use.** Almost every one has options you did not know about.

- **Type the surprising examples.** The behaviours in Chapters 2, 3 and 10 stick when you have seen them, not when you have read them.
- **Write tests for the next bug you fix**, and keep them.
- **Measure before optimising** - `time` settles arguments in seconds (Chapter 30).
- **Read other people's Tcl**, especially the Tcllib sources: they are compact, idiomatic and heavily reviewed.